Linnet

Half-edge graphs and subgraph-first algorithms

---

1 Overview

Linnet is a graph library for tensor networks, Feynman diagrams, and other workflows where the boundary of a subgraph matters. Its central representation is a half-edge graph. An ordinary internal edge is a pair of half-edges, while an external edge has one dangling half-edge. A node owns incident half-edges, and a subgraph is a selection of half-edges rather than a detached copy of the graph.

> **Why half-edges**
> Cutting an internal edge can turn its two halves into boundary edges without changing the degree of either endpoint. This makes operations such as extraction, excision, contraction, sewing, traversal, and cut analysis natural for physics graphs. Linnet algorithms therefore accept subgraph views even when the caller wants to operate on the full graph.

1.1 Choose a task

- To add Linnet to a Rust project and validate a graph with a boundary, follow the Rust guide and the exact `HedgeGraphBuilder` reference.
- To understand DOT endpoints and their numbering, read DOT input and graph indices.
- To parse and inspect a DOT graph from Python, follow the source-built Python guide.
- To select an internal subgraph and compute a cycle basis, continue with the first-graph tutorial.
- To enumerate cycles, cuts, or connected regions, use the graph-algorithms guide with the `HedgeGraph` reference.
- To render a tree of existing DOT files and rebuild only changed figures, use the Clinnet command-line guide.
- To draw an editable graph in Typst, begin with the Typst guide, then continue to the Linnest Typst guide layout coordinates are deliberately separate from graph identity.

1.2 A working model

A Linnet workflow has four parts:

- construct a graph with `HedgeGraphBuilder`, or parse a DOT graph;
- retain the typed identifiers returned for half-edges, nodes, and edges;
- describe the region of interest with one of the subgraph representations;
- run traversal or mutation operations against that subgraph and the graph's involution.

The involution is the pairing map for half-edges. A paired entry represents an internal edge; an identity entry represents a dangling edge and records its flow. Edge orientation and flow are related concepts, but they are not interchangeable. Consult the API reference for the exact variants and conversion rules in your Linnet release.

> **Indexes belong to one graph**
> `Hedge`, `NodeIndex`, and `EdgeIndex` are compact typed indexes, not durable object IDs. Graph edits can change storage and mappings. Keep the result returned by an operation, and do not apply an index or subgraph bitset to an unrelated graph just because its numeric values happen to fit.

1.3 Construction and interchange

The Rust package can be added directly from crates.io:

```toml
[dependencies]
linnet = "0.17"
```

Programmatic construction uses `HedgeGraphBuilder`. DOT support provides a convenient human-readable interchange and debugging format; it can carry global, node, edge, and half-edge attributes. Treat DOT as a representation boundary, not as a substitute for the typed graph invariants. Validate or handle parser errors before running algorithms.

The `drawing` feature adds layout support. Layout is separate from graph identity: coordinates may change without changing the topology. Linnet itself does not provide a supported renderer; use DOT, Clinnet, or Linnest at the presentation boundary.

1.4 Related crates

One graph layer, several consumers
Spenso uses Linnet for tensor-network connectivity and adds tensor contraction semantics. GammaLoop uses the graph model in collider workflows and adds process generation, integration, and persistent state.

The rendered guides, native Rustdoc, and generated Python API are the canonical references. Contributors can follow the stores, invariants, mutation flow, and feature boundaries in the Linnet implementation architecture.

2 Build your first Linnet graph

Linnet's half-edge graph model is available through three surfaces. Choose the language that owns the graph in your workflow; the same concepts—paired internal half-edges, explicit boundaries, typed identifiers, and validated topology—carry across them.

Rust
Use the published `linnet` crate when a Rust application owns graph construction, algorithms, or serialization. This is the most complete and thoroughly tested interface.

Use Linnet from Rust →

Python
Use `linnet_py` for DOT-backed graph inspection and algorithms from Python. The binding is currently a source-built developer preview rather than a published wheel.

Use Linnet from Python →

Typst
Use Linnest when a Typst document owns graph construction, layout, and the final CeTZ drawing. The package is currently bundled with the repository rather than Typst Universe.

Use Linnet from Typst →

Begin with Rust for application code, Python for an existing Python graph workflow, or Typst when the desired result is an editable figure. The interface guide explains where layout, rendering, and interchange belong.

3 Using Linnet from Rust

Use Linnet when a problem is naturally expressed as a half-edge graph with an explicit boundary. This example builds the smallest such graph, validates its involution, and emits DOT that can be inspected or handed to a renderer.

3.1 Create a Rust project

```
cargo new linnet-quickstart
cd linnet-quickstart
cargo add linnet@0.17
```

Replace `src/main.rs` with:

```
use linnet::half_edge::{
    builder::HedgeGraphBuilder, involution::Flow, HedgeGraph,
};

fn main() {
    let mut builder = HedgeGraphBuilder::new();
    let source = builder.add_node("source");
    let sink = builder.add_node("sink");

    builder.add_edge(source, sink, "propagator", false);
    builder.add_external_edge(source, "incoming", false, Flow::Sink);
    builder.add_external_edge(sink, "outgoing", false, Flow::Source);

    let graph: HedgeGraph<&str, &str> = builder.build();
    graph.check().expect("the half-edge involution is valid");

    assert_eq!(graph.n_externals(), 2);
    println!("{}", graph.dot_label(&graph.full_filter()));
}
```

Run it with `cargo run`. The output is a DOT graph with one paired internal edge and two dangling boundary edges. `graph.check()` is the important first invariant: run it immediately after constructing or parsing a graph, before applying graph algorithms.

> **Half-edges make the boundary explicit**
> An internal edge is a pair under the graph's involution. A dangling external edge maps to itself and carries a `Flow`. The `NodeIndex` values returned by the builder belong to this graph; do not reuse their numeric representation as a durable identifier.

Continue with the graph tutorial to select a subgraph and compute a cycle basis. Use the Clinnet guide to render existing DOT files, or the Typst guide to construct and draw an editable graph with Linnest.

`linnet` is a library, so add it to a project with `cargo add`; `cargo install` and Cargo Binstall apply to executables. The separately versioned Clinnet renderer can be installed with `cargo install clinnet --locked`.

4 Using Linnet from Python

The `linnet_py` extension provides native Linnet graphs to Python 3.10 and newer. It is a real binding with runtime tests, but it is not currently published to PyPI.

> **Developer preview: build from source**
> There is no supported `pip install linnet-py` release yet. The similarly named `linnet` project on PyPI is unrelated. Use this page from a GammaLoop checkout until official `linnet-py` wheels are published.

4.1 Build the extension

Install a stable Rust toolchain and Python 3.10 or newer, then run:

```
git clone https://github.com/alphal00p/gammaloop.git
cd gammaloop
python3.10 -m venv .venv
. .venv/bin/activate
python -m pip install "maturin>=1.7,<2.0"
maturin develop --locked \
  --manifest-path crates/linnet-py/Cargo.toml \
  --features extension-module,abi3
```

The declared Python dependencies install `typst` 0.15.0 alongside the extension. Python rendering uses that in-process binding; a `typst` command in `PATH` is neither required nor invoked.

4.2 Parse and inspect a graph

Save this as `linnet_quickstart.py`:

```
import linnet_py as lp

codec = lp.DotCodec.topology()
graph = lp.Graph.from_dot(
    """
    digraph G {
      A;
      B;
      A -> B;
    }
    """,
    codec,
)

whole = graph.full_subgraph()
assert graph.n_nodes == 2
assert graph.n_edges == 1
assert len(graph.nodes_of(whole)) == 2
assert len(graph.edges_of(whole)) == 1

print(graph.to_dot())
```

Run `python linnet_quickstart.py`. Success means the assertions pass and the graph is printed as DOT. `Graph` also exposes typed nodes, half-edges, subgraphs, cycles, oriented cuts, and traversal trees; use the Python reference for the exact current surface.

Continue with Rendering graphs from Python to explore a complete graph with custom drawing selectors. The live Python playground also lets you edit DOT and tune the layout while watching the solver run.

Use the Rust guide when you need Linnet's complete crate API, or the Typst guide when the final result should be a drawing.

5 Using Linnet from Typst

Linnest brings Linnet's graph model, layout algorithms, and generic drawing styles into Typst through a bundled WebAssembly plugin. This first document draws two nodes joined by one internal edge.

> **Bundled source package**
> Linnest `0.1.0` is not currently published on Typst Universe. Keep the repository's complete `crates/linnest/`

> `typst/` directory together: its Typst sources load the adjacent `linnest.wasm` plugin. Typst `0.15.0` or newer is required.

5.1 Create the document

From a GammaLoop checkout, save this as `linnest-quickstart.typ` in the repository root:

```
#import "crates/linnest/typst/src/lib.typ": draw, graph, layout

#let g = graph.build({
  graph.node(<left>, label: [left])
  graph.node(<right>, label: [right])
  graph.edge(
    graph.source(<left>),
    <propagator>,
    graph.sink(<right>),
    label: [$p$],
  )
})

#draw(layout(g))
```

Compile it from the same repository root:

```
typst compile --root . linnest-quickstart.typ linnest-quickstart.pdf
```

The PDF should contain two labeled nodes and one labeled edge. Construction, layout, and drawing are separate stages: keep the graph value when you need queries or subgraphs before rendering it.

Continue with the Linnest guide for DOT parsing, explicit styling, and subgraph drawing, then use the Typst API reference for graph, layout, drawing, subgraph, and application-template composition.

6 Tutorial

In this tutorial, you will construct a half-edge graph in Rust, validate its invariants, find its one independent cycle inside an explicitly selected subgraph, and print DOT. The builder returns typed node indexes; Linnet's subgraph filters then select the half-edges on which an algorithm operates.

For the shortest installation and first-run path, begin with Using Linnet from Rust. Then return here to work with a boundary, subgraphs, and cycle algorithms.

6.1 Prerequisites

Create a Rust binary project and add the current Linnet release:

```
cargo new linnet-first-graph
cd linnet-first-graph
cargo add linnet@0.17
```

No drawing feature or external layout program is needed to construct the graph and emit DOT.

6.2 Build a graph with a boundary

Replace `src/main.rs` with the following program:

```rust
use linnet::half_edge::{
    builder::HedgeGraphBuilder,
    involution::Flow,
    subgraph::{SubSetOps, SuBitGraph},
    HedgeGraph,
};

fn main() {
    let mut builder = HedgeGraphBuilder::new();
    let a = builder.add_node("a");
    let b = builder.add_node("b");
    let c = builder.add_node("c");

    builder.add_edge(a, b, "ab", false);
    builder.add_edge(b, c, "bc", false);
    builder.add_edge(c, a, "ca", false);
    builder.add_external_edge(a, "incoming", false, Flow::Sink);
    builder.add_external_edge(c, "outgoing", false, Flow::Source);

    let graph: HedgeGraph<&str, &str> = builder.build();
    graph.check().expect("the half-edge involution is valid");

    let boundary: SuBitGraph = graph.external_filter();
    let internal = graph.full_filter().subtract(&boundary);
    let (basis, _) = graph.cycle_basis_of(&internal);
    assert_eq!(basis.len(), 1);
    println!("{}", graph.base_dot());
}
```

Run it with `cargo run`. Success means the assertion passes and stdout contains a DOT `digraph` with nodes `a`, `b`, and `c`, three paired internal edges, and two dangling boundary edges. The external edges do not add a cycle, so the triangle's cycle-basis rank remains one.

> **Verification scope and cost**
> The docs harness compiles and runs this Rust program; it syntax-checks the setup commands without creating a project or using the network. Success requires `graph.check()` and the subgraph cycle-rank assertion to pass. The run is small; a clean external Cargo build is dependency-dominated and can take minutes, while the graph operation itself should finish in well under a second.

> **Keep the returned indexes with their graph**
> `a`, `b`, and `c` are compact `NodeIndex` values owned by this graph construction. Do not serialize their numeric representation as a durable identity or reuse them against another graph. Mutating operations return the mappings needed to relate old and new storage.

6.3 Make the first useful variation

Add a fourth internal edge between `a` and `b`, run the program again, and change the assertion to `basis.len() == 2`. This demonstrates the key distinction: parallel paired half-edges add an independent cycle, while dangling identity half-edges describe the graph boundary excluded by `internal`.

From here, use a subgraph view to run traversal, cuts, or contraction against only the region you intend to modify. Enable Linnet's `drawing` feature only when you need layout rather than plain DOT interchange.

6.4 Troubleshooting and next steps

- A type-inference error around `build()` means Rust cannot choose node storage or data types; keep the explicit `HedgeGraph<&str, &str>` annotation from the example.

- A failed `check()` points to an involution or storage invariant. Validate immediately after parsing or construction, before applying graph algorithms.
- If an algorithm includes an unexpected dangling edge, check that you passed `internal` rather than `graph.full_filter()`, then inspect the `Flow` assigned to each external edge.
- Continue with the subgraph-and-algorithms manual, then consult the Rust API reference for the exact return mappings of mutating operations and additional builder methods.

7 Subgraphs, traversal, cuts, and drawing

Linnet algorithms operate on a graph together with an explicit subgraph view. This lets one algorithm work on the full graph, a selected region, or the boundary created by cutting paired half-edges. Choose the subgraph representation by operation: bit-backed sets are efficient for repeated set algebra, while borrowed or mapped views avoid copying when identity must remain attached to the owning graph.

7.1 Traversal and connectivity

A traversal needs a starting half-edge or node, the active subgraph, and a policy describing which adjacent half-edges may be crossed. Results retain typed indexes into the original graph. Breadth-first traversal is useful for distance layers and connectivity; depth-first traversal is useful for trees, back edges, and decompositions. A traversal tree is a derived view, not a new graph, so mutations must be coordinated with the indexes it contains.

Connectivity and traversal results depend on the chosen subgraph. A traversal tree retains its parent relation, so callers can inspect immediate children, strict ancestors, and the fundamental cycle closed by a non-tree internal edge without reconstructing an adjacency model. Boundary queries distinguish half-edges already selected at a split from every boundary half-edge incident to a touched node; dangling half-edges are available as a separate structural selection.

Linnet has two independent notions of direction. `DirectionBasis::Underlying` follows the source/sink roles stored by the involution. This is the default in Python and is the basis used by GammaLoop momentum routing. `DirectionBasis::Superficial` additionally applies an edge's `Default`, `Reversed`, or `Undirected` orientation; a superficially undirected edge contributes no directed arc. Reachability, topological order, transitive closure, and transitive reduction take this choice explicitly. Neither mode rewrites stored flow or orientation.

Directed algorithms ignore dangling half-edges and edges split by the active subgraph boundary, because neither represents a complete arc inside that selection. Topological ties use node-index order for the initial ready set and deterministic edge-discovery order thereafter. Linnet does not currently expose a dedicated articulation-point or biconnected-component API; derive those from traversal data only when your application owns that algorithm and its tests.

7.2 Cycles and cuts

`cycle_basis` returns an independent cycle vector together with the spanning forest used to construct it. Use `cyclotomatic_number` when the numerical cycle rank itself is the result. `all_cycles` expands a basis and can grow exponentially; prefer a basis when assigning loop momenta. Native callers supply a maximum accepted cycle rank. Python callers supply `max_results`, which bounds the complete non-empty basis-combination search space rather than only the circuits that survive filtering. Both guards are checked before powerset construction. Symmetric differences combine cycle bitsets without inventing new graph identities.

Cut enumeration separates required left and right node sets and returns the left region, cut content, and right region. A cut edge is represented through its half-edges, preserving which side owns each endpoint. Validate direction/flow after enumeration when a physical Cutkosky or tensor-network interpretation depends on orientation.

> **Complexity is part of the API choice**
> Enumerating every cycle or every admissible cut is inherently combinatorial. Filter the subgraph and endpoint sets first, and use basis or connectivity operations when enumeration is not required by the caller.

7.3 Enumerate separating cuts

This complete example builds a square with a diagonal and asks for every cut separating opposite nodes. It is the smallest graph that makes the distinction between a path and a cut family visible without external data or drawing support.

```
use linnet::half_edge::{HedgeGraph, builder::HedgeGraphBuilder};

fn main() {
    let mut builder = HedgeGraphBuilder::new();
    let a = builder.add_node(());
    let b = builder.add_node(());
    let c = builder.add_node(());
    let d = builder.add_node(());

    builder.add_edge(a, b, (), true);
    builder.add_edge(b, c, (), true);
    builder.add_edge(c, d, (), true);
    builder.add_edge(d, a, (), true);
    builder.add_edge(b, d, (), true);

    let graph: HedgeGraph<(), (), ()> = builder.build();
    graph.check().expect("the half-edge involution is valid");
    let cuts = graph.all_cuts_from_ids(&[a], &[c]);

    assert_eq!(cuts.len(), 4);
    println!("separating cuts: {}", cuts.len());
}
```

The expected invariant is four distinct left/cut/right partitions separating `a` from `c`. Changing topology can change that count, while changing only node or edge payloads does not. The native `HedgeGraph` Rustdoc and builder Rustdoc give the exact compiled type boundaries for this revision.

The Python binding delegates to that same native enumeration and returns named graph-bound results rather than nested tuples:

```
partitions = graph.all_cuts(["a"], ["c"])
for partition in partitions:
    source_side = partition.source_side
    boundary = partition.boundary
    cut_edges = partition.edges
    target_side = partition.target_side
```

The source and target sides are structural `Subgraph` views. `boundary.left` and `boundary.right` select the two oriented half-edges of every cut edge. The result is eagerly materialized and sorted by half-edge membership at the Python boundary; do not use all-cut enumeration where one witness or a minimum-cardinality cut is the actual question.

`all_bonds(min_size=..., max_size=...)` is the related native enumeration of inclusion-minimal cutsets whose two induced sides are connected. A bond size is its boundary half-edge count; it is not a max-flow result. `find_bond` short-circuits after one witness in the same inclusive size range. Both full enumerations can be combinatorial.

`tadpoles(externals)` identifies a validated terminal set structurally and returns the native tadpole components in stable order. Terminals must resolve to distinct structural nodes; empty or duplicate sets, out-of-range nodes, and zero-crown terminals are errors instead of panic paths.

> **Interpret a cut mismatch structurally**
> A failed `graph.check()` indicates a storage or involution invariant and must be resolved before enumeration. Overlapping, empty, isolated, or closed terminal groups are rejected. For validated groups, zero cuts means

> the native enumerator found no admissible separating partition, commonly because the groups occupy disconnected components. An unexpected nonzero count means the constructed edges or endpoint sets do not describe the graph you intended; inspect those before adding post-enumeration filters.

7.4 Mutation boundaries

Insertion extends a graph with the same declarative node and edge specs accepted by the builder. `split_edge` separates one paired edge into two dangling edges, while `connect` joins two exact dangling half-edges. Extraction copies a selected region, excision separates a graph along its boundary, sewing joins compatible dangling half-edges, and contraction identifies structure. These operations return replacement views or indexes where needed. Keep only the topology-independent payloads across an edit: graph-bound Python views are revision checked and become stale after every successful topology change.

Transitive reduction is an owning transformation over a directed acyclic graph. The Python method returns a new graph, preserves arbitrary payload identity and copied drawing configuration, and leaves views into the source graph live. Transitive closure remains a Rust-only operation until the Python API has an explicit factory for every new edge and half-edge value it synthesizes.

7.5 DOT and drawing

DOT parsing and emission are interchange/debugging boundaries. Attributes can carry graph, node, edge, and half-edge data, but the typed Rust invariants are reconstructed by the parser; handle parser errors before running algorithms. The `drawing` feature enables layout; rendering belongs to a DOT consumer, Clinnet, or Linnest. Layout coordinates are presentation data and may vary without changing topology, subgraph membership, or edge flow.

Tensor contraction uses Spenso
Linnet determines connectivity and graph transformations. Tensor compatibility, contraction cost, and execution belong to Spenso even when a Spenso network delegates its graph algorithms to Linnet.

8 DOT input and graph indices

Linnet reads DOT into a graph of nodes, edges, and half-edges. This guide explains how DOT endpoints become graph records and how their indices are assigned. The same rules apply when DOT enters through Python, Linnest, or GammaLoop; a consumer can subsequently transform or reorder the parsed graph.

8.1 Edges and half-edges

An internal edge has two half-edges, one attached to each endpoint. A dangling edge has one half-edge attached to its real node. Represent its missing endpoint with an invisible DOT node:

```
digraph example {
  ext [style=invis];
  ext -> a;
  a -> b;
  b -> ext;
}
```

This graph has two nodes, three edges, and four half-edges. The invisible `ext` is not a graph node. `ext -> a` contributes one incoming half-edge at `a`; `b -> ext` contributes one outgoing half-edge at `b`; `a -> b` contributes one half-edge at each node. A self-loop also contributes two half-edges, both attached to the same node. Connecting two invisible endpoints is invalid.

One invisible node can represent several dangling endpoints: its incident edges remain separate edges, not a connection through `ext`. Keep shared invisible nodes otherwise attribute-free, since their attributes are merged into each attached dangling edge.

8.2 Indices, not arbitrary labels

An edge attribute `id=2` requests edge index 2. A numeric endpoint port such as `a:3` requests half-edge index 3 at `a`. These are positions in separate, densely indexed graph collections:

Record	DOT spelling	Allowed indices
Node	`a [id=0]`	From `0` through `V - 1`, where `V` is the number of visible nodes.
Edge	`a -> b [id=2]`	From `0` through `E - 1`, where `E` is the total number of edges.
Half-edge	`a:3 -> b`	From `0` through `H - 1`, where `H` is the total number of half-edges.

With `I` internal edges and `D` dangling edges, `E = I + D` and `H = 2 I + D`. Count the complete parsed graph, including dangling edges. An invisible endpoint contributes no extra half-edge; put a dangling half-edge's numeric port on its real node.

Each explicit index must occur only once within its collection, across the entire graph. `a:0 -> b:0` requests the same half-edge index twice and is rejected, even though the endpoints are different nodes. Two edges with `id=0` also collide. In contrast, an edge index 0 and a half-edge index 0 can coexist: they identify different kinds of records. The node-index collection is independent of both.

Indices must be nonnegative and in range. On the three-edge, four-half-edge graph above, `id=3` and a numeric port `a:4` are out of bounds. An explicit index does not create empty records to extend the collection. Use ordinary attributes such as `name` or `label` for application names or displayed text.

Node identifiers that parse as nonnegative integers, including quoted numeric identifiers, also request that node index unless a node `id` attribute overrides it. Thus a one-node graph containing only `7;` is out of bounds; `7 [id=0];` explicitly assigns its sole node to index 0. Alphabetic identifiers such as `a` leave the index unspecified unless they have an `id`.

Write numeric half-edge ports without quotes: `a:3` or `a:3:n`. Named ports and compass points such as `a:output:n` and `a:n` do not reserve half-edge indices. Currently, quoted numeric ports such as `a:"3"` also remain port labels rather than fixing an index.

> **Numeric ports are global**
> `a:3` means half-edge 3 of the whole graph, not the fourth connection at node `a`. It neither selects an edge index nor encodes a screen coordinate. The two halves of an internal edge can have unrelated indices; do not infer them from its edge index.

8.3 Omission and inferred order

You can omit every index, supply every index, or fix only the records referenced elsewhere. The parser reserves the explicitly requested positions, then fills the remaining positions in ascending order. It handles all three collections independently, so specifying an edge's `id` does not also specify either of its half-edge indices.

The order used to fill omitted indices comes from parsing the DOT endpoints, not simply from the order of lines in the file:

- Nodes are traversed in lexicographic DOT identifier order.
- Edges are sorted by their source endpoint and then their target endpoint. An endpoint compares its DOT identifier spelling first, then any port or compass information. Ports therefore participate in sorting, even when they are not numeric indices. Equal endpoint pairs retain their order in the flattened DOT input, so statement order still matters for parallel edges with identical endpoints.
- Half-edges follow that edge traversal: an internal edge contributes its source half-edge before its sink half-edge; a dangling edge contributes only its real half-edge.

Each collection skips its explicit reservations when assigning omitted indices. The sorting uses DOT spellings, not numeric magnitude, and happens before the explicit index permutations.

For example, this input deliberately puts `b -> ext` before `a -> b`:

```
digraph indices {
  ext [style=invis];
  b -> ext;
  a:3 -> b [id=2];
  ext -> a:0;
}
```

The endpoint traversal is `a -> b`, `b -> ext`, then `ext -> a`. Edge 2 and half-edges 0 and 3 are already reserved. The resulting graph is:

Edge index	Connection	Source half-edge	Sink half-edge
`0`	`b -> ext`	`2` at `b`	None
`1`	`ext -> a`	None	`0` at `a`
`2`	`a -> b`	`3` at `a`	`1` at `b`

Changing an endpoint name, adding a port, or inserting an edge can change inferred indices. Supply the indices that an external reference depends on, and inspect or export the parsed graph before using numeric references. Graph edits after parsing can also change indices; keep any index mappings returned by the operation.

8.4 Chains, subgraphs, and repeated defaults

The parser expands edge chains and subgraph endpoints before counting and assigning indices. For example, `a -> b -> c` creates two edges. Its trailing attributes apply to both, so `a -> b -> c [id=0]` requests edge 0 twice and is rejected. Likewise, `a -> {b c} [id=0]` creates a duplicate index. A numeric port in the middle of a chain is also repeated: `a -> b:0 -> c` requests half-edge 0 on both incident edges at `b` and is rejected. Write separate edge statements if these edges or half-edges need explicit indices, or omit the indices and inspect the resulting order.

The same uniqueness rule applies when node or edge defaults supply `id` values. A subgraph does not start a new index space: all visible nodes, expanded edges, and half-edges belong to the containing graph. Parallel edges remain distinct records.

8.5 Inspect and draw

Use the Python quickstart to parse and inspect graph records, or the live DOT playground to edit a diagram in the browser. For drawing an existing DOT collection, use the Clinnet guide.

GammaLoop's DOT input guide explains which indices select external kinematics, identify numerator data, or appear in momentum labels. Amplitude kinematics follow increasing dangling half-edge order. GammaLoop uses the same order for initial drawing rows, with incoming and outgoing legs in separate columns; its cross-section rows follow numeric `is_cut` order. Its example deliberately gives edges different indices from their half-edges: changing drawing order does not renumber the edge-based q labels. Cross-section physics import can subsequently sew and renumber records, as described in that guide.

9 Live Python playground

Edit DOT below and watch the layout update as the solver runs. The notebook loads automatically when it comes into view. Open the collapsible layout controls to change the solver settings. Editing DOT or a slider restarts the layout; pause and resume keep the current positions.

The first visit downloads the browser Python runtime and notebook dependencies. Python and Linnet then run locally in your browser through WebAssembly; no Python installation or remote notebook server is needed. An internet connection is needed for the initial downloads.

Open the streaming layout notebook source to run this example locally. The online documentation embeds the interactive notebook.

9.1 Geometry and final drawings

The live SVG previews the solver's positions and edge geometry. Use Linnest or the Python rendering API for final Typst drawings with measured labels, styles, and decorations.

9.2 Try the Python API

The Python quickstart has an editable notebook containing its graph example. Edit the Python cell and use its play button to inspect the resulting graph.

The Python rendering guide provides a complete editable workflow with custom styles and arbitrary Python payloads.

The Python reference documents the full API. Its examples can depend on earlier definitions, local files, or rendering dependencies, so they are not all runnable as independent notebook cells yet.

10 Rendering graphs from Python

A Linnet graph can carry ordinary Python objects and render directly in a notebook. This guide uses a document-processing workflow to explore the default drawing, layout selection, and styles computed from application data. Begin with the Python quickstart for installation and basic graph queries.

The notebook below loads automatically when it comes into view. Python, Linnet, and the Typst renderer run in your browser through WebAssembly. The first visit downloads their dependencies; no local Python installation or notebook server is needed.

10.1 Try the rendering API

Choose a layout and enable the custom line theme or half-edge IDs. Each change rebuilds the example graph and redraws it. The cells are editable, so you can change the workflow or its selectors and run the affected cell with its play button.

The online guide includes an interactive notebook. Open the rendering notebook source to run the same example locally.

10.2 What to explore

- Switch between **Force** and **Stable layered**. Both render the same graph; force layout tunes geometric positions, while the layered layout organizes the workflow in its chosen direction.
- Enable **Custom Python line theme**. Edge selectors inspect each channel's protocol to choose its color, label, and pattern. Endpoint selectors place direction marks using source/sink flow.
- Enable **Half-edge IDs** to see how the incoming request and outgoing publication differ from internal edges. The notebook reports the node, edge, and half-edge counts below the drawings.
- Switch the node store and inspect the payload checks. Node, edge, and endpoint dataclass instances remain attached by identity, including their shared runtime object.

10.3 The rendering boundary

Returning a `Graph` from a cell invokes its SVG representation. `graph.to_svg()` explicitly produces the same kind of drawing; the notebook shows both forms. The graph's `RenderConfig` combines layout options, drawing defaults, and Python selectors. Only topology and the selectors' typed drawing results pass to Typst; application payloads stay in Python.

The optional theme demonstrates that domain conventions belong in those selectors. It does not change the workflow records or require them to be serializable. For the complete types and method signatures, use the Python API reference.

To experiment with a DOT-backed physics example, continue to GammaLoop DOT input. For a continuously updating view of the force solver itself, use the live DOT playground.

11 Render DOT figures with Clinnet

Clinnet is Linnet's command-line companion for documentation and diagram workflows. The Cargo package is named `clinnet`, while the installed executable is named `linnet`. It recursively finds DOT files, renders one PDF per graph through Typst, and assembles those PDFs into a grid. Use the Rust library for graph algorithms, the `linnet-py` `Graph` API for typed programmatic drawing, and Clinnet when the input is already DOT and the desired result is a browsable set of figures.

11.1 Install and render a directory

Clinnet requires Typst 0.15.0 or newer. Install both commands, then confirm the executable's current flags rather than relying on an older pre-subcommand invocation:

```
cargo install clinnet
cargo install typst-cli --version 0.15.1 --locked
linnet --help
linnet draw examples
```

The final command scans `examples/` recursively. With default paths it creates:

- `build/figs/`, mirroring the input directory and containing one PDF per DOT file;
- `build/grid.pdf`, the combined document;
- `build/templates/`, containing the default figure, grid, layout, Linnest, and Kurvst assets;
- `build/.cache/figures.json`, the per-figure content cache; and
- `build/.cache/run-metadata.json`, the plan used by targeted redraw commands.

> **What a second run reuses**
> A figure is reused only when its DOT source, selected figure template, explicit `--style` files, `--input` values, and the bundled `.typ`/`.wasm` assets in `build/templates/` have the same content hash. Clinnet does not recursively discover arbitrary imports beside a custom template; pass those files with `--style` when they must invalidate the cache. The grid is still regenerated from the current figure index. Removing a DOT file removes its stale figure output during the next full `draw`; deleting the figure cache forces every remaining figure to rebuild.

11.2 Configure the V1 templates

Use `--figure-template` for each individual figure and `--grid-template` for the combined PDF. Repeat `--style` for extra files whose contents must invalidate the figure cache. Repeat `--input key=value` for the CLI's recognized typed configuration overrides:

```
linnet --build-dir build draw graphs \
  --figure-template templates/figure.typ \
  --grid-template templates/grid.typ \
  --style templates/colors.typ \
  --input steps=250 \
  --input theme=dark \
  --input accent=teal \
  --input columns=4 \
  --output-path build/review.pdf
```

Clinnet parses values before constructing the native Typst configuration. Figure `steps` and `seed` become generic layout options; every other accepted figure input is staged as typed `config.options` for the selected template to interpret. In the example, `theme` and `accent` are application-defined settings rather than Clinnet modes. Grid overrides cover its rows, columns, page geometry, alignment, labels, and page numbers. Inputs are never published as `sys.inputs`.

Each figure render stages the DOT topology and an ephemeral entrypoint. That entrypoint imports the selected template, constructs `config`, adds `data-path`, and calls the mandatory `render(config)` export:

```
#import "layout.typ": layout

#let render(config) = {
  set page(width: auto, height: auto)
  context layout(config)
}
```

The Typst command receives only the entrypoint, output, and root paths, so large configurations do not produce large process argument lists. `--output-path` selects the final grid PDF; it is not a figure configuration field. Plain strings stay escaped data, and custom templates do not have a legacy top-level compilation path. The embedded defaults include the canonical Linnest and Kurvst package trees, so a custom template can use the same graph-layout and curve primitives. Continue to the Linnest Typst API for graph construction, layout, inspection, and drawing rather than copying its wrapper surface into a Clinnet template guide.

> **Generic and GammaLoop templates**
> Clinnet's bundled template is generic: it attaches typed element records, applies final labels and styles before ordered layout passes, and draws the result. GammaLoop writes a separate V1 bundle under `drawings/templates/`, including its generated `edge-style.typ`. Select that bundle's `figure.typ` when particle colors and decorations, momentum annotations, amplitude ordering, or cut-matched cross-section labels are required. Both templates implement the same `render(config)` contract, while only GammaLoop's template assigns meaning to its domain options.

`--style` records an imported file as a cache dependency; it does not inject or execute that file. The selected template must import it normally through Typst's module system. Clinnet does not recursively discover every custom import, so repeat `--style` for imports whose changes must invalidate cached figures.

> **Templates must share a Typst project root**
> Clinnet chooses the narrowest existing directory that contains the template, staged topology, and explicit renderer dependencies. A Typst import outside that tree, a missing style file, or a template moved without its dependencies fails before a PDF is produced. Keep templates and imported assets beneath one project directory and read the full Typst error printed by Clinnet.

11.3 Make a targeted rebuild

Run a complete `draw` once so that the build metadata records the input root, templates, output paths, styles, and figures. You can then rebuild one known DOT figure without rescanning the directory, or rebuild only the grid:

```
linnet --build-dir build redraw-figure graphs/box.dot --input theme=light
linnet --build-dir build redraw-grid --output-path build/review.pdf
```

`redraw-figure` starts with the inputs recorded by the full draw and applies new inputs as per-key overrides. `redraw-grid` reuses the cached figure list; if any `--input` is supplied to that command, the supplied list replaces the grid's recorded input list, so repeat every value the grid template still needs. A target not present in the preceding plan must be added with a new full `draw`.

Use `--no-parallel` for deterministic sequential diagnosis, or `--jobs N` to bound parallel figure compilation. These options affect scheduling, not cache identity or graph semantics.

11.4 Source and release boundaries

The implementation and exact Clap definitions are in the Clinnet command source. Clinnet has its own package version and version history, independent of the Rust `linnet` library. Record the Clinnet and Typst versions when reporting a rendering or cache problem.

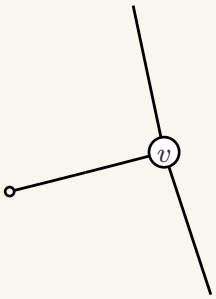
12 Build and draw with Linnest

Linnest is the Typst and WebAssembly interface to Linnet's graph model, DOT parser, layout algorithms, graph queries, and generic drawing styles. This guide follows one graph from construction through layout and drawing; the separate Typst API reference carries the supported symbol inventory.

Choose the interface deliberately
Use Linnet's Rust or Python interfaces for graph construction and algorithms in an application. Use Linnest when the graph, layout, or final drawing is owned by a Typst document.

12.1 Linnet

The `linnet` crate wrapped by this package is built around a half-edge graph data structure. This means that instead of a graph being represented as a set of nodes and edges, it is represented as a set of half-edges H, and a set of vertices V. The graph structure is then encoded through two maps. The first map, $\partial : H \longrightarrow V$ maps each half-edge to its corresponding vertex. The preimage of any vertex v is the set of half-edges that map to it, called the crown of v. The second map, $\iota : H \longrightarrow H$, is an involution that *glues* half-edges together to form edges. If a half-edge is glued to itself, we call that an external half-edge. This means that linnet graphs are strictly more capable than normal edge and vertex graphs.



Native half-edges also make subgraphs more granular because they can be encoded as sets of half-edges. This directly supports vertex-induced subgraphs: the union of the crowns of a set of vertices.

12.2 Linnest

Linnest is the Typst and WebAssembly interface to Linnet. It provides layout algorithms, DOT parsing, and selected graph algorithms without a separate runtime process.

Graphs can be constructed in two ways: parse a DOT string with `parse`:

```
#let g = parse("digraph { a -> b }")
```

or build from edges and nodes with `build`, using a Fletcher-inspired syntax:

```
#let g = build({
  node(<a>, label: [$v$])
```

```
  node(<b>)
  edge(source(<a>), <a-b>, sink(<b>), label: [e])
  edge(source(<a>), <in-a1>, label: [e])
  edge(source(<a>), <in-a2>, label: [e])
})
```

In either case, `type(g)` is `dictionary`: graph values wrap an archived Linnet graph together with native Typst data. Rust owns topology, layout state, statement metadata, and internal opaque payload bytes. User data captured from Typst stays in Typst and is merged back into query records.

The main use case is to place nodes and edges on a canvas with the `layout` function and render the result with `draw`.

- `graph` for construction, parsing, inspection, directed cuts, joins, and graph algorithms.
- `subgraph` for half-edge selection, annotations, and inspection.
- `layout` for the separate layout pass.
- `draw` for rendering a laid-out graph object with CeTZ.

Domain-specific styles are ordinary Typst data and callbacks composed with `graph.style` and `draw`; Linnest does not reserve a domain or export a physics module.

12.2.1 Choose an import path

Linnest and Kurvst are currently bundled source packages, not Typst Universe packages. A Clinnet run writes both package trees below `build/templates/`. From a custom template in that directory, import Linnest with:

```
#import "crates/linnest/typst/src/lib.typ": draw, graph, layout, subgraph
```

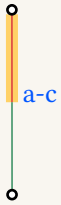
From this repository's `crates/linnest/typst/examples/` directory, the equivalent checkout-relative import is `../src/lib.typ`. Keep the package directory and its `linnest.wasm` file together when copying it elsewhere. The examples below use the checkout-relative form because they are also compiled as repository tests.

12.2.2 Minimal Build Example

```
#import "../src/lib.typ": draw, graph, layout, subgraph
#import graph: build, dot, edge, edges, node, nodes, parse, sink, source

#let g = build({
  node(<a>)
  node(<c>)
  edge(
    source(<a>, compass: "e"),
    <a-c>,
    sink(<c>, compass: "w"),
    label: [a-c],
    statements: (
      color: "0055ff",
      source-color: "d72638",
      sink-color: "1b7f4c",
    ),
  )
},
  name: "demo",
)
#let g = layout(g)
#let east = subgraph.compass(g, "e")
#let edge-records = edges(g, subgraph: east)
#let dot-text = dot(g)
#let edge-label(edge) = text(fill: rgb("#" + edge.color))[#edge.label]
```

```
#let source-style(edge) = (stroke: rgb("#" + edge.source-color) + 0.5pt)
#let sink-style(edge) = (stroke: rgb("#" + edge.sink-color) + 0.5pt)
#html.frame(draw(
  g,
  subgraph: east,
  edge-label: edge-label,
  edge-label-style: (anchor: "south"),
  source-style: source-style,
  sink-style: sink-style,
))
```



12.2.3 Graph Objects

Graph values combine archived Linnet topology with native Typst data. The focused graph reference documents their constructors, transforms, queries, and update operations.

12.3 Continue into the API

Use the graph module for construction and queries, the layout functions for placement, and the drawing functions for CeTZ output. Domain-template composition and subgraph objects have their own focused reference pages.

13 Choose a Linnet interface

Use Rust when Linnet is part of an application or library, Python for standalone graph workflows, and Linnest when a Typst document owns layout and drawing. Each interface shares the half-edge model while exposing the conventions natural to its language.

13.1 Typst package

Linnest combines Typst functions with a WebAssembly plugin. Follow the Linnest workflow guide for a first drawing, then use the focused Typst API reference for its graph, layout, drawing, subgraph, and template-composition surfaces. The curve helpers are Kurvst re-exports and stay documented in their canonical GammaLoop guide.

13.2 Rust package

The `linnet` package is organized into these principal areas:

- `half_edge` provides `HedgeGraph`, builders, the involution and edge data, subgraph types, traversal trees, and graph algorithms;
- `parser` provides DOT parsing and DOT-backed graph data;
- `half_edge::layout` provides layout helpers when the `drawing` feature is enabled;
- `permutation`, `tree`, and `union_find` provide supporting algorithms and stores.

The generic graph types separate edge, vertex, and half-edge payloads from the node-storage implementation. Start with the Rust orientation, then use the revision-specific Rustdoc for complete generic bounds, implementations, and method signatures; choose a concrete storage type according to the workflow and mutation behavior you need.

Cargo features enable optional capabilities:

- `serde` adds serialization traits;
- `bincode` adds binary encoding support;
- `drawing` enables Linnet's layout module and its optional geometry dependency;
- `symbolica` adds Symbolica interoperability.

An item shown in an all-features API reference may not be available in a default build. Check its required feature and your `Cargo.toml` before using it.

Rendering runs through Typst
Linnet can compute layout coordinates, but the supported renderer is not a native Rust drawing backend. Clinnet invokes an external Typst 0.15 executable for command-line figure batches; `linnet-py` uses the `typst` Python package to compile the same Linnest render contract in-process from its own native graph-spec preparation. Use Clinnet for batch rendering, or use Linnest when a Typst document owns the final drawing.

13.3 Standalone Python distribution

Distribution and import have different names
Install the Python distribution `linnet-py`, then import `linnet_py`. It requires Python 3.10 or newer. It is a standalone extension and is not a `symbolica.community` module. Its package version is independent of the Rust `linnet` version. Record both versions when diagnosing compatibility between Rust and Python code.

Try the live Python playground for editable DOT and a browser notebook, or start with the runnable graph example in the Python quickstart.

The Python binding separates arbitrary application data from drawing data. `Node.data`, `Edge.data`, and `HalfEdge.data` may contain any Python object. Linnet retains those objects by reference: it does not copy, compare, hash, pickle, stringify, or send them to Typst. The matching `drawing` properties contain only validated, typed values that the renderer can stage safely.

Build a graph declaratively with node specifications, endpoint specifications, and edge specifications. The payload classes below belong to the example application; they are not Linnet types:

```
from dataclasses import dataclass

from linnet_py import build, edge, node, sink, source


@dataclass
class UserNodeData:
    state: object


@dataclass
class UserEdgeData:
    interaction: object


@dataclass
class UserHalfEdgeData:
    port: object


incoming_data = UserNodeData(state=object())
outgoing_data = UserNodeData(state=object())
propagator_data = UserEdgeData(interaction=object())
```

```
incoming = node("in", data=incoming_data)
outgoing = node("out", data=outgoing_data)
graph = build(
    incoming,
    outgoing,
    edge(
        source(incoming),
        "propagator",
        sink(outgoing),
        data=propagator_data,
    ),
    name="example",
)

assert graph.node("in").data is incoming_data
assert graph.edge("propagator").data is propagator_data
```

`graph.node("in")`, `graph.edge("propagator")`, and `graph.half_edge(0)` return live views. Names and integer indices are accepted for node and edge lookup. Topology properties such as `index`, `incidence`, `source`, `sink`, `pair`, and `flow` are read-only, while `data` and every field on `drawing` are mutable. Endpoint-specific `source(..., data=...)` and `sink(..., data=...)` payloads are optional; use them only when the application has distinct half-edge data. Drawing fields include labels and styles as well as placement, routing, rank, size, bend, decoration, and half-edge anchor information. Put a template-specific field such as a particle or cut identifier in `drawing.extensions`; built-in field names are reserved and Linnet assigns no domain meaning to extensions.

`Graph.map(node=..., edge=..., source=..., sink=...)` evaluates the supplied callbacks against live element views and returns a new graph. An omitted callback preserves the exact Python data object. Drawing and rendering configuration are copied by value. Reordering nodes or edges and reversing an edge preserve each data/drawing association, but invalidate every older live view; using a stale view raises `ReferenceError`.

Incremental edits reuse the declarative constructors instead of introducing a second set of drawing dictionaries. `add_node` accepts a `NodeSpec`; `add_edge` accepts the same internal or dangling `EdgeSpec` used by `build`. Declarative endpoints resolve a passed `NodeSpec`, name, index, or live-node key. Incremental endpoints resolve a current live `Node`, name, index, or a named `NodeSpec`:

```
from linnet_py import Compass, Graph, edge, node, sink, source

graph = Graph()
graph.add_node(node("in", data=UserNodeData(object()), label="incoming"))
graph.add_node(node("out", data=UserNodeData(object())))

propagator = graph.add_edge(edge(
    source("in", data=UserHalfEdgeData(object()), compass=Compass.E),
    "propagator",
    sink(graph.node("out"), data=UserHalfEdgeData(object()), compass=Compass.W),
    data=UserEdgeData(object()),
    label="dependency",
))
```

Each successful insertion is one atomic topology revision and returns a fresh view. Reacquire a view by name or index after the next edit. Validation failures do not publish the candidate and therefore leave existing views live. Arbitrary data keeps Python object identity; specs and their drawing dictionaries are copied by value and remain reusable.

`split_edge(at, replacement, ...)` turns a paired edge into two dangling edges. The selected half-edge receives the detached replacement `EdgeValue`; the opposite half retains the original edge value. `connect(source, sink, replacement, ...)` performs the topological inverse, with the first dangling half-edge becoming

the source. The two dangling edge records are replaced by the supplied value, name, and orientation. Use `delete(graph.subgraph(...))` for removal; there are no duplicate `remove_node` or `remove_edge` aliases.

The long-lived topology is Linnet's native `HedgeGraph`; it is not a separate Python adjacency model. `NodeStore.Vec` is the default dense node store, while `NodeStore.Forest` selects the identification-preserving forest store. Pass `node_store=` to `Graph`, `build`, or a DOT importer, inspect `graph.node_store`, and use `graph.to_node_store(...)` for an explicit non-mutating conversion. Derived graphs preserve their input backend; append and join use the left graph's backend.

A `Subgraph` is a graph-bound, revision-checked native half-edge selection, augmented only with explicit zero-crown node IDs so isolated nodes remain first-class members. Construct one from named or indexed nodes and edges, exact half-edge indices, or live-view predicates, then use the same object with Linnet's topology algorithms and owning transformations:

```python
from linnet_py import DirectionBasis

# Explicit selections are unioned. Selecting a node includes its incident crown.
selected = graph.subgraph(nodes=["in"], edges=["propagator"])

# Predicates see the same live views as normal graph access.
propagators = graph.filter(
    edge=lambda value: isinstance(value.data, UserEdgeData),
)

component_copies = [graph.concretize(part) for part in graph.connected_components()]
tree = graph.depth_first_traverse("in", subgraph=selected)
cycles, covered = graph.cycle_basis(selected)

# Boundary operations return composable structural selections.
inside_boundary = graph.internal_boundary(selected)
incident_boundary = graph.boundary(selected)
external = graph.external_half_edges()

# Directed algorithms default to the underlying source/sink flow.
assert graph.is_reachable("in", "out")
underlying_order = graph.topological_order()
superficial_order = graph.topological_order(
    direction=DirectionBasis.Superficial,
)

# Enumeration is explicit about its potentially exponential search space.
circuits = graph.all_cycles(subgraph=selected, max_results=31)
one_bond = graph.find_bond(subgraph=selected, min_size=1, max_size=1)
tadpole_components = graph.tadpoles(["in"])

# The result owns a reduced topology; views into graph remain live.
reduced = graph.transitive_reduction()

for partition in graph.all_cuts(["in"], ["out"]):
    source_region = partition.source_side
    cut_edges = partition.edges
    target_region = partition.target_side
    relative_orientation = partition.boundary.orientation(cut_edges[0])

# Inclusion-minimal connected bipartition boundaries of size one.
bridge_bonds = graph.all_bonds(subgraph=selected, min_size=1, max_size=1)

# concretize() is non-mutating; insertion, split/connect, extract(), delete(),
# contract(), append_mut(), and join_mut() stale old views and selections.
```

```
detached = graph.concretize(propagators)
removed = graph.extract(propagators)
```

`Subgraph` supports immutable union, intersection, difference, symmetric difference, complement, and their Python operators. The binding also exposes connectivity and component queries, DFS and BFS trees, tree parents/children/ancestors and fundamental cycles, boundary/crown queries, bridges, cycle bases and bounded circuit enumeration, spanning forests, cut partitions, bond queries, tadpole components, contraction, disconnected append, and callback-driven joining of dangling half-edges. `append()` and `join()` return a graph while their `*_mut()` counterparts update the left graph; the right graph is unchanged. Transformations retain arbitrary data by Python identity and copy drawing/configuration by value. Callback failures and name collisions never publish a candidate topology. Join callbacks receive live views, so any explicit `data` or `drawing` mutations they perform take effect immediately and are not rolled back. Every cut terminal must be part of a disjoint source or target node group and have an incident half-edge, while each combined terminal group must have a boundary. An isolated node mixed into an otherwise connected terminal group is rejected instead of being silently dropped. `all_cuts` eagerly materializes the native admissible separating partitions and can be combinatorial. Its result is sorted by structural half-edge indices, while `CutPartition.boundary.left` and `.right` retain the orientation of each boundary half-edge. `all_bonds` enumerates inclusion-minimal connected bipartition boundaries in a size range; it is not a minimum-cardinality or max-flow operation. `find_bond` returns one deterministic native witness without materializing the complete family. `all_cycles(max_results=...)` checks the full non-empty cycle-space candidate count before enumeration; `max_results` therefore bounds work even when only a few combinations are circuits. Tadpole terminals must be distinct, valid, and incident to at least one half-edge; identification-history aliases of the same structural node are duplicates.

`DirectionBasis.Underlying` is the default for reachability, topological order, and transitive reduction. It follows the intrinsic half-edge source/sink roles used by GammaLoop momentum routing. `DirectionBasis.Superficial` applies each edge's drawing-independent `Orientation`: reversed edges reverse the arc and undirected edges contribute no arc. A direction choice is local to the algorithm and never changes stored flow or orientation. `transitive_reduction()` returns a new graph with retained arbitrary payload objects shared by identity; Python transitive closure is intentionally absent until synthesized edges have an explicit data/drawing factory. When `is_reachable(..., subgraph=...)` is used, both endpoint nodes must belong to that selection; an endpoint outside it is a `ValueError`, rather than an implicit unreachable result.

Traversal-tree relationship methods validate that a node belongs to that exact tree revision. `parent` returns `None` at the root, `children` returns immediate children, and `ancestors` runs from the immediate parent through the root. `fundamental_cycle` returns `None` for a tree edge and rejects dangling edges or edges whose endpoints are outside the traversal tree. An `OrientedCut` exposes its edges, the orientation of each edge relative to the cut, and its winding number against a cycle from the same graph revision.

13.3.1 Typed Typst configuration

Plain Python strings are always escaped Typst strings, never source expressions. `None` maps to Typst `none`, `AUTO` maps to `auto`, and `INHERIT` leaves the value from the preceding configuration layer intact. The closed native-value model also includes `Length`, `Ratio`, `RelativeLength`, `Angle`, `Fraction`, `Color`, `Stroke`, `Dash`, `Insets`, `Mark`, `TextLabel`, and `MathSymbol`, plus recursively validated arrays and string-keyed dictionaries. Finite layout, placement, routing, anchor, pattern, mark, and debug choices use their exported enums; the structured Python reference lists the choices for each option.

`GraphStyleOptions`, `LayoutOptions`, and `DrawOptions` cover the corresponding generic Linnest surfaces. `DrawingSelectors` maps arbitrary Python data to detached typed drawing values at render time. Layout passes retain their order:

```
from linnet_py import (
    Color,
    DrawingSelectors,
    EdgeDrawing,
```

```
    LayoutAlgorithm,
    LayoutDirection,
    LayoutOptions,
    Length,
    RenderConfig,
    Stroke,
)

layouts = LayoutOptions(
    algorithm=LayoutAlgorithm.Force,
    direction=LayoutDirection.Right,
    steps=200,
).then(label_steps=80)
graph.render_config = RenderConfig(
    title="Example",
    layouts=layouts,
    selectors=DrawingSelectors(
        edge=lambda value: EdgeDrawing(
            label="dependency",
            style={
                "stroke": Stroke(
                    paint=Color("navy"),
                    thickness=Length.pt(0.8),
                ),
            },
        ) if isinstance(value.data, UserEdgeData) else None,
    ),
)

output = graph.render("diagram.pdf")
svg = graph.to_svg()
graph  # the final expression in a notebook renders inline
```

The selected template's defaults are overlaid by the graph's `render_config` and then by a sparse per-call `config`. `render(output, config=None)` writes PDF, SVG, or PNG according to the output suffix. `to_svg(config=None)` returns SVG text, and `_repr_svg_()` supports notebooks. These are the only high-level rendering methods; there are no raw command-line inputs or string-expression escape hatches. The Python distribution depends on `typst` 0.15.0 and compiles in-process without looking up or launching a Typst executable. Generated inputs and imported modules remain alive until compilation completes. The wheel has no Clinnet dependency and embeds the Linnest, Kurvst, CeTZ 0.5.1, and oxifmt 1.0.0 assets required by its default renderer, so that path does not depend on an installed Typst package cache or a network fetch.

For inspection tools, `prepare_render(config=None)` evaluates selectors once and returns a `PreparedRender`. Its `typst_source` property is the exact generated entrypoint subsequently used by `to_svg()` or `render(path)` on that preparation. It keeps the ephemeral entrypoint, versioned topology-only CBOR graph spec, bundled assets, and snapshots of referenced user template/module source trees alive. Editing or deleting the original files after preparation therefore cannot change that preparation. The graph spec contains structural names, indices, incidence, flow, and orientation, while typed drawing values travel in `config.elements`; it is not a hidden DOT serialization and never inspects arbitrary Python `.data`.

The shipped notebook display is generic and model-neutral, including for graphs with dangling edges. It never infers amplitude, cross-section, particle, or momentum semantics from topology. An application that needs those concepts selects its own template and passes that template's closed native settings through `RenderConfig.template_options`. `INHERIT` leaves the preceding configuration layer unchanged, while `None` becomes Typst `none`.

```
domain_template = RenderConfig(
    template="templates/domain-figure.typ",
```

```
    source_root=".",
    template_options={
        "theme": "review",
        "accent": Color("teal"),
    },
)
```

`template_options` is intentionally a string-keyed dictionary because the selected template, not Linnet, defines that open schema. Its values still use the closed native-value model; it is not a dictionary of Typst source strings.

`source_root` identifies the project tree staged for file-based templates and modules. Set it when a source imports files outside its own directory, such as `../shared.typ`. Without an explicit root, Linnet uses the nearest ancestor containing `typst.toml`, or otherwise the source file's parent directory; it does not infer project boundaries from generic directory names such as `src`.

`graph.render_config` is a live mutable object: assign its fields directly, or replace the whole configuration. The nested style, layout, drawing, and selector option objects are immutable values, and their getters return copies, so replace those fields rather than trying to mutate a nested object in place. `overlay()` returns a new configuration, and a per-call overlay never mutates the graph's stored defaults.

Python drawing selectors belong on `RenderConfig.selectors`. They may inspect arbitrary `.data`, but must return the corresponding detached `NodeDrawing`, `EdgeDrawing`, or `HalfEdgeDrawing`, or `None` when no defaults apply. Selector fields fill only drawing fields that are absent on the element, so explicit element drawing remains authoritative. Only the validated drawing snapshot crosses the Python-to-Typst boundary. The example above happens to color an application-defined edge payload; custom particle styling works the same way and is not a special Linnet mode:

```
from dataclasses import dataclass

from linnet_py import (
    Color,
    DrawingSelectors,
    EdgeDrawing,
    Length,
    MathSymbol,
    RenderConfig,
    Stroke,
)


@dataclass
class ParticleStyle:
    color: Color
    symbol: str


def particle_drawing(value):
    if not isinstance(value.data, ParticleStyle):
        return None
    return EdgeDrawing(
        label=MathSymbol(value.data.symbol),
        style={"stroke": Stroke(paint=value.data.color, thickness=Length.pt(0.8))},
    )


particle_style_example = RenderConfig(
    selectors=DrawingSelectors(edge=particle_drawing),
)
```

This is ordinary userland settings composition: Linnet neither defines `ParticleStyle` nor inspects it. GammaLoop's richer particle decorations, momentum annotations, and diagram modes belong to GammaLoop's own template rather than to a `PhysicsOptions` type in `linnet_py`. The editable DOT physics notebook parses ordinary DOT with an explicit application codec, then composes particle, momentum, edge-index, and node-index layers using the same generic API. Run it from a checkout with `nix develop -c uvx --from marimo==0.24.0 --with-editable crates/linnet-py marimo edit crates/linnet-py/examples/physics_render_settings.py`. The streaming layout notebook previews the force solver as it runs. Its DOT editor and collapsible sliders restart the simulation; pause/resume preserves the current state. The viewer creates SVG topology once, then updates coordinates from `LayoutStream.from_dot(...)` frames. It also runs in an exported Marimo WebAssembly notebook with the bundled Emscripten wheel.

```
from linnet_py import LayoutStream

stream = LayoutStream.from_dot("digraph { a -> b; b -> c; c -> a; }", every=4)
node_names, endpoints = stream.node_names, stream.endpoints
for frame in stream:
    positions = frame.nodes
    edge_positions = frame.edges
```

The first frame has iteration zero; subsequent frames advance up to `every` iterations until a final `done` frame. Batch size affects observation frequency, not the random initialization, cooling, or depth-collapse schedule. No history is retained by the iterator. Frames own their position data; changing an earlier frame's returned arrays cannot affect later frames. Node names and endpoint indices stay in the same order. This proof of concept supports a single DOT graph and the force algorithm. Its numeric geometry includes explicit DOT placement constraints and per-edge spring lengths, but not Typst label measurement, styling, or final label relaxation. It does not alter an existing Python `Graph` or its payloads.

Typst callbacks that require measured geometry use an explicit module function reference:

```
from linnet_py import TypstModule

styles = TypstModule.file("styles.typ")
graph.node("in").drawing.label = styles.content("incoming_label")
graph.edge("propagator").drawing.style = styles.function("edge_style").bind(
    emphasis=True
)
graph.edge("propagator").drawing.extensions = {
    "lane-map": styles.value("lane_map"),
}
```

`TypstModule.file` and `TypstModule.package` are the only routes for custom executable Typst. `value`, `content`, and `function` select a typed export; `call` and `bind` construct typed applications. The binding never accepts a raw Typst expression and never calls Typst `eval`.

A custom rendering template is a Typst module with one mandatory V1 export:

```
#let render(config) = {
  assert(config.schema == "linnet-render-config")
  assert(config.version == 1)
  let options = config.at("options", default: (:))
  // Read typed element values and template-owned options, then render content.
  config.elements.nodes.at(0).label
}
```

The bundled generic template uses this contract and ignores unknown domain concepts. A custom template that needs the native topology can import Linnet's `graph.typ` and call `graph.from-spec(read(config.at("graph-spec-path"), encoding: none))`, attach native values with Linnet's `attach-elements(g, config.elements)`,

and delegate to `layout-graph(config, g)`. Final labels and styles are measured before layout so their sizes affect spacing.

GammaLoop's separate `figure.typ` consumes DOT through `data-path`; it cannot be selected directly by this Python graph-spec adapter. GammaLoop supplies ordinary Typst particle/momentum callbacks and optional external-edge placement without Python. Python callers retain this API's own drawing records and selectors, or import styling through `TypstModule`; the physics notebook is an independent example of that interface.

13.3.2 Explicit DOT codecs

DOT conversion is never implicit because arbitrary Python data and rich drawing values have no unique DOT representation. Supply a `DotCodec` to `Graph.from_dot`, `Graph.from_dot_set`, `Graph.from_dot_file`, and `Graph.to_dot`:

```
from dataclasses import dataclass

from linnet_py import (
    DotCodec,
    DotEdgeData,
    DotHalfEdgeData,
    DotVertexData,
    EdgeValue,
    Graph,
    HalfEdgeValue,
    NodeDrawing,
    NodeValue,
    TextLabel,
)


@dataclass
class VertexPayload:
    name: str | None
    kind: str | None


@dataclass
class EdgePayload:
    kind: str | None


@dataclass
class PortPayload:
    statement: str | None

codec = DotCodec(
    encode_node=lambda value: DotVertexData(
        name=value.data.name,
        statements={"kind": value.data.kind or ""},
    ),
    decode_node=lambda dot: NodeValue(
        data=VertexPayload(dot.name, dot.statements.get("kind")),
        drawing=NodeDrawing(label=TextLabel(dot.name or "")),
    ),
    encode_edge=lambda value: DotEdgeData(
        statements={"kind": value.data.kind or ""},
    ),
    decode_edge=lambda dot: EdgeValue(
        data=EdgePayload(dot.statements.get("kind")),
    ),
```

```
    encode_half_edge=lambda value: DotHalfEdgeData(statement=value.data.statement),
    decode_half_edge=lambda dot: HalfEdgeValue(data=PortPayload(dot.statement)),
)

source = r'''digraph demo {
  incoming [kind="external"]
  outgoing [kind="external"]
  incoming -> outgoing [kind="propagator"]
}'''
graph = Graph.from_dot(source, codec)
canonical = graph.to_dot()  # reuses the codec stored by from_dot
```

Codec callbacks receive detached `NodeValue`, `EdgeValue`, or `HalfEdgeValue` snapshots with only `data` and `drawing`; they never receive graph or index context. `GlobalData` remains separate, as in the Rust serializer contract, and is not staged during rendering. An explicit codec passed to `to_dot` overrides the stored codec, and callback exceptions propagate without publishing a partial graph.

Use `DotCodec.linnest()` to retain canonical, non-executable DOT-representable drawing fields while dropping arbitrary element `.data`. It rejects Typst module references, calls, and bindings on both encode and decode; use a custom codec when trusted executable values must be reconstructed explicitly. Use `DotCodec.topology()` to drop element data and drawing. Both helpers preserve graph and element names as well as the separate `GlobalData`. Round trips promise canonical semantics rather than the original whitespace or attribute order. Consult the structured Python API for every drawing field, option, enum, and DOT data constructor.

14 Linnest Typst API

Linnest documents a small supported surface around Linnet's graph model. Start with the workflow guide if you want to build and draw a graph; use these pages when you need an exact function, parameter, or default.

14.1 Supported surface

- `graph` is a module for construction, parsing, inspection, data transforms, joins, cycles, and forests.
- `layout` and `layout-sequence` are top-level functions. They are documented together because both come from `layout.typ`.
- `draw`, `edge-halves`, and `to-cetz-edge-halves` are top-level drawing functions from `draw.typ`.
- `subgraph` is a module for constructing and inspecting zero-copy subgraph values.
- Domain styles and templates explains how application-owned callbacks compose the generic drawing surface.
- `curve` is a re-export of Kurvst, not a second Linnest API. Use the canonical Kurvst curve and path reference.

Modules and functions import differently
Import `graph` and `subgraph` as modules when you want qualified calls such as `graph.build`. Import `layout`, `layout-sequence`, and `draw` directly from `lib.typ`; they are functions rather than nested modules.

```
#import "crates/linnest/typst/src/lib.typ": draw, graph, layout, subgraph
#import graph: build, edge, node, sink, source
```

The same package also exports the `layouts` module for callers that need the underlying layout namespace; its `layouts.layout` and `layouts.sequence` aliases are linked from the layout page. Bindings exposed only as a consequence of Typst module loading, such as serialization helpers or imported dependency modules, remain implementation details rather than supported API. The focused reference uses the supported functions and modules above.

15 graph module

Use `graph` for graph construction, DOT parsing, inspection, data transforms, joins, and graph algorithms. Import the module for qualified calls, or import selected functions from it when a short construction DSL is clearer.

15.1 Concepts

15.1.1 Graph Specs

The Typst construction API is half-edge first: create node items, create source and sink half-edge endpoints, then pass edge items to `graph.build`. `graph.build` accepts both comma-separated items and ordinary Typst code blocks. Typst labels such as `<a>`, `<h1>`, and `<e1>` are API names; node and edge names are emitted back from `nodes(g)` and `edges(g)` as Typst labels. They are resolved before the wire format is sent to the Rust plugin. Numeric `id` arguments choose graph indexes or ordering: on nodes, `id` fixes the resulting node index and must be unique and in bounds. On nodes, edges, sources, and sinks, extra named arguments are captured as opaque Typst data fields: `edge(source(<a>, style: (stroke: red + 0.7pt)), sink(<b>), kind: "link")` stores `(style: ..)` in the source data and `(kind: "link")` in the edge data. These user data values are not sent through the Rust plugin boundary. Typst sends an internal opaque payload with correlation data, asks Rust to resolve the graph indexes, then stores user data in arrays at those resolved graph, node, edge, and half-edge ids. `graph.info`, `graph.nodes`, and `graph.edges` merge those native values into the returned records. A captured `label` data field on nodes and edges is display content used by the default drawing style; use `statements: (label: "...")` when a flat metadata label string is needed. Statements are flat metadata used by DOT; they cannot nest. Values are scalar strings/numbers/booleans. Use data fields for structured Typst data or content.

`graph.build` and `graph.parse` also accept `default-node-data`, `default-edge-data`, `default-source-data`, and `default-sink-data`. These defaults are merged into the corresponding data; captured data fields on nodes, edges, sources, and sinks override the defaults. Drawing callbacks decide how those fields affect the result. For example, endpoint styles can read `data.style` from the source and sink half-edge records:

```
#let g = build({
  node(<a>, label: [a])
  node(<b>, label: [b])
  edge(
    source(<a>, style: (stroke: red + 0.7pt)),
    <e>,
    sink(<b>, style: (stroke: blue + 0.7pt)),
    label: [connection],
    kind: "dependency",
  )
},
  default-edge-data: (kind: "propagator"),
)
#let endpoint-style(edge, side) = {
  let endpoint = edge.at(side + "-half-edge", default: none)
  if endpoint == none { return (:) }
  endpoint.data.at("style", default: (:))
}
#draw(
  layout(g),
  source-style: edge => endpoint-style(edge, "source"),
  sink-style: edge => endpoint-style(edge, "sink"),
  edge-label: edge => edge.at("label", default: none),
)
```

When parsing DOT, the same native data arrays can be filled from selected string fields. Rust parses DOT into topology and statement metadata; Typst applies defaults and evaluates selected fields afterward. The selected fields are evaluated with the record's merged `fields` dictionary in scope. Since node names are labels, use `#str(name)` when a name should become visible text:

```
#let g = parse(
  "digraph g { a [label=\"A\"]; a -> b [label=\"$p$\", source=\"out\", sink=\"in\"] }",
  eval-node-fields: ("label",),
  eval-edge-fields: ("label",),
  eval-source-fields: ("statement",),
  eval-sink-fields: ("statement",),
).first()
#nodes(g).first().data.label
#edges(g).first().data.label
```

The same transform can run after construction. This is useful for global edge statements that should apply to every edge while still seeing local edge fields:

```
#let g = build({
  node(<a>)
  node(<b>)
  edge(source(<a>), sink(<b>), statements: (mom: "p"))
}, default-edge-statements: (display-label: "$#mom$"))
#let g = graph.eval-fields(g, eval-edge-fields: ("display-label",))
#edges(g).first().data.at("display-label")
```

```
#let g = build({
  node(<a>)
  node(<c>)
  edge(source(<a>), <e1>, sink(<c>))
})
```

Named nodes and edges can be updated after construction without scanning in the caller. The update replaces the native data, or it can be a callback receiving `(data, record)`:

```
#let g = build({
  node(<a>)
  node(<c>)
  edge(source(<a>), <e1>, sink(<c>))
})
#let g = update-node-data(g, <a>, (label: [A]))
#let g = update-edge-data(g, <e1>, (data, edge) => (
  label: [$p$],
  source: edge.source.node,
))
#node-data(g, <a>).label
#edge-data(g, <e1>).label
```

`source(..)` and `sink(..)` accept a node reference plus optional `name`, `id`, `statement`, and `compass`. `name` is a Typst label name for the half-edge; `id` is numeric:

```
edge(source(<a>, name: <h1>, id: 0), <e1>, sink(<c>, id: 2), label: [a-c])
```

One half-edge creates an external edge. The side is determined by the constructor, so there is no public `flow` argument:

```
edge(<incoming>, sink(<a>))
edge(source(<c>), <outgoing>)
```

`graph.edge(..., spring-length: 1.5)` multiplies that edge's existing preferred spring length by `1.5` in force/anneal layout. The factor must be positive, finite, and dimensionless; it applies to each node-to-control-point

spring, including the existing doubled rest length for dangling edges. The argument defaults to `none`, preserving local or default `spring-length` statements; a missing statement means `1`. An explicit argument overrides those statements. This is not a guaranteed rendered length, a rendering option, or a change to global layout `spring.length`; tree/dot/stable-layered layout is unaffected.

`graph.build` does not interpolate statement strings on the Rust side. Use `graph.eval-fields` or `graph.map` when a default statement should turn into Typst content or structured data. The evaluation scope includes the record's merged fields, so default edge statements can still refer to local edge fields:

```
#let g = build({
  node(<a>)
  node(<c>)
  edge(
    source(<a>),
    <a-c>,
    sink(<c>),
    label: [a-c],
    statements: (color: "0055ff", label: "a-c"),
  )
},
  default-edge-statements: (
    color: "000000",
    display-label: "$#label$",
  ),
)
#let g = graph.eval-fields(g, eval-edge-fields: ("display-label",))
```

15.1.2 Graph object API

Graph objects are Typst dictionaries wrapping archived Rust graph bytes plus native Typst data arrays for graph, node, edge, source, and sink data. The Rust graph may also carry an internal opaque payload, but that payload is used only by the Typst wrapper and is not exposed in public records. Build or parse graph objects with `graph`, transform graph objects with `layout`, and pass objects back to `graph` or `subgraph` for inspection. Subgraphs are also dictionaries: they combine an archived half-edge selection with a topology signature and Typst-only annotations, rather than exposing raw bytes as public inputs.

- `graph.parse(input)` parses one or more DOT digraphs and returns an array of graph objects. Its `eval-graph-fields`, `eval-node-fields`, `eval-edge-fields`, `eval-source-fields`, and `eval-sink-fields` arguments are convenience arguments for `graph.eval-fields`.
- `graph.build(..)` constructs one graph object from a stream of node and edge items.
- `graph.map(graph, ..)` maps graph, node, edge, source, and sink records to new native data or structural patches without changing topology. `node` and `edge` accept a callback or a dictionary keyed by current record names. Each dictionary value is a patch, a callback receiving the full record, or `none`. Edge patches may set `spring-length` directly; it is validated and stored as the layout statement. Unlisted records remain unchanged; unknown names are errors.
- `graph.cut(graph, left: left, right: right, boundary: patch)` opens a weighted directed cut into a new graph, preserving Typst data and recording origins. `graph.boundaries(view)` queries its current boundary endpoints.
- `graph.node-data(graph, <name>)` and `graph.edge-data(graph, <name>)` return one named node or edge data.
- `graph.update-node-data(graph, <name>, data)` and `graph.update-edge-data(graph, <name>, data)` update one named node or edge data. Direct replacements and callbacks run in Typst with `(data, record)`.
- `graph.eval-fields(graph, ..)` evaluates selected record fields into data entries. It works on parsed and built graph objects.
- `node(..)` returns a node item.
- `source(..)` and `sink(..)` return half-edge endpoints.
- `edge(..)` returns an edge item built from source/sink endpoints and an optional edge name.

- `layout(graph, ..)` runs layout as an explicit second step. Its settings are named parameters so calls stay descriptive and Tidy can document each field.
- `draw(graph, ..)` draws a laid-out graph object with CeTZ.
- `dot(graph)` returns a DOT string for inspection or export.

15.1.3 Graph Queries

`graph.info(g)` returns graph metadata. `nodes(g)` returns node records, and `edges(g)` returns edge records. Node and edge record `name` values are Typst labels when present. Pass `subgraph: sg` to filter nodes or edges by a compatible subgraph object. Nodes must be incident to a selected half-edge; an edge is included if either half is selected, but its record keeps both available endpoints. These queries do not extract or open topology.

After `graph.cut`, node, edge, and half-edge records carry `origin` relative to the immediate input graph. Boundary nodes and dangling cut edges also carry `boundary`. Use `graph.boundaries(view)` for a uniform endpoint query, including current positions after placement or layout. Auxiliary nodes remain visible in `graph.nodes(view)`; filter with `node.boundary == none` to exclude them.

`graph.join(left, right, key: "statement")` joins dangling half edges with opposite flows. The supported keys are `"statement"`, `"compass"`, `"port-label"`, and `"id"`; unmatched half edges remain in partial joins. Matching identities must be unique within each input. Node and edge labels that occur in both inputs are rejected. Graph, node, edge, and endpoint data is preserved: right-only dictionary fields are retained and conflicting fields use the left value. The left graph supplies the name and layout configuration. Use `graph.join-edges` when the matching key is an edge-level statement rather than half-edge metadata.

`graph.cycles(g)` returns subgraph objects for a cycle basis. `graph.forests(g)` returns subgraph objects for spanning forests. Both return arrays of the same topology-bound dictionaries accepted by `subgraph`, graph queries, `layout`, and `draw`, not arrays of raw selection bytes.

15.2 graph

- from-spec()
- build()
- parse()
- node()
- source()
- sink()
- edge()
- group()
- pin()
- start()
- pos()
- map()
- style()
- eval-fields()
- info()
- dot()
- nodes()
- edges()
- node-data()
- edge-data()
- update-node-data()
- update-edge-data()
- cut()
- boundaries()
- join()
- join-edges()

- cycles()
- forests()

15.2.1 from-spec

Construct a graph from a versioned Linnest graph-spec CBOR document.

This is the native renderer boundary. Unlike `parse`, it does not parse DOT.

15.2.1.1 Parameters

```
from-spec(input) -> dictionary
```

15.2.2 build

Build one graph object from a stream of node and edge items.

Use `node`, `source`, `sink`, and `edge` to create graph items. Positional items may be passed as comma-separated arguments or yielded from a Typst code block.

15.2.2.1 Parameters

```
build(
  ..items: array,
  name: none string,
  data: any,
  statements: dictionary,
  default-node-statements: dictionary,
  default-edge-statements: dictionary,
  default-node-data: any,
  default-edge-data: any,
  default-source-data: any,
  default-sink-data: any
) -> dictionary
```

15.2.2.1.1 ..items array

Node and edge items returned by `node` and `edge`. The best way to use these is to use a code scope, so that the edges and nodes append each other:

```
#let g = build({
  node(<a>, label: [a])
  node(<b>, label: [b])
  node(<c>, label: [c])
  edge(source(<a>), sink(<b>))
  edge(source(<b>), sink(<c>))
  edge(source(<a>), sink(<c>))
})
```

15.2.2.1.2 name `none` or `string`

Graph name.

```
#let g = build(name: "My Graph", {
})
#info(g).name
```

Default: `none`

15.2.2.1.3 data `any`

Native Typst graph data.

```
#let g = build(data: (a:(b:(1, ))), {
})
#info(g).data
```

Default: `none`

15.2.2.1.4 statements `dictionary`

Flat graph statements, that get turned into a string to string dictionary in rust. Used by DOT parsing; values cannot nest.

```
#let g = build(statements: (a:1), {
})
#info(g).global-statements
```

Default: `(:)`

15.2.2.1.5 default-node-statements `dictionary`

Flat default node statements. Used by DOT; values cannot nest.

Default: `(:)`

15.2.2.1.6 default-edge-statements `dictionary`

Flat default edge statements. Used by DOT parsing; values cannot nest. These are applied/delegated to all edges.

```
#let g = build(default-edge-statements: (a:1), {
   node(<a>)
   edge(source(<a>))
   edge(sink(<a>))
})
#edges(g).map(e=>e.statements)
```

Default: `(:)`

15.2.2.1.7 default-node-data `any`

Default data merged into every node data. Captured node data fields override it.

```
#let g = build(default-node-data: (a:1), {
   node(<a>)
   node(<b>,a:2)
   node(<c>,b:(a:1))
})
#nodes(g).map(n=>n.data)
```

Default: `none`

15.2.2.1.8 default-edge-data `any`

Default data merged into every edge data. Captured edge data fields override it.

```
#let g = build(default-edge-data: (a:1,b:[$m_mu$]), {
   node(<a>,id:0)
   edge(source(<a>),id:1)
   edge(sink(<a>),<e>,id:0,b:[#set text(font:"Reforma")
   This is a test: ])
})
#edges(g).map(e=>e.data.b).join()
```

Default: `none`

15.2.2.1.9 default-source-data `any`

Default data merged into every source half-edge data. Captured source data fields override it.

```
#let g = build(default-source-data: (a:1), {
   node(<a>)
   edge(source(<a>))
   edge(sink(<a>))
})
#edges(g).map(e=>if e.source != none {e.source.data} else {none})
```

Default: `none`

15.2.2.1.10 default-sink-data `any`

Default data merged into every sink half-edge data. Captured sink data fields override it.

```
#let g = build(default-sink-data: (a:1), {
   node(<a>)
   edge(source(<a>))
   edge(sink(<a>))
})
#edges(g).map(e=>if e.sink != none {e.sink.data} else {none})
```

Default: `none`

15.2.3 parse

Parse one or more DOT graphs into graph objects.

Default data are applied before `eval-*` fields are evaluated, so default data strings can refer to parsed record fields such as `#str(name)`. Parsed fields take precedence over default data fields, so DOT `label="..."` overrides `default-node-data: (label: ...)`.

Linnest uses `dot-parser` for DOT syntax and then gives special meaning to a small set of attributes. Every other attribute is preserved as a flat string statement. Preserved statements are visible through `info`, `nodes`, `edges`, `map`, and `eval-fields`, but they do not become native Typst `data` unless a matching `eval-*` argument is passed.

````
#let src = ```dot
digraph { a [label="A"]; }
```
#let n = nodes(parse(src.text).first()).first()
#n.statements.label
#n.data
````

Graph-level handling:

- The DOT graph name becomes `graph.info(g).name`. A graph-level `name` attribute can name an anonymous graph; a named `graph`/`digraph` header takes precedence.
````

```
#let src = ```dot
digraph header_wins { graph [name="ignored"]; a }
```
#info(parse(src.text).first()).name
```

- Top-level `key=value` statements and `graph [key=value]` attributes become `graph.info(g).global-statements`, except for the consumed `name`. They do not configure layout, even when a key looks like a layout option; pass layout options directly to `layout`.

```
#let src = ```dot
digraph { steps=1; graph [subtitle="nested"]; a }
```
#let statements = info(parse(src.text).first()).global-statements
#statements.steps
#statements.subtitle
```

- `node [key=value]` and `edge [key=value]` become `default-node-statements` and `default-edge-statements`. They are merged into each parsed node or edge before local attributes, so local attributes override defaults.

```
#let src = ```dot
digraph { node [color=gray]; edge [particle=g]; a [color=red]; a -> b }
```
#let g = parse(src.text).first()
#nodes(g).first().statements.color
#edges(g).first().statements.particle
```

Node handling:

- The DOT node id becomes the node `name` returned by `nodes`. If the DOT node id is numeric, it is consumed as the node index instead and the public `name` is `none`.

```
#let src = ```dot
digraph { alpha; 1; }
```
#nodes(parse(src.text).first()).map(n => n.name)
```

- `id=<n>` is also consumed as an explicit node index. Explicit node indexes must be unique and in bounds after parsing.

```
#let src = ```dot
digraph { a [id=1]; b [id=0]; }
```
#nodes(parse(src.text).first()).map(n => n.name)
```

- `style=invis` marks the DOT node as a dangling external endpoint. It is not returned by `nodes`. Its remaining attributes are copied onto the external edge that touches it, except `shape` and `label`; `style` and `id` are consumed.

```
#let src = ```dot
digraph { ext [style=invis, column=left, label=skip]; ext -> a [id=0]; }
```
#let g = parse(src.text).first()
#nodes(g).map(n => n.name)
#edges(g).first().statements.column
#edges(g).first().statements.at("label", default: none)
```

- Node `style` is consumed for every node; only `style=invis` has special behavior. Use another attribute name for drawing style metadata that must survive parsing.

```
#let src = ```dot
digraph { a [style=filled, "draw-style"=filled]; }
```
#nodes(parse(src.text).first()).first().statements
```

- `pos` is parsed as node placement. Simple numeric values are exposed as `node.pos`; extended placement values are used by `layout`. `pin` is an explicit layout constraint and is not exposed as a public statement. The `pos-x-set` and `pos-y-set` record fields say which axes were explicitly supplied, including for partial placements whose resolved `pos` contains both coordinates. `dot` preserves them as canonical Linnest DOT extension attributes.

```
#let src = ```dot
digraph { a [id=0, pos="0,0!"]; b [id=1, pos="ref(node:0)+2,0!"]; }
```
#let parsed-nodes = nodes(parse(src.text).first())
#parsed-nodes.map(n => n.pos)
#parsed-nodes.map(n => n.statements.at("pin", default: none))
```

- `shift` is parsed as a drawing shift and also remains available as a statement. `eval` is retained for legacy round-tripping. Other node attributes are preserved as node statements.

```
#let src = ```dot
digraph { a [shift="0.1,0", eval=legacy, label=A]; }
```
#let n = nodes(parse(src.text).first()).first()
#n.shift
#n.statements
```

Edge handling:

- `id=&lt;n&gt;` is consumed as the edge index. It is not preserved as a statement; drawing callbacks expose the resulting stable edge index as `eid`. Explicit edge indexes must be unique and in bounds. Without explicit ids, edge order is parser-internal, not DOT input order.

```
#let src = ```dot
digraph { a -> b [id=1, label=later]; b -> c [id=0, label=first]; }
```
#let parsed-edges = edges(parse(src.text).first())
#parsed-edges.map(e => e.edge)
#parsed-edges.map(e => e.statements.label)
#parsed-edges.map(e => e.statements.at("id", default: none))
```

- `dir=forward`, `dir=back`, and `dir=none` become edge orientations `"default"`, `"reversed"`, and `"undirected"`. If omitted, directed DOT edges are `"default"` and undirected DOT edges are `"undirected"`.

```
#let src = ```dot
digraph { a -> b [id=0]; b -> c [id=1, dir=back]; c -> d [id=2, dir=none]; }
```
#edges(parse(src.text).first()).map(e => e.orientation)
```

- `source="..."` and `sink="..."` are consumed as endpoint `statement` values and removed from edge statements. On an external edge, use the attribute matching the real endpoint side in the DOT edge: `ext -> a` uses `sink=...`, while `a -> ext` uses `source=...`.
```

````
#let src = ```dot
digraph { ext [style=invis]; ext -> a [id=0, sink=in]; a -> ext [id=1, source=out]; }
```
#let endpoint-statement(endpoint) = if endpoint == none { none } else
{ endpoint.at("statement", default: none) }
#let parsed-edges = edges(parse(src.text).first())
#parsed-edges.map(e => endpoint-statement(e.source))
#parsed-edges.map(e => endpoint-statement(e.sink))
#parsed-edges.map(e => e.statements.at("source", default: none))
````

- `pos` is parsed as the edge control-point placement. `pin` is an explicit edge layout constraint. `shift`, `label-pos`, `label-angle`, and `bend` are parsed into edge geometry fields; except for `pin`, they also remain available as statements. The `pos-x-set` and `pos-y-set` record fields identify explicitly supplied axes, including partial placements; `dot` preserves them as canonical Linnest DOT extension attributes. Other edge attributes are preserved as edge statements.

````
#let src = ```dot
digraph { a -> b [id=0, pos="0,1!", pin="x:@edge", shift="0.1,0", "label-
pos"="0,1.2", "label-angle"="0.3rad", bend="0.4rad", particle=g]; }
```
#let e = edges(parse(src.text).first()).first()
#e.pos
#e.shift
#e.label-pos
#e.label-angle
#e.bend
#e.statements.particle
#e.statements.at("pin", default: none)
````

- Attributes copied from an invisible endpoint node are merged into the external edge statements unless their keys are `shape` or `label`.

````
#let src = ```dot
digraph { ext [style=invis, column=left, shape=none, label=skip]; ext -> a [id=0]; }
```
#edges(parse(src.text).first()).first().statements
````

Port and half-edge handling:

- DOT ports of the form `node:port` and `node:port:compass` are preserved on source/sink endpoint records. Numeric ports also assign explicit half-edge ids. Non-numeric ports are exposed only as `port-label`.

````
#let src = ```dot
digraph { a:left:e -> b:0:w [id=0]; }
```
#let e = edges(parse(src.text).first()).first()
#e.source.port-label
#e.sink.hedge
````

- Compass values are exposed as `compass`; valid DOT compass names include `n`, `ne`, `e`, `se`, `s`, `sw`, `w`, `nw`, `c`, and `_`.

````
#let src = ```dot
digraph { a:n -> b:sw [id=0]; }
```
````
````

```
#let e = edges(parse(src.text).first()).first()
#e.source.compass
#e.sink.compass
```

- Explicit half-edge ids from numeric ports must be unique and in bounds. They are retained for half-edge ordering and for `join` with `key: "id"`; query and eval records expose the resulting half-edge index as `hedge`.

```
#let src = ```dot
digraph { a:1 -> b:0 [id=0]; }
```
#let e = edges(parse(src.text).first()).first()
#e.source.hedge
#e.sink.hedge
```

Placement fields:

- `pos="x,y"` gives a starting coordinate; `pos="x,y!"` pins both axes.

```
#let src = ```dot
digraph { a [id=0, pos="0,0"]; b [id=1, pos="2,0!"]; a -> b; }
```
#let parsed = parse(src.text).first()
#nodes(parsed).map(n => n.pos)
#nodes(layout(parsed, steps: 10, seed: 1)).map(n => n.pos)
```

- `pos="ref(node:<id>)+dx,dy!"` and `pos="ref(edge:<id>)+dx,dy!"` reference explicit DOT node or edge ids, not names and not implicit parse order.

```
#let src = ```dot
digraph { a [id=0, pos="1,1!"]; b [id=1, pos="ref(node:0)+2,0!"]; a -> b [id=0,
pos="ref(node:1)+0,1!"]; }
```
#let g = parse(src.text).first()
#nodes(g).at(1).pos
#edges(g).first().pos
```

- Axis form accepts `x:<coord>` and `y:<coord>` entries. `!` applies to the individual axis, for example `pos="x:2!,y:0"`.

```
#let src = ```dot
digraph { a [id=0, pos="x:2!,y:0"]; }
```
#nodes(parse(src.text).first()).first().pos
```

- Group coordinates use `@name`, `@+name`, or `@-name` in axis form and must be pinned with `!`, for example `pos="x:@-left!,y:@row!"`.

```
#let src = ```dot
digraph { ext [style=invis]; ext -> a [id=0, pos="x:@-left!,y:@row!"]; }
```
#edges(layout(parse(src.text).first(), layout-algo: "tree")).first().pos
```

- `pin` accepts numeric point constraints, `x:<coord>`, `y:<coord>`, and grouped constraints such as `x:@left`, `x:@+right`, or `@row`.
```

```
#let src = ```dot
digraph { a -> b [id=0, pin="x:@+right"]; }
```
#edges(layout(parse(src.text).first(), layout-algo:
"tree")).first().statements.at("pin", default: none)
```

#### 15.2.3.1 Parameters

```
parse(
  input: string bytes,
  default-node-data: any,
  eval-node-fields: string array,
  default-edge-data: any,
  eval-edge-fields: string array,
  default-source-data: any,
  eval-source-fields: string array,
  default-sink-data: any,
  eval-sink-fields: string array,
  eval-graph-fields: string array,
  eval-mode: string,
  scope: dictionary
) -> array
```

15.2.3.1.1 input `string` or `bytes`

DOT source text containing one or more `graph` or `digraph` definitions.

15.2.3.1.2 default-node-data `any`

Default data merged into every node data. Captured node data fields override it, but bare dot statements don't set data fields. To turn statements into data fields, use `eval-node-fields`.

```
#let a = ```dot
digraph {
a -> b -> c -> d -> a
a -> a
b -> d
c [particle="q"]
}
```
#let g = parse(a.text,default-node-data:(particle:"g")).at(0)
#nodes(g).map(n=>n.data.particle)
```

Default: `none`

15.2.3.1.3 eval-node-fields `string` or `array`

Node fields to evaluate into node data. The eval scope includes node statements plus `node`, `name`, and `pos`.

````
#let src = ```dot
digraph { a [label="#str(name)"]; }
```
#nodes(parse(src.text, eval-node-fields: "label").first()).first().data.label
````

Default: `()`

15.2.3.1.4 default-edge-data `any`

Default data merged into every edge data. Captured edge data fields override it, but bare dot statements don't set data fields. To turn statements into data fields, use `eval-edge-fields`.

Default: `none`

15.2.3.1.5 eval-edge-fields `string` or `array`

Edge statement fields to evaluate into `graph.edges(g).at(i).data`. The eval scope includes edge statements plus `edge`, `orientation`, endpoint records, placement fields, and existing edge data. Drawing callbacks later expose the edge index as `eid`.

````
#let src = ```dot
digraph { a -> b [id=0, label="#orientation"]; }
```
#edges(parse(src.text, eval-edge-fields: "label").first()).first().data.label
````

Default: `()`

15.2.3.1.6 default-source-data `any`

Default data merged into every source half-edge data. Captured source data fields override it.

Default: `none`

15.2.3.1.7 eval-source-fields `string` or `array`

Source half-edge fields to evaluate into `edge.source.data`. The eval scope includes source endpoint fields `statement`, `port-label`, `compass`, `node`, and `hedge`, plus the surrounding edge fields.

````
#let src = ```dot
digraph { a:left:e -> b [id=0, source="#port-label"]; }
```
#edges(parse(src.text, eval-source-fields:
"statement").first()).first().source.data.statement
````

Default: `()`
````

15.2.3.1.8 default-sink-data `any`

Default data merged into every sink half-edge data. Captured sink data fields override it.

Default: `none`

15.2.3.1.9 eval-sink-fields `string` or `array`

Sink half-edge fields to evaluate into `edge.sink.data`. The eval scope includes sink endpoint fields `statement`, `port-label`, `compass`, `node`, and `hedge`, plus the surrounding edge fields.

```
#let src = ```dot
digraph { a -> b:0:w [id=0, sink="#str(hedge)"]; }
```
#edges(parse(src.text, eval-sink-fields:
"statement").first()).first().sink.data.statement
```

Default: `()`

#### 15.2.3.1.10 eval-graph-fields `string` or `array`

Graph statement fields to evaluate into `graph.info(g).data`. The eval scope includes graph statements and direct graph record fields.

```
#let src = ```dot
digraph {
  title = "Strong graph";
}
```
#info(parse(src.text, eval-graph-fields: "title").first()).data.title
```

Default: `()`

15.2.3.1.11 eval-mode `string`

Typst `eval` mode used for string field values.

Default: `"markup"`

15.2.3.1.12 scope `dictionary`

Additional Typst names available while evaluating field values.

Default: `(:)`

15.2.4 node

Create a graph node item for `build`.

A Typst label is the node name used by `source`, `sink`, and `pos`. The optional numeric `id` fixes the resulting graph node index. Extra named arguments are captured as node data fields, so `node(<a>, label: [A], color: red)` stores `(label: [A], color: red)`. The default draw style uses `data.label` as the visible node label when present.

15.2.4.1 Parameters

```
node(
  ..args: label,
  name: none label,
  id: none int,
  pos: none dictionary,
  shift: none string array dictionary,
  statements: dictionary
) -> array
```

15.2.4.1.1 ..args `label`

Optional positional node name. Must be a Typst label when provided; extra named arguments become data fields.

15.2.4.1.2 name `none` or `label`

Typst node name for references.

Default: none

15.2.4.1.3 id `none` or `int`

Numeric graph node index. Must be unique and in bounds when provided.

Default: none

15.2.4.1.4 pos `none` or `dictionary`

Node placement.

Default: none

15.2.4.1.5 shift `none` or `string` or `array` or `dictionary`

Drawing shift stored as a statement.

Default: none

15.2.4.1.6 statements `dictionary`

Additional flat node statements. Used by DOT; values cannot nest.

Default: `(:)`

15.2.5 source

Create a source half-edge endpoint.

`node` may be a node name like `<a>` or a numeric node index. `name` gives the half-edge a Typst name; `id` is a numeric half-edge order/index override. Extra named arguments are captured as source data fields.

15.2.5.1 Parameters

```
source(
  node: label int,
  ..args: any,
  name: none label,
  id: none int,
  statement: none string,
  compass: none string
) -> dictionary
```

15.2.5.1.1 node `label` or `int`

Referenced node, either by Typst label name or numeric node index.

15.2.5.1.2 ..args `any`

Extra named arguments become source data fields.

15.2.5.1.3 name `none` or `label`

Optional half-edge name used for later references.

Default: `none`

15.2.5.1.4 id `none` or `int`

Optional numeric half-edge index/order override.

Default: `none`

15.2.5.1.5 statement `none` or `string`

DOT-ish flat statement used for matching/joining dangling half edges.

Default: `none`

15.2.5.1.6 compass `none` or `string`

DOT compass point such as `"n"`, `"s"`, `"e"`, or `"w"`.

Default: `none`

15.2.6 sink

Create a sink half-edge endpoint.

`node` may be a node name like `<a>` or a numeric node index. `name` gives the half-edge a Typst name; `id` is a numeric half-edge order/index override. Extra named arguments are captured as sink data fields.

15.2.6.1 Parameters

```
sink(
  node: label int,
  ..args: any,
  name: none label,
  id: none int,
  statement: none string,
  compass: none string
) -> dictionary
```

15.2.6.1.1 node `label` or `int`

Referenced node, either by Typst label name or numeric node index.

15.2.6.1.2 ..args `any`

Extra named arguments become sink data fields.

15.2.6.1.3 name `none` or `label`

Optional half-edge name used for later references.

Default: `none`

15.2.6.1.4 id `none` or `int`

Optional numeric half-edge index/order override.

Default: `none`

15.2.6.1.5 statement `none` or `string`

DOT-ish flat statement used for matching/joining dangling half edges.

Default: `none`

15.2.6.1.6 compass `none` or `string`

DOT compass point such as `"n"`, `"s"`, `"e"`, or `"w"`.

Default: `none`

15.2.7 edge

Create a graph edge item for `build`.

Positional arguments may contain one `source`, one `sink`, and optionally one Typst label used as the edge name. The numeric `id` chooses the edge order. Extra named arguments are captured as edge data fields, so `edge(source(<a>), sink(<b>), weight: 3)` stores `(weight: 3)`. `style` is first-class drawing metadata: `auto` asks `draw` for its configured/default style, a dictionary or callback patches that style, and `none` hides the painted edge. The default draw style uses `data.label` as the visible edge label when present.

15.2.7.1 Parameters

```
edge(
  ..args: any,
  name: none label,
  id: none int,
  orientation: string,
  pos: none dictionary,
  shift: none string array dictionary,
  label-pos: none string array dictionary,
  label-angle: none int float string,
  bend: none int float string,
  spring-length: none int float,
  style: auto dictionary array function none,
  statements: dictionary
) -> array
```

15.2.7.1.1 ..args `any`

Source/sink half-edges and optional edge name; extra named arguments become data fields.

15.2.7.1.2 name `none` or `label`

Typst edge name.

Default: `none`

15.2.7.1.3 id `none` or `int`

Numeric edge order/index override.

Default: `none`

15.2.7.1.4 orientation `string`

Edge orientation: `"default"`, `"reversed"`, or `"undirected"`.

Default: `"default"`

15.2.7.1.5 pos `none` or `dictionary`

Edge placement.

Default: `none`

15.2.7.1.6 shift `none` or `string` or `array` or `dictionary`

Drawing shift stored as a statement.

Default: `none`

15.2.7.1.7 label-pos `none` or `string` or `array` or `dictionary`

Edge label position stored as a statement.

Default: `none`

15.2.7.1.8 label-angle `none` or `int` or `float` or `string`

Edge label angle stored as a statement.

Default: `none`

15.2.7.1.9 bend `none` or `int` or `float` or `string`

Edge bend stored as a statement.

Default: `none`

15.2.7.1.10 spring-length `none` or `int` or `float`

Positive finite dimensionless multiplier of each incidence's preferred spring length in force/anneal layout, including dangling-edge doubling. `none` preserves statements/defaults; a missing statement means `1`. Does not set rendered length or change global `spring.length`.

Default: `none`

15.2.7.1.11 style `auto` or `dictionary` or `array` or `function` or `none`

Static logical-edge style, callback, `auto` fallback, or `none` to hide.

Default: `auto`

15.2.7.1.12 statements `dictionary`

Additional flat edge statements. Used by DOT; values cannot nest.

Default: `(:)`

15.2.8 group

Create a grouped placement coordinate.

`side: "+"` keeps the solved coordinate non-negative and `side: "-"` keeps it non-positive. `start` supplies an initial value without fixing the coordinate; multiple starts for one group are averaged. Groups are layout constraints and therefore require pin placement, which is the `pos` default. Node and edge coordinates with the same group name and side share one axis coordinate throughout layout.

```
#group("right", side: "+", start: 4)
```

15.2.8.1 Parameters

```
group(
  name: string int bool,
  side: none string,
  start: none int float
) -> dictionary
```

15.2.8.1.1 name `string` or `int` or `bool`

Group identifier shared by positions constrained to the same coordinate.

15.2.8.1.2 side `none` or `string`

Optional sign constraint: `"+"`, `"-"`, `"positive"`, or `"negative"`.

Default: `none`

15.2.8.1.3 start `none` or `int` or `float`

Optional initial coordinate; unlike `pin`, this remains movable.

Default: `none`

15.2.9 pin

Mark one coordinate as a layout constraint.

15.2.9.1 Parameters

`pin(value: int float dictionary) -> dictionary`

15.2.9.1.1 value `int` or `float` or `dictionary`

Coordinate value to constrain.

15.2.10 start

Mark one coordinate as an initial layout value only.

15.2.10.1 Parameters

`start(value: int float) -> dictionary`

15.2.10.1.1 value `int` or `float`

Coordinate value to use as the layout seed.

15.2.11 pos

Create a first-class graph placement.

The default `mode: "pin"` turns numeric coordinates into layout constraints and also makes the XY coordinates immediately drawable without a layout pass. Use `start(value)` for an individual coordinate that should only seed the layout, or `pin(value)` to pin an individual numeric coordinate when `mode: "start"` is used. Grouped coordinates are always layout constraints for their axis.

`z` is auxiliary depth for force-based layout only, in the same numeric layout units as x and y (not Typst lengths). Use `pos(z: pin(2))` for a hard raw-depth pin or `pos(z: start(2))` for a movable initial depth. A bare `pos(z: 2)` follows `mode`, defaulting to `"pin"`. The global depth scale and flattening schedule may reduce effective depth to zero without changing a hard raw-depth pin. This is not a rendered third coordinate: node/edge output `pos` and drawing remain 2D. Statements retain the configured raw pin/seed, not the solved free depth, as `"pos-z"` (finite numeric text) and `"pos-z-mode"` (`"pin"` or `"start"`). Other layout backends do not use z.

Both `build` and `map` accept z placements on nodes and edge control points. A z-only placement leaves XY placement and constraints alone, including automatic edge midpoint seeds. Omitting z preserves existing depth metadata. Z groups, relative depth references, `dz`, and non-finite values are not supported; `ref`, `dx`, and `dy` continue to affect XY only.

```
#pos(x: group("right", side: "+"), y: start(10), z: pin(2))
```

15.2.11.1 Parameters

```
pos(
  x: none int float dictionary,
  y: none int float dictionary,
  z: none int float dictionary,
  ref: none label int,
  dx: none int float,
  dy: none int float,
  mode: string
) -> dictionary
```

15.2.11.1.1 x `none` or `int` or `float` or `dictionary`

Absolute or grouped x coordinate.

Default: `none`

15.2.11.1.2 y `none` or `int` or `float` or `dictionary`

Absolute or grouped y coordinate.

Default: `none`

15.2.11.1.3 z `none` or `int` or `float` or `dictionary`

Auxiliary force-layout depth, numeric or wrapped in `pin` / `start`.

Default: `none`

15.2.11.1.4 ref `none` or `label` or `int`

Node reference for relative XY placement, by name or numeric index.

Default: `none`

15.2.11.1.5 dx `none` or `int` or `float`

Relative x offset from `ref`.

Default: `none`

15.2.11.1.6 dy `none` or `int` or `float`

Relative y offset from `ref`.

Default: `none`

15.2.11.1.7 mode `string`

Placement mode: `"pin"` constrains layout, `"start"` only seeds it.

Default: `"pin"`

15.2.12 map

Map graph records with native-data and structural patches. Position patches merge supplied axes: omitted coordinates retain their existing values, pin/start modes and coordinate groups.

`node` and `edge` accept either a callback for every record or a name-keyed dictionary. Dictionary keys are exact string names: `a` matches `<a>`, and `"D1.0"` matches the `<D1.0>` fragment produced by `cut`. Each entry is a constant patch dictionary, a callback, or `none`. Unlisted records and unnamed records are skipped by name-keyed dictionaries. Every supplied name must exist, even when its entry is `none`. Unknown names and invalid mapper, entry, or callback return types are errors. There are no numeric-ID keys, array selectors, wildcard/default selectors, or recursive merges. `graph`, `source`, and `sink` still accept only a callback or `none`.

The callbacks receive the same full decoded records in either form, plus a `fields` dictionary merging native dictionary data, statements, and direct record fields, with later sources taking precedence. A callback returns `none` to leave the record unchanged, `(data: value)` to set new native data, or structural fields such as `pos`, `shift`, `spring-length`, and `statements` to patch data seen by later layout calls. `spring-length` is validated as a positive finite multiplier and stored in the edge statements used by the layout engine. Structural patches are applied separately from native Typst data, so content and stored functions in the existing `data` value are preserved. Unchanged fields in a returned full record are not reapplied; this preserves partial XY placements when patching z or data. Source and sink callbacks may likewise patch `statement`, `port-label`, and `compass` before subgraph and layout operations run.

Constant patches and callback results use identical semantics. Node structural keys are `pos`, `shift`, and `statements`; edges also accept `label-pos`, `label-angle`, `bend`, and `spring-length`. With no `data` key, other keys shallow-merge into existing native dictionary data (or replace non-dictionary data with that patch). An explicit `data` key instead replaces native data, ignoring other non-structural keys; `data: none` leaves native data unchanged. Native Typst content and stored functions are preserved, not serialized or invoked. A mapper or entry of `none`, an empty name map, an empty patch, or a callback returning `none` leaves the corresponding records unchanged.

```
#let g = build({ node(<a>) })
#let g = map(g, node: node => (data: (label: [A])))
#nodes(g).first().data.label
```

```
#let g = build({
  node(<a>); node(<b>)
  edge(<D1>, source(<a>), sink(<b>), weight: 2)
})
```

```
#let g = map(g,
  node: (a: (label: [A]), b: none),
  edge: (D1: e => (weight: e.fields.weight + 1)),
)
#edge-data(g, <D1>).weight
```

15.2.12.1 Parameters

```
map(
  graph_: dictionary,
  graph: none function,
  node: none function dictionary,
  edge: none function dictionary,
  source: none function,
  sink: none function
) -> dictionary
```

15.2.12.1.1 graph_ `dictionary`

Graph object to transform.

15.2.12.1.2 graph `none` or `function`

Callback for graph metadata records.

Default: `none`

15.2.12.1.3 node `none` or `function` or `dictionary`

Callback for all node records, or name-keyed patches/callbacks/no-ops.

Default: `none`

15.2.12.1.4 edge `none` or `function` or `dictionary`

Callback for all edge records, or name-keyed patches/callbacks/no-ops.

Default: `none`

15.2.12.1.5 source `none` or `function`

Callback for source half-edge records.

Default: `none`

15.2.12.1.6 sink `none` or `function`

Callback for sink half-edge records.

Default: `none`

15.2.13 style

Attach layout-relevant drawing style to a graph.

Node style is measured immediately and stored as `layout-width` / `layout-height` statements for later layout calls. Edge labels are measured as `label-width` / `label-height` statements for label placement. `draw` uses the stored node, edge, endpoint, and label styles by default.

15.2.13.1 Parameters

```
style(
  graph_: dictionary,
  scope: dictionary,
  unit: int float length ratio,
  node-label: auto content string function none,
  node-label-style: dictionary function,
  node-style: dictionary function none,
  edge-style: dictionary array function none,
  source-style: dictionary array function none,
  sink-style: dictionary array function none,
  edge-label: content string function none,
  edge-label-style: dictionary function
) -> dictionary
```

15.2.13.1.1 graph_ `dictionary`

Graph object to style.

15.2.13.1.2 scope `dictionary`

Extra scope visible to label/style callbacks.

Default: `(:)`

15.2.13.1.3 unit `int` or `float` or `length` or `ratio`

Coordinate length for one graph-layout unit. Numbers are interpreted as em.

Default: `1`

15.2.13.1.4 node-label `auto` or `content` or `string` or `function` or `none`

Node label content or callback. `auto` uses node data/statement label or node name.

Default: `auto`

15.2.13.1.5 node-label-style `dictionary` or `function`

CeTZ content style for node labels. Its padding contributes to measured node size.

Default: `(:)`

15.2.13.1.6 node-style `dictionary` or `function` or `none`

CeTZ node shape style or callback. Explicit numeric radii contribute to measured node size.

Default: `(:)`

15.2.13.1.7 edge-style `dictionary` or `array` or `function` or `none`

Logical-edge style dictionary, layers, or callback used as drawing defaults. Explicit draw and record styles override these defaults. `none` hides edges.

Default: `(:)`

15.2.13.1.8 source-style `dictionary` or `array` or `function` or `none`

Source endpoint defaults, composed before explicit draw and record styles.

Default: `(:)`

15.2.13.1.9 sink-style `dictionary` or `array` or `function` or `none`

Sink endpoint defaults, composed before explicit draw and record styles.

Default: `(:)`

15.2.13.1.10 edge-label `content` or `string` or `function` or `none`

Edge label content or callback. `none` leaves edge labels unstyled and unmeasured.

Default: `none`

15.2.13.1.11 edge-label-style `dictionary` or `function`

CeTZ content style for edge labels. Its padding contributes to measured edge-label size.

Default: `(:)`

15.2.14 eval-fields

Evaluate selected fields into native data entries.

Each selected field is read from the record's merged `fields` dictionary, evaluated in a scope containing those fields, and written to `data.<field>`.

```
#let g = build({
  node(<a>)
  node(<b>)
  edge(source(<a>), sink(<b>), statements: (mom: "p"))
```

```
}, default-edge-statements: (display-label: "$#mom$"))
#let g = eval-fields(g, eval-edge-fields: ("display-label",))
#edges(g).first().data.at("display-label")
```

15.2.14.1 Parameters

```
eval-fields(
  graph_: dictionary,
  eval-graph-fields: string array,
  eval-node-fields: string array,
  eval-edge-fields: string array,
  eval-source-fields: string array,
  eval-sink-fields: string array,
  eval-mode: string,
  scope: dictionary
) -> dictionary
```

15.2.14.1.1 graph_ `dictionary`

Graph object whose selected statement fields should be evaluated.

15.2.14.1.2 eval-graph-fields `string` or `array`

Graph statement fields to evaluate into `graph.info(g).data`.

Default: `()`

15.2.14.1.3 eval-node-fields `string` or `array`

Node statement fields to evaluate into node data.

Default: `()`

15.2.14.1.4 eval-edge-fields `string` or `array`

Edge statement fields to evaluate into edge data.

Default: `()`

15.2.14.1.5 eval-source-fields `string` or `array`

Source half-edge fields to evaluate into source data.

Default: `()`

15.2.14.1.6 eval-sink-fields `string` or `array`

Sink half-edge fields to evaluate into sink data.

Default: `()`

15.2.14.1.7 eval-mode `string`

Typst `eval` mode used for string field values.

Default: `"markup"`

15.2.14.1.8 scope `dictionary`

Additional Typst names available while evaluating field values.

Default: `(:)`

15.2.15 info

Return graph metadata.

The result has `name`, `global-statements`, `default-edge-statements`, and `default-node-statements`.

```
#let g = build({ node(<a>) }, name: "demo")
#info(g).name
```

15.2.15.1 Parameters

`info(graph: dictionary) -> dictionary`

15.2.15.1.1 graph `dictionary`

Graph object returned by `build`, `parse`, `layout`, or another graph API.

15.2.16 dot

Serialize a graph object to DOT.

```
#let g = build({
  node(<a>)
  node(<b>)
  edge(source(<a>), sink(<b>))
}, name: "demo")
#dot(g).contains("digraph demo")
```

15.2.16.1 Parameters

`dot(graph: dictionary) -> string`

15.2.16.1.1 graph `dictionary`

Graph object to serialize.

15.2.17 nodes

Return node records, optionally filtered by a subgraph object.

Node `name` values are Typst labels when present.

```
#let g = build({
  node(<a>)
  node(<b>)
})
#nodes(g).map(node => str(node.name)).join(", ")
```

15.2.17.1 Parameters

```
nodes(
  graph: dictionary,
  subgraph: none dictionary
) -> array
```

15.2.17.1.1 graph `dictionary`

Graph object to inspect.

15.2.17.1.2 subgraph `none` or `dictionary`

Optional subgraph filter; only nodes incident to selected half edges are returned.

Default: `none`

15.2.18 edges

Return edge records, optionally filtered by a subgraph object.

Edge `name` values are Typst labels when present.

```
#let g = build({
  node(<a>)
  node(<b>)
  edge(source(<a>, compass: "e"), sink(<b>))
})
#edges(g).len()
```

15.2.18.1 Parameters

```
edges(
  graph: dictionary,
  subgraph: none dictionary
) -> array
```

15.2.18.1.1 graph `dictionary`

Graph object to inspect.

15.2.18.1.2 subgraph `none` or `dictionary`

Optional subgraph filter; selected edges retain their full endpoint records.

Default: `none`

15.2.19 node-data

Return one named node's native data.

`name` is a Typst label such as `<a>` or the corresponding string name.

```
#let g = build({ node(<a>, label: [A]) })
#node-data(g, <a>).label
```

15.2.19.1 Parameters

```
node-data(
  graph_: dictionary,
  name: label string
) -> any
```

15.2.19.1.1 graph_ `dictionary`

Graph object to inspect.

15.2.19.1.2 name `label` or `string`

Node name as a Typst label or its string form.

15.2.20 edge-data

Return one named edge's native data.

`name` is a Typst label such as `<e>` or the corresponding string name.

```
#let g = build({
  node(<a>)
  node(<b>)
  edge(source(<a>), <e>, sink(<b>), label: [$p$])
})
#edge-data(g, <e>).label
```

15.2.20.1 Parameters

```
edge-data(
  graph_: dictionary,
  name: label string
) -> any
```

15.2.20.1.1 graph_ `dictionary`

Graph object to inspect.

15.2.20.1.2 name `label` or `string`

Edge name as a Typst label or its string form.

15.2.21 update-node-data

Update one named node's native data.

`update` may be a replacement data value or a function `(data, node) => new-data`.

```
#let g = build({ node(<a>) })
#let g = update-node-data(g, <a>, (label: [A]))
#nodes(g).first().data.label
```

15.2.21.1 Parameters

```
update-node-data(
  graph_: dictionary,
  name: label string,
  update: any function
) -> dictionary
```

15.2.21.1.1 graph_ `dictionary`

Graph object to update.

15.2.21.1.2 name `label` or `string`

Node name as a Typst label or its string form.

15.2.21.1.3 update `any` or `function`

Replacement data or `(data, node) => new-data` callback.

15.2.22 update-edge-data

Update one named edge's native data.

`update` may be a replacement data value or a function `(data, edge) => new-data`.

```
#let g = build({
  node(<a>)
  node(<b>)
  edge(source(<a>), <e>, sink(<b>))
})
#let g = update-edge-data(g, <e>, (label: [$p$]))
#edges(g).first().data.label
```

15.2.22.1 Parameters

```
update-edge-data(
  graph_: dictionary,
  name: label string,
  update: any function
) -> dictionary
```

15.2.22.1.1 graph_ `dictionary`

Graph object to update.

15.2.22.1.2 name `label` or `string`

Edge name as a Typst label or its string form.

15.2.22.1.3 update `any` or `function`

Replacement data or `(data, edge) => new-data` callback.

15.2.23 cut

Open paired edges along a weighted directed half-edge cut.

`left` and `right` are disjoint subgraphs containing opposite halves of each cut edge. Their names identify boundary sides, not underlying source/sink flow or superficial drawing orientation. A half-edge annotation `(winding: n)` requests `n` same-direction seam passages; absent winding defaults to one. If both sides specify a winding they must agree. Net counts of cancelling recrossings are not a supported description of an opening.

The input is unchanged. Every cut edge yields source and sink stubs plus `n - 1` middle segments on auxiliary, unpainted boundary nodes. Fragment names are `<original.0>` through `<original.n>` in underlying source-to-sink order. Unnamed edges use `__linnest_cut_edge_ID` as the prefix; name collisions are errors. Cut geometry is reset for re-layout, while physical data and orientation are retained. Arbitrary Typst data, including content and callbacks, is remapped without serialization. Generated middle endpoint hedges inherit the data of the opposite original hedge, corresponding to the joined boundary side.

Output node/edge/half-edge records have `origin` relative to the input graph. Edge origins contain `(edge, name, segment, winding)`; uncut segments are `none`. Generated nodes have no original node. Boundary nodes and dangling cut edges also have `boundary`: `(edge, node, hedge, side, crossing, data, cut-data, origin)`, where `data` is the side's hedge annotation and `cut-data` its subgraph-wide data. `crossing` counts from zero along underlying flow. Boundary `hedge` and `origin` describe the input to the cut that created that boundary, even after subsequent cuts; `edge` and `node` always locate its current anchor. This differs from each record's immediate-input `origin`. The optional callback patches only endpoints created by this operation, exactly like `map`; earlier boundary placements and annotations are retained.

```
#let g = build({
  node(<a>)
  node(<b>)
  edge(source(<a>), <e>, sink(<b>), label: [$k$])
})
#let left = subgraph.select(g, sink: (<e>,))
#let left = subgraph.with-data(g, left, hedge: (winding: 2))
#let right = subgraph.select(g, source: (<e>,))
#let opened = cut(g, left: left, right: right)
#edges(opened).len()
```

15.2.23.1 Parameters

```
cut(
  graph_: dictionary,
  left: dictionary,
  right: dictionary,
  boundary: none dictionary function
) -> dictionary
```

15.2.23.1.1 graph_ `dictionary`

Graph to open, without modifying it.

15.2.23.1.2 left `dictionary`

Selected left half-edges, optionally annotated with winding.

Default: `none`

15.2.23.1.3 right `dictionary`

Involution partners defining the right side.

Default: `none`

15.2.23.1.4 boundary `none` or `dictionary` or `function`

Placement/data patch for each newly created boundary node or dangling edge.

Default: `none`

15.2.24 boundaries

Inspect all generated cut endpoints with their current solved positions.

Returns the boundary records described by `cut` plus `pos`. For a dangling endpoint, `node` is `none` and `pos` is its edge position; for a middle segment endpoint it is the auxiliary node position. This makes boundary annotations independent of their internal node/edge representation.

15.2.24.1 Parameters

```
boundaries(graph_: dictionary) -> array
```

15.2.24.1.1 graph_ `dictionary`

Graph returned by `cut`, optionally styled, patched, or laid out.

15.2.25 join

Join two graphs by matching dangling half-edge metadata on `key`.

Supported key values are `"statement"`, `"compass"`, `"port-label"`, and `"id"`. The join is partial: unmatched dangling half-edges remain. Matching identities must be unique within each flow, and node/edge names must not collide across the inputs. Native Typst data is preserved; when both sides provide a dictionary, right-only fields are retained and left-side fields take precedence.

```
#let left = build({
  node(<a>)
  edge(sink(<a>, statement: "j"))
})
#let right = build({
  node(<b>)
  edge(source(<b>, statement: "j"))
})
#edges(join(left, right, key: "statement")).len()
```

15.2.25.1 Parameters

```
join(
  left: dictionary,
  right: dictionary,
  key: string
) -> dictionary
```

15.2.25.1.1 left `dictionary`

Left graph object.

15.2.25.1.2 right `dictionary`

Right graph object.

15.2.25.1.3 key `string`

Dangling half-edge match key: `"statement"`, `"compass"`, `"port-label"`, or `"id"`.

Default: `"statement"`

15.2.26 join-edges

Join two graphs by matching dangling edge records on an edge statement key.

This is distinct from graph.join: it matches edge-level statements rather than half-edge metadata. The same partial-join, collision, and data-merge rules apply.

15.2.26.1 Parameters

```
join-edges(
  left: dictionary,
  right: dictionary,
  key: string
) -> dictionary
```

15.2.26.1.1 left `dictionary`

Left graph object.

15.2.26.1.2 right `dictionary`

Right graph object.

15.2.26.1.3 key `string`

Nonempty edge statement key used for matching.

Default: `"statement"`

15.2.27 cycles

Return subgraph objects for the graph's cycle basis.

```
#let g = build({
  node(<a>)
  node(<b>)
  edge(source(<a>), sink(<b>))
})
#cycles(g).len()
```

15.2.27.1 Parameters

`cycles(graph: dictionary) -> array`

15.2.27.1.1 graph `dictionary`

Graph object to analyze.

15.2.28 forests

Return subgraph objects for the graph's spanning forests.

```
#let g = build({
  node(<a>)
  node(<b>)
  edge(source(<a>), sink(<b>))
})
#forests(g).len()
```

15.2.28.1 Parameters

`forests(graph: dictionary) -> array`

15.2.28.1.1 graph `dictionary`

Graph object to analyze.

16 Layout functions

`layout` and `layout-sequence` are top-level functions exported by `lib.typ`, not members of a public `layout` module. Their generated entries come from the shared `layout.typ` implementation and use the public export names.

16.1 Concepts

16.1.1 Placements

`graph.pos` creates a first-class placement. The default `mode: "pin"` turns a coordinate into a fixed layout constraint and, for x/y, a drawable position. Use `mode: "start"` when the coordinate should only seed the layout, or use `graph.pin(value)` / `graph.start(value)` to choose a mode for one axis:

```
#let g = build({
  node(<a>, pos: graph.pos(x: -2, y: 0))
  node(<c>, pos: graph.pos(ref: <a>, dx: 4, dy: 0))
  edge(source(<a>), <a-c>, sink(<c>), pos: graph.pos(x: 0, y: 1.2))
})
```

Auxiliary `z` is force-layout depth in the same numeric layout units as x/y, not a Typst length or a rendered coordinate. Both `graph.build` and `graph.map` accept `graph.pos(z: graph.pin(2))` for a hard raw-depth pin and `graph.pos(z: graph.start(2))` for a movable initial depth. A bare `graph.pos(z: 2)` follows `mode`, defaulting to `"pin"`:

```
#let g = graph.map(g,
  node: node => (pos: graph.pos(z: graph.pin(2))),
  edge: edge => (pos: graph.pos(z: graph.start(-1))),
)
```

A z-only placement leaves XY coordinates, constraints, and partial-position flags alone, including automatic edge midpoint seeds. Omitting z preserves existing depth metadata during unrelated XY patches. `ref`, `dx`, and `dy` affect XY only; z groups, relative depth references, `dz`, and non-finite z values are not supported.

Node and edge `statements` expose `"pos-z"` as finite numeric text and `"pos-z-mode"` as `"pin"` or `"start"`. These describe the supplied raw pin or seed; output `pos` remains a two-coordinate x/y record. The force solver multiplies raw depth by a global scale that flattens to zero: a hard raw pin remains fixed even when its effective depth is zero. Other layout backends do not use z. Auxiliary depth does not change draw order or provide an over/under rendering layer.

`graph.group` links one XY coordinate across several nodes or edge control points. A `side` of "+" keeps the coordinate positive, and "-" keeps it negative. Use `start` to seed the shared coordinate without fixing it. Matching node and edge groups are one degree of freedom during force and annealing layouts, so repulsion and springs act on their combined force rather than being reconciled afterward. If several members supply starts for one group, their mean initializes the shared coordinate. GammaLoop external-edge columns use this to keep incoming and outgoing external legs on opposite sides while pairing rows by a shared `y` group:

```
#let edge-items = (
  edge(
    source(<right-ext>),
    sink(<center>),
    pos: graph.pos(x: graph.group("right", side: "+", start: 4), y: graph.group("edgee0")),
  ),
  edge(
    source(<center>),
    sink(<left-ext>),
    pos: graph.pos(x: graph.group("left", side: "-"), y: graph.group("edgee0")),
  ),
)
```

Raw DOT input uses the same placement model through the `pos` attribute. The standard Graphviz subset is preserved: `pos="x,y"` is a starting coordinate and `pos="x,y!"` is pinned. Linnest extends the value with explicit id references and axis entries. In axis entries, `!` belongs to that axis: numeric entries without `!` are starting

coordinates, numeric entries with `!` are fixed constraints, and grouped entries must use `!` because groups are constraints.

```
digraph {
  a [id=0 pos="0,0!"]
  b [id=1 pos="ref(node:0)+4,0!"]
  a -> b [id=0 pos="ref(node:1)+0,1!"]
  b -> c [id=1 pos="ref(edge:0)+1,0!"]
  c [id=2 pos="x:2!"]
  d [id=3 pos="x:2!,y:1"]
  ext -> a [id=2 pos="x:@-left!,y:@edge0!"]
}
```

The DOT grammar is intentionally id-based: `ref(node:0)` uses a node `id` and `ref(edge:0)` uses an edge `id`. Bare names and implicit edge order are not placement references. The `@` syntax denotes grouped coordinate constraints; `@+name` and `@-name` keep the grouped coordinate on the positive or negative side respectively.

```
pos="x,y"                   // start x and y
pos="x,y!"                  // pin x and y
pos="x:<coord>"             // start numeric x
pos="x:<coord>!"            // pin x
pos="y:<coord>!"            // pin y
pos="x:<coord>!,y:<coord>"  // pin x, start numeric y
pos="x:<coord>,y:<coord>!"  // start numeric x, pin y
```

16.1.2 Layout Model

`layout` starts from a traversal-tree placement, then optimizes the positions of graph nodes and edge control points. The initial tree spacing is

$$L = \lambda \sqrt{\frac{WH}{\max(n, 1)}}$$

,

with horizontal spacing $\tau_x L$ and vertical spacing $\tau_y L$. Here λ is `length-scale`, W is `viewport-w`, H is `viewport-h`, τ_x is `tree-dx`, and τ_y is `tree-dy`. These fields set the geometry scale for both layout modes. Isolated subgraph layout uses the number of selected incident nodes for n; other layouts use the full graph node count.

For deterministic, non-iterative placement, use `layout-algo: "tree"` or `layout-algo: "dot"`. `"tree"` places a traversal forest by levels. `"dot"` uses a directed layered placement for acyclic inputs, falling back to tree placement when the selected edges contain a cycle. Both modes also work on a subgraph:

```
#let g = parse("digraph partial { a -> b; b -> c; c -> d; d -> a }").at(0)
#let tree = graph.forests(g).at(0)
#let g = layout(g, layout-algo: "tree", subgraph: tree)
#draw(g)
```

Only nodes touched by the selected subgraph are placed. Edge control points are then resolved from the current node positions, so edges outside the selected subgraph are drawn as straight lines unless their own position constraints say otherwise.

`layout-algo: "force"` and `layout-algo: "anneal"` also accept `subgraph`. The default `solver.subgraph-mode: "fixed-boundary"` keeps nodes and edge control points outside the selection fixed while retaining their spring, repulsion, centroid, and crossing interactions. Set `solver: (subgraph-mode: "isolated")` to omit unselected half edges, nodes, control points, and labels from those interactions. Selected halves of otherwise paired edges become dangling boundaries of the isolated solver domain. Complement coordinates and labels remain unchanged, and

an empty selection is a no-op. An isolated selection cannot split a grouped coordinate; expand the selection or remove that group.

Set `layout-nodes: "fixed"` to keep every node at its current `pos` for this layout pass and move only edge control points. With `subgraph`, only edges in the selected subgraph are moved; all other edge control points keep their current positions. The fixed-node policy is temporary; the returned graph stores the resulting coordinates as `pos`, but it does not turn them into persistent `pin` constraints. This is useful after one node-placement pass when a later pass should route or relax selected edges without disturbing the node layout:

```
#let g = parse("digraph partial { a [pos=\"0,0\"]; b [pos=\"4,0\"]; c [pos=\"8,0\"]; a ->
b; b -> c }").at(0)
#let first = subgraph.bits(g, (true, true, false, false))
#let g = layout(g, layout-algo: "tree", layout-nodes: "fixed", subgraph: first)
#draw(g)
```

Related controls can be supplied as shallow semantic records. These records only name graph-layout concepts; they do not select a drawing domain or style:

```
#let common = layouts.options(
  spring: (strength: 8, length: 0.5),
  repulsion: (edge-node: 0.05, dangling: 2),
  constraints: (side-strength: 5),
  labels: (steps: 0),
  solver: (algorithm: "force", steps: 50),
)
#let spacious = layouts.options(
  base: common,
  spring: (length: 0.7),
  repulsion: (dangling-centroid: 3),
)
#let g = layout(g, ..spacious)
```

`layouts.options` performs a sparse update: the second spring record above changes `length` without losing the inherited `strength`. Grouped fields override their corresponding flat arguments. Flat arguments remain useful for numerical experiments and controls that do not have a semantic grouping.

The shared spring/charge model uses the following coefficients:

$$c_{\mathrm{vv}} = \beta L^3$$

$$c_{\mathrm{ev}} = \beta \gamma_{\mathrm{ev}} L^3$$

$$c_{\mathrm{ee}} = \beta \gamma_{\mathrm{ee}} L^3$$

$$c_{\mathrm{center}} = \beta g_{\mathrm{center}}$$

$$c_{\mathrm{dangling}} = \beta \gamma_{\mathrm{dangling}} L^3$$

$$c_{\mathrm{dangling\text{-}centroid}} = \beta \gamma_{\mathrm{dangling\text{-}centroid}} L^3$$

The Typst parameter names are `beta` for β, `gamma-ev` for γ_{ev}, `gamma-ee` for γ_{ee}, `g-center` for g_{center}, `gamma-dangling` for $\gamma_{\mathrm{dangling}}$, and `gamma-dangling-centroid` for $\gamma_{\mathrm{dangling\text{-}centroid}}$. The spring stiffness k is `k-spring`, and the softening constant ε is `eps`.

In `layout-algo: "anneal"`, linnest minimizes an energy:

$$E = \sum_{i<j} \frac{1}{2} \frac{c_{\mathrm{vv}}}{d(v_i, v_j) + \varepsilon} + \sum_{i,e} \frac{c_{\mathrm{ev}}}{d(v_i, e) + \varepsilon} + \sum_{(v,e)\ \text{incident}} \frac{1}{2} k(\ell_e - d(v,e))^2$$

$$+ \sum_{\text{local edge pairs}} \frac{1}{2} \frac{c_{\mathrm{ee}}}{d(e_i, e_j) + \varepsilon} + \sum_{\text{dangling pairs}} \frac{1}{2} \frac{c_{\mathrm{dangling}}}{d(e_i, e_j) + \varepsilon} + \sum_{\text{dangling } e} \frac{c_{\text{dangling-centroid}}}{d(e, \overline{v}) + \varepsilon} + \sum_i \frac{1}{2} c_{\mathrm{center}} d(v_i, 0)^2 + p_{\mathrm{cross}} N_{\mathrm{cross}}$$

.

Here $\overline{v}$ is the mean node position. The centroid term pushes every dangling endpoint away from that mean; its equal-and-opposite reaction is shared over the nodes so it introduces no net force. p_{cross} is `crossing-penalty` and N_{cross} is the number of detected edge crossings. The quadratic center term pulls nodes toward the origin at every radius. `temp`, `step`, `seed`, `steps`, `epochs`, `cool`, `accept-floor`, `step-shrink`, and `incremental-energy` belong to this simulated annealing mode. `crossing-penalty` is also anneal-only; force mode does not currently add a crossing force.

In `layout-algo: "force"`, linnest applies the direct forces corresponding to the same vertex-vertex, edge-vertex, incidence spring, local edge-edge, dangling-edge, dangling-centroid, and center terms. `step` is the integration step, `delta` clamps per-step movement, `steps` and `epochs` set the iteration budget, and `cool` shrinks the step after each epoch. `early-tol` can stop the run when movement is small only after the effective depth scale reaches zero; small movement or a cooled-to-zero step cannot skip flattening.

The force-only `depth-scale` (default `1.0`) and `flattening-end` (default `0.5`) control auxiliary depth. `depth-scale` must be finite and non-negative; `flattening-end` must be finite and in `0..=1`. They can be supplied as flat arguments or in `solver: (depth-scale: 1.0, flattening-end: 0.5)`; grouped values override flat ones. The old depth-spring controls are not aliases.

Effective depth is `scale * raw-z`. The integrator starts with auxiliary raw depths to break overlaps and smoothly reduces `scale` from `depth-scale` to zero over the initial `flattening-end` fraction of the total `steps × epochs` iteration budget. For normalized progress `u` from 0 to 1 over that interval, the smoothstep scale is `depth-scale * (1 - u)^2 * (1 + 2 * u)`. The remaining iterations relax projected overlaps with effective depth exactly zero, so repulsion cannot be satisfied by invisible separation. `flattening-end: 0` or `depth-scale: 0` starts in 2D; `flattening-end: 1` still evaluates the final iteration on the exact plane.

There is a single original iteration budget and cooling schedule: flattening neither adds a second phase nor restarts `step`. Unpinned raw depths stay bounded by the initial spread and supplied depth magnitudes; hard raw-depth pins remain fixed even after their effective depth has flattened to zero. XY pins, shared coordinates, and fixed subgraph boundaries apply throughout; label layout runs only after this single graph pass. Output positions remain 2D, and auxiliary z does not alter draw order. Flattening is exact, while force convergence remains limited by the iteration budget and movement tolerance.

`directional-force` is applied in both modes as an extra bias derived from pin/port direction constraints.

After either graph layout mode, labels are relaxed separately. If L_l is the label target distance and q_l is the label repulsion strength, then $L_l = \alpha_l L$ and $q_l = \beta_l L^2$. The Typst names are `label-length-scale` for α_l and `label-charge` for β_l. `label-spring` is the spring constant pulling each label toward its target. `label-layout: "normal"` uses a perpendicular offset target. With `label-layout: "dangling-tangent"`, paired edges still use that perpendicular target, but dangling half-edge labels are offset along the edge direction away from the attached node. With `label-layout: "fixed-length"`, relaxation keeps the label at distance L_l from the edge point and only rotates it under repulsive forces. `label-steps`, `label-step`, `label-early-tol`, and `label-max-delta-scale` control the label relaxation iteration.

`graph.style` measures node and label sizes before layout; those sizes enter repulsion during label relaxation. For labels with measured width and height, a subsequent box-collision pass starts from the relaxed positions. It keeps each position when its box clears node boxes and previously placed label boxes, otherwise searching nearby candidates. This correction may change the distance from the edge point, including in `"fixed-length"` mode.

Repulsion considers edge layout points, and box correction considers nodes and labels; neither guarantees clearance from complete edge curves or drawing layers such as momentum arrows. Attached path labels may also be positioned again during drawing, as described above.

16.2 Qualified aliases

The exported `layouts` module exposes the same implementations for callers that prefer qualified access; these aliases link back to the canonical top-level entries below.

16.2.1 layouts.layout

Use `layout` for the shared signature and parameter reference.

16.2.2 layouts.sequence

Use `layout-sequence` for the shared multi-pass reference.

16.3 Reference

- options()
- layout()
- layout-sequence()

16.3.1 options

Construct reusable semantic layout options or sparsely update existing ones.

Each option group is a plain dictionary. Supplying `base` preserves every field not mentioned by a patch, including fields inside a patched group. Pass the result to `layout` with argument spreading.

```
#let common = options(
  spring: (strength: 8, length: 0.5),
  repulsion: (edge-node: 0.05, dangling: 2),
  solver: (algorithm: "force", steps: 50),
)
#let spacious = options(
  base: common,
  spring: (length: 0.7),
  repulsion: (dangling-centroid: 3),
)
#let g = layout(g, ..spacious)
```

16.3.1.1 Parameters

```
options(
  ..patches: arguments,
  base: dictionary
) -> dictionary
```

16.3.1.1.1 ..patches `arguments`

Semantic option-group patches.

16.3.1.1.2 base `dictionary`

Existing option dictionary to update.

Default: `(:)`

16.3.2 layout

Apply the linnest layout pass to a graph object.

This is intentionally a second step: construct or parse a graph first, then call `layout`. Set `layout-algo` to `"force"` for deterministic force integration, `"anneal"` for simulated annealing, `"tree"` for a traversal tree placement, `"dot"` for a Graphviz-like layered placement, or `"stable-layered"` for a stable railroad-inspired layered placement.

Force layout uses one `steps` × `epochs` iteration budget and one cooling schedule. Effective auxiliary depth is `scale * raw-z`: `scale` follows a smoothstep from `depth-scale` to zero over the initial `flattening-end` fraction of that budget. Remaining iterations are planar, without restarting `step` or cooling. `early-tol` can stop the run only after scale reaches zero. Unpinned raw depths remain bounded; hard raw-depth pins are preserved even when their effective depth is zero. Output positions remain 2D, and auxiliary depth does not change draw order; label layout runs after the single graph pass.

```
#let g = graph.parse("digraph partial { a -> b;a -> b;a:s -> b:s; b:s -> c:s; c:s ->
d:s; d:s -> a:s }").at(0)
#let south = subgraph.compass(g,"s")
#let gf = layout(layout(g, layout-algo: "force"), layout-algo: "tree", layout-nodes:
"fixed",subgraph:south)
#let ga = layout(g, layout-algo: "force")
#let gt = layout(layout(g, layout-algo: "tree",layout-roots: (2)), layout-algo:
"anneal", layout-nodes: "fixed",gamma-ee:0.1,gamma-ev:.75,beta:5,length-scale:0.4)
#grid(columns: 3, gutter: 2cm, draw(gf), draw(ga), draw(gt))
```

```
#let g = graph.parse("digraph partial { a -> b; b -> c; c -> d; d -> a }").at(0)
#let tree = graph.forests(g).at(0)
#let g = layout(g, layout-algo: "tree", subgraph: tree)
#graph.edges(g).map(edge => edge.pos)
```

16.3.2.1 Parameters

```
layout(
  graph: dictionary,
  spring: none dictionary,
  repulsion: none dictionary,
  constraints: none dictionary,
  labels: none dictionary,
  solver: none dictionary,
  subgraph: none dictionary,
  viewport-w: float,
  viewport-h: float,
  tree-dx: float,
  tree-dy: float,
  steps: int,
  seed: int,
  step: float,
  step-shrink: float,
  cool: float,
  accept-floor: float,
  early-tol: float,
  temp: float,
  delta: float,
  beta: float,
  k-spring: float,
  g-center: float,
  epochs: int,
  crossing-penalty: float,
  gamma-dangling: float,
  gamma-dangling-centroid: float,
  gamma-ee: float,
  directional-force: float,
  label-length-scale: float,
  label-spring: float,
  label-charge: float,
  label-steps: int,
  label-layout: string,
  label-step: float,
  label-early-tol: float,
  label-max-delta-scale: float,
  gamma-ev: float,
  eps: float,
  incremental-energy: bool,
  layout-algo: string,
  layout-nodes: string,
  layout-direction: string,
  rank-align: string,
  layout-roots: array,
  rank-same: array,
  route-edge-weight: float,
  route-exit-weight: float,
  route-label-width-scale: float,
  route-label-width-cap: float,
  depth-scale: float,
  flattening-end: float,
  length-scale: float
) -> dictionary
```

16.3.2.1.1 graph `dictionary`

Graph object returned by `graph.build` or `graph.parse`.

16.3.2.1.2 spring `none` or `dictionary`

Semantic incidence-spring options. Supported fields are `strength` and `length`. These override the corresponding flat parameters below.

Default: `none`

16.3.2.1.3 repulsion `none` or `dictionary`

Semantic repulsion options. Supported fields are `strength`, `centering`, `edge-node`, `edge-edge`, `dangling`, `dangling-centroid`, and `softening`. These override the corresponding flat parameters below.

Default: `none`

16.3.2.1.4 constraints `none` or `dictionary`

Semantic constraint options. Supported fields are `side-strength`, `node-movement`, `direction`, `rank-alignment`, `roots`, and `same-rank`. These override the corresponding flat parameters below.

Default: `none`

16.3.2.1.5 labels `none` or `dictionary`

Semantic label-layout options. Supported fields are `distance`, `spring`, `repulsion`, `steps`, `model`, `step`, `tolerance`, and `max-movement`. These override the corresponding flat parameters below.

Default: `none`

16.3.2.1.6 solver `none` or `dictionary`

Semantic solver options. Supported fields are `algorithm`, `steps`, `epochs`, `seed`, `step`, `step-shrink`, `cooling`, `acceptance-floor`, `tolerance`, `temperature`, `max-movement`, `incremental-energy`, `subgraph-mode`, `crossing-penalty`, `depth-scale`, and `flattening-end`. These override the corresponding flat parameters below.

Default: `none`

16.3.2.1.7 subgraph `none` or `dictionary`

Optional subgraph object to lay out. With `"tree"`, other edges are drawn from the resulting node positions. With `"dot"` and `"stable-layered"`, the subgraph determines rank constraints, while all paired edges between included nodes get dummy routing vertices and edge positions. With `"force"` and `"anneal"`, `solver.subgraph-mode: "fixed-boundary"` keeps outside points fixed but interacting; `"isolated"` excludes them from the solver and label-layout interactions. The selection must have compatible topology, and isolated selections cannot cross grouped coordinates.

Default: `none`

16.3.2.1.8 viewport-w `float`

Width of the layout viewport used to derive the natural spring length. Applies to both `"force"` and `"anneal"`.

Default: `10.0`

16.3.2.1.9 viewport-h `float`

Height of the layout viewport used to derive the natural spring length. Applies to both `"force"` and `"anneal"`.

Default: `10.0`

16.3.2.1.10 tree-dx `float`

Horizontal spacing multiplier for traversal-tree and layered placement. For `"force"` and `"anneal"`, this scales the initial placement.

Default: `0.9`

16.3.2.1.11 tree-dy `float`

Vertical spacing multiplier for traversal-tree and layered placement. For `"force"` and `"anneal"`, this scales the initial placement.

Default: `1.2`

16.3.2.1.12 steps `int`

Iterations per epoch. In `"force"` mode this is the number of force integration steps within the single depth-flattening schedule; in `"anneal"` mode this is the number of proposals per temperature epoch.

Default: `30`

16.3.2.1.13 seed `int`

Seed for deterministic initialization, force-mode jitter, and annealing proposals. Applies to both modes.

Default: `2`

16.3.2.1.14 step `float`

Initial movement scale. `"force"` multiplies computed forces by this value, with no restart when depth reaches zero; `"anneal"` uses it as a proposal step size in natural spring-length units.

Default: `0.81`

16.3.2.1.15 step-shrink `float`

Anneal-only step shrink factor, applied when an epoch's acceptance ratio falls below `accept-floor`.

Default: `0.21`

16.3.2.1.16 cool `float`

Cooling factor applied once per epoch. `"anneal"` cools `temp`; `"force"` shrinks `step`.

Default: `0.85`

16.3.2.1.17 accept-floor `float`

Anneal-only acceptance-ratio threshold below which `step` is shrunk by `step-shrink`.

Default: `0.15`

16.3.2.1.18 early-tol `float`

Force-only early stop threshold for maximum movement in one step, as a multiple of the natural spring length, eligible only after effective depth scale reaches zero. The annealing schedule stores this value but does not currently use it for stopping.

Default: `1e-6`

16.3.2.1.19 temp `float`

Anneal-only initial temperature used in the Metropolis acceptance test, scaled by natural spring length squared.

Default: `0.3`

16.3.2.1.20 delta `float`

Force-mode maximum movement clamp per point and per step, as a multiple of the natural spring length.

Default: `0.4`

16.3.2.1.21 beta `float`

Base repulsion strength for vertex-vertex interactions. Also scales `gamma-ev`, `gamma-ee`, `gamma-dangling`, `gamma-dangling-centroid`, and `g-center`. Applies to both modes through the shared spring energy.

Default: `50.0`

16.3.2.1.22 k-spring `float`

Spring stiffness for node-to-edge incidence lengths. Applies to both modes.

Default: `11.0`

16.3.2.1.23 g-center `float`

Centering strength relative to `beta`. Applies to both modes.

Default: `0.002`

16.3.2.1.24 epochs `int`

Number of epochs. Both modes run up to `steps` iterations inside each epoch. Force mode shares this budget and its cooling schedule across depth flattening and the remaining planar relaxation.

Default: `30`

16.3.2.1.25 crossing-penalty `float`

Anneal-only fixed energy penalty per detected edge crossing. The direct force integrator does not currently add a crossing force.

Default: `30.0`

16.3.2.1.26 gamma-dangling `float`

Repulsion for dangling half edges, relative to `beta`. Applies to both modes through the shared spring energy.

Default: `5.0`

16.3.2.1.27 gamma-dangling-centroid `float`

Repulsion of every dangling endpoint from the current node centroid, relative to `beta`. The equal-and-opposite reaction is shared over the nodes, avoiding translational drift. Applies to both modes.

Default: `0.0`

16.3.2.1.28 gamma-ee `float`

Local edge-edge repulsion, relative to `beta`. Applies to both modes.

Default: `0.1`

16.3.2.1.29 directional-force `float`

Bias that pushes points in directions implied by pin/port constraints. Force mode treats this as a force and scales it by natural spring length; anneal mode treats it as a dimensionless multiplier on proposal steps. Applies to both modes.

Default: `5.0`

16.3.2.1.30 label-length-scale `float`

Edge-label target offset as a multiple of the graph spring length. Label layout runs after both graph layout modes.

Default: `0.6`

16.3.2.1.31 label-spring `float`

Spring strength pulling each label toward its target offset in the spring-based label layouts.

Default: `23.0`

16.3.2.1.32 label-charge `float`

Repulsion strength between labels and graph points, scaled by spring length squared. Label layout runs after both modes.

Default: `3.0`

16.3.2.1.33 label-steps `int`

Maximum number of post-layout label relaxation steps. Set to `0` to skip label placement. Applies after both modes.

Default: `20`

16.3.2.1.34 label-layout `string`

Edge-label relaxation model. `"normal"` uses a perpendicular offset, `"dangling-tangent"` uses the edge direction for dangling half-edge labels and a perpendicular offset for paired edges, and `"fixed-length"` keeps each label at a fixed distance from its edge point and only lets that segment rotate.

Default: `"normal"`

16.3.2.1.35 label-step `float`

Label relaxation step size. Applies after both modes.

Default: `0.15`

16.3.2.1.36 label-early-tol `float`

Label relaxation early stop threshold. Applies after both modes.

Default: `1e-3`

16.3.2.1.37 label-max-delta-scale `float`

Label movement clamp as a multiple of spring length. Applies after both modes.

Default: `0.5`

16.3.2.1.38 gamma-ev `float`

Edge-vertex repulsion, relative to `beta`. Applies to both modes.

Default: `0.01`

16.3.2.1.39 eps `float`

Softening epsilon used in inverse-square force/energy terms. Applies to both modes.

Default: `1e-4`

16.3.2.1.40 incremental-energy `bool`

Whether annealing updates cached energy by local deltas. This is anneal-only; force mode computes direct forces instead of energies.

Default: `true`

16.3.2.1.41 layout-algo `string`

Layout algorithm. Use `"force"` for direct force integration, `"anneal"` for simulated annealing against the spring energy, `"tree"` for a traversal-tree placement, `"dot"` for a Graphviz-like layered placement, or `"stable-layered"` for a stable railroad-inspired layered placement.

Default: `"force"`

16.3.2.1.42 layout-nodes `string`

Node movement policy. `"layout"` lets the layout algorithm move nodes. `"fixed"` keeps every node at its current position for this layout pass and only moves edge control points. With `subgraph`, only edges in the subgraph are moved; other edge control points stay at their current positions.

Default: `"layout"`

16.3.2.1.43 layout-direction `string`

Direction for traversal-tree and layered rank placement. `"down"` places increasing ranks downward; `"right"` swaps the layout axes so increasing ranks go left-to-right and measured node/label widths reserve rank-axis space.

Default: `"down"`

16.3.2.1.44 rank-align `string`

Alignment of real nodes inside a rank along the rank axis. `"center"` keeps node centers aligned, while `"start"` / `"left"` and `"end"` / `"right"` align the corresponding measured node-box side.

Default: `"center"`

16.3.2.1.45 layout-roots `array`

Ordered node indices used as preferred roots for `"tree"`, `"dot"`, and `"stable-layered"`. Roots outside the selected node set are ignored. Remaining components are laid out afterward in graph order.

Default: `()`

16.3.2.1.46 rank-same `array`

Subgraph objects whose incident nodes should share a dot/stable-layered rank; node-index arrays and callbacks receiving the graph are also accepted. These are layout hints supplied by Typst rather than parsed graph structure.

Default: `()`

16.3.2.1.47 route-edge-weight `float`

Dot/stable-layered also honors a node statement `layout-rank` as an exact non-negative integer rank. Nodes with the same `layout-rank` are placed on the same horizontal layer, and larger ranks are placed lower. Relative layout weight for paired edges outside the dot/stable-layered rank subgraph. Lower values make these edges guide routing without dominating the rank tree.

Default: `0.15`

16.3.2.1.48 route-exit-weight `float`

Extra horizontal straightening weight for the first or last segment of an edge with `source-route-exit` or `sink-route-exit` set to a vertical side.

Default: `4.0`

16.3.2.1.49 route-label-width-scale `float`

Multiplier for measured edge-label width when sizing non-rank dummy routing vertices in dot/stable-layered layout.

Default: `1.0`

16.3.2.1.50 route-label-width-cap `float`

Maximum non-rank dummy label width as a multiple of `tree-dx`. Set to `0` or a negative value to disable the cap.

Default: `2.0`

16.3.2.1.51 depth-scale `float`

Force-only initial multiplier of raw auxiliary depth, finite and >= 0. Effective z is this smoothly decreasing scale times raw z, not a spring pulling raw coordinates toward the plane. A value of 0 starts in 2D; flattening preserves hard raw-depth pins while removing their invisible separation from the visible 2D force calculation.

Default: `1.0`

16.3.2.1.52 flattening-end `float`

Force-only fraction of the total `steps` × `epochs` iteration budget over which the depth scale reaches zero by smoothstep, finite and in 0..=1. Zero starts in 2D; 1 still evaluates the final iteration on the exact plane. Neither value adds a phase or restarts the cooling schedule.

Default: `0.5`

16.3.2.1.53 length-scale `float`

Natural spring-length multiplier. This scales the graph's preferred edge length and dimensional force/energy terms so changing only this value mostly zooms the result instead of retuning the force ratios. Applies to both modes.

Default: `0.35`

16.3.3 layout-sequence

Apply multiple layout passes in order.

Each pass is a dictionary of named arguments accepted by `layout`, excluding the graph itself.

16.3.3.1 Parameters

```
layout-sequence(
  graph: dictionary,
  passes: array
) -> dictionary
```

16.3.3.1.1 graph `dictionary`

Graph object returned by `graph.build` or `graph.parse`.

16.3.3.1.2 passes `array`

Array of layout option dictionaries.

17 Drawing functions

`draw`, `edge-halves`, and `to-cetz-edge-halves` are top-level functions exported by `lib.typ`. They turn an already laid-out graph into CeTZ content and expose the edge geometry used by custom renderers.

17.1 Concepts

17.1.1 Drawing

Draw styling is Typst-native. Pass dictionaries or callbacks to `draw`; edge callbacks receive the merged `scope`, edge statements, source/sink half-edge records, and the edge index:

```
#let edge-label(edge) = text(fill: rgb("#" + edge.color))[#edge.display-label]
#let source-style(edge) = (stroke: red + 0.5pt)
#let sink-style(edge) = (stroke: blue + 0.5pt)
#draw(layout(g), edge-label: edge-label, source-style: source-style, sink-style: sink-
style)
```

Edge labels are drawn with CeTZ content at the layout label position. Use `edge-label-style: (anchor: "south")`, or an edge-data callback returning a style dictionary, to choose which point of the label is anchored there.

Add overlays with `draw-after: (g, bounds) => { ... }`. The callback receives the positioned graph and its rendered bounds before overlays and canvas padding. The numeric `left`, `right`, `top`, `bottom`, `width`, and `height` fields use graph units; `top` is the greatest y coordinate. Bounds include drawn shapes, curves, and label boxes, following CeTZ's convention of excluding extra stroke thickness. An empty drawing supplies zeros for every field. Overlay elements share `unit` and can enlarge the final canvas without affecting graph layout. `draw-after` also accepts an array of CeTZ elements or `none`.

```
#draw(layout(g), draw-after: (g, bounds) => {
  cetz.draw.rect(
    (bounds.left - 0.2, bounds.bottom - 0.2),
    (bounds.right + 0.2, bounds.top + 0.2),
    stroke: gray,
  )
})
```

Marks can follow graph orientation as a first-class draw option. Put the same mark layer on both halves and set `mark-orientation: "edge"`; default-oriented edges mark the source half, reversed edges mark the sink half with the marker flipped, and undirected edges suppress the mark. Use `mark-position: "center"` or a numeric ratio from `0` to `1`, then `mark-shift` for a signed arc-length adjustment along the derived path. This positioning is independent of whether the layer is straight or patterned. `"center-if-dangling"` centers a dangling mark but keeps an orientation-selected paired mark at the source/sink split point. Heads span a chord between two points on the full carrier, not a tangent. Triangle and straight heads place their geometric tip and the center of their back on the curve; other marks use their declared tip/base anchors. Centered heads straddle the requested arc position, so the chord midpoint can lie off the curve. Near endpoints the sampling interval moves inward, and short carriers compress the head to fit. Longitudinal fitting preserves the head's width and stroke thickness. Interior marks overlay the unshortened curve. With shortening enabled, filled end heads meet the shaft at the inward contact; straight open heads keep the shaft running to their tip.

```
#let oriented-arrow = (
  stroke: black + 0.7pt,
  mark: (end: (symbol: ">", fill: black, anchor: "center", shorten-to: auto), scale: 0.75),
  mark-position: "center-if-dangling",
  mark-orientation: "edge",
)
#draw(layout(g), source-style: oriented-arrow, sink-style: oriented-arrow)
```

17.2 Reference

- edge-halves()
- to-cetz-edge-halves()
- draw()

17.2.1 edge-halves

Split a laid-out graph edge into source and sink half-edge paths.

The returned dictionary has `source`, `sink`, `curve`, and `split-gap`. The split point is the edge layout point, so a zero-gap pair joins smoothly there. `split-gap` reports the effective gap when a requested half-gap is longer than either half-edge.

17.2.1.1 Parameters

```
edge-halves(
  edge: dictionary,
  nodes: array,
  omega: float,
  source-outset: int float,
  sink-outset: int float,
  split-gap: int float,
  accuracy: float
) -> dictionary
```

17.2.1.1.1 edge `dictionary`

Edge record returned by `graph.edges(layout(g))`.

17.2.1.1.2 nodes `array`

Node records returned by `graph.nodes(layout(g))`.

17.2.1.1.3 omega `float`

Hobby curl used for the source-to-edge and edge-to-sink curves.

Default: `1.0`

17.2.1.1.4 source-outset `int` or `float`

Arc-length trim applied at the source node side.

Default: `0`

17.2.1.1.5 sink-outset `int` or `float`

Arc-length trim applied at the sink node side.

Default: `0`

17.2.1.1.6 split-gap `int` or `float`

Total arc-length gap centered on the edge layout point.

Default: `0`

17.2.1.1.7 accuracy `float`

Arc-length accuracy used while trimming.

Default: `0.001`

17.2.2 to-cetz-edge-halves

Draw the two halves of a laid-out graph edge through CeTZ.

`source-style` applies from the source node to the edge layout point, and `sink-style` applies from the edge layout point to the sink node.

17.2.2.1 Parameters

```
to-cetz-edge-halves(
  edge: dictionary,
  nodes: array,
  unit: int float length ratio,
  omega: float,
  source-outset: int float,
  sink-outset: int float,
  split-gap: int float,
  accuracy: float,
  source-style: dictionary,
  sink-style: dictionary
) -> content
```

17.2.2.1.1 edge `dictionary`

Edge record returned by `graph.edges(layout(g))`.

17.2.2.1.2 nodes `array`

Node records returned by `graph.nodes(layout(g))`.

17.2.2.1.3 unit `int` or `float` or `length` or `ratio`

Coordinate length for one graph-layout unit. Numbers are interpreted as em.

Default: `1`

17.2.2.1.4 omega `float`

Hobby curl used for the source-to-edge and edge-to-sink curves.

Default: `1.0`

17.2.2.1.5 source-outset `int` or `float`

Arc-length trim applied at the source node side.

Default: `0`

17.2.2.1.6 sink-outset `int` or `float`

Arc-length trim applied at the sink node side.

Default: `0`

17.2.2.1.7 split-gap `int` or `float`

Total arc-length gap centered on the edge layout point.

Default: `0`

17.2.2.1.8 accuracy `float`

Arc-length accuracy used while trimming.

Default: `0.001`

17.2.2.1.9 source-style `dictionary`

CeTZ style for the source half edge.

Default: `(:)`

17.2.2.1.10 sink-style `dictionary`

CeTZ style for the sink half edge.

Default: `(:)`

17.2.3 draw

Draw a graph object with CeTZ.

The graph must already have positions, either from `layout` or from explicit `pos` values passed to graph node or edge items. Paired edges without an explicit edge position use the midpoint of their endpoint nodes. Node and edge Typst style dictionaries or callbacks are evaluated and forwarded to CeTZ.

```
 #let positioned = graph.build({
   graph.node(<left>, label: [left], pos: graph.pos(x: 0, y: 0))
   graph.node(<right>, label: [right], pos: graph.pos(ref: <left>, dx: 2.5, dy: 0))
   graph.edge(graph.source(<left>), graph.sink(<right>))
   graph.edge(graph.source(<right>), <right-out>, pos: graph.pos(ref: <right>, dx: 0.9, dy:
0.7))
 },
   name: "positioned",
 )
 #draw(positioned, source-style: (stroke: black + 0.7pt), sink-style: (stroke: black +
0.7pt))
```

```
 #let parallel-base-edge = 0
 #let source-patterns = (
   "coil",
```

```
  "zigzag",
  "coil",
  "wave",
  "zigzag",
  "coil",
  "wave",
  "zigzag",
)
#let sink-patterns = (
  "coil",
  "zigzag",
  "coil",
  "zigzag",
  "coil",
  "wave",
  "coil",
  "wave",
)
#let parallel-layer(edge, mark: none) = if edge.eid == parallel-base-edge {
  (
    offset: -0.5,
    length: 1.5,
    ratio: 0.5,
    resolve-length: "min",
    stroke: (paint: rgb("#2f6f4e"), thickness: 1.1pt, cap: "round"),
    mark: mark,
  )
} else { none }
#let stack(base, layer) = if layer == none { base } else { (base, layer) }
#let source-stroke(edge) = if edge.eid == parallel-base-edge {
  (paint: gray, thickness: 0.75pt, cap: "round")
} else {
  (paint: red, thickness: 1.2pt, cap: "round")
}
#let sink-stroke(edge) = if edge.eid == parallel-base-edge {
  (paint: gray, thickness: 0.75pt, cap: "round")
} else {
  (paint: blue, thickness: 1.2pt, cap: "round", dash: "dotted")
}
#let source-style(edge) = {
  let base = (
    stroke: source-stroke(edge),
    pattern: source-patterns.at(edge.eid),
    pattern-amplitude: 0.18,
    pattern-wavelength: 0.55,
    pattern-coil-longitudinal-scale: 1.6,
  )
  stack(base, parallel-layer(edge))
}
#let sink-style(edge) = {
  let base = (
    stroke: sink-stroke(edge),
    pattern: sink-patterns.at(edge.eid),
    pattern-amplitude: 0.18,
    pattern-wavelength: 0.55,
    pattern-coil-longitudinal-scale: 1.6,
  )
  stack(base, parallel-layer(edge, mark: (end: (symbol: "straight"), scale: 0.75)))
}
#let g = graph.build({
```

```
  graph.node(<a>, label: [a hi])
  graph.node(<c>)
  graph.node(<d>)
  graph.node(<e>)
  graph.edge(graph.source(<a>), <ac>, graph.sink(<c>), pos: graph.pos(x: 0, y: 0.75, mode:
"pin"))
  graph.edge(graph.source(<c>), <ca>, graph.sink(<a>, compass: "e"))
  graph.edge(graph.source(<c>), <cd>, graph.sink(<d>, compass: "e"))
  graph.edge(graph.source(<e>), <ed>, graph.sink(<d>, compass: "e"))
  graph.edge(graph.source(<e>), <ea>, graph.sink(<a>, compass: "e"))
  graph.edge(graph.source(<d>), <d-out>)
  graph.edge(graph.source(<e>, compass: "e"), <e-out>)
  graph.edge(graph.source(<a>), <a-out>)
},
  name: "demo",
)
#let layed-out = layout(g)
#let east = subgraph.compass(layed-out, "e")
#draw(layed-out, subgraph: east, source-style: source-style, sink-style: sink-style)
```

```
#let p = graph.build({
  graph.node(<pa>, label: [a], pos: graph.pos(y: 0, mode: "pin"))
  graph.node(<pc>, label: [c], pos: graph.pos(y: 0, mode: "pin"))
  graph.node(<pc1>, label: [c], pos: graph.pos(y: 0, mode: "pin"))
  graph.node(<pc2>, label: [c], pos: graph.pos(y: 0, mode: "pin"))
  graph.edge(graph.source(<pa>), <e0>, graph.sink(<pc>))
  graph.edge(graph.source(<pc2>), <e1>, graph.sink(<pc1>))
},
  name: "parallel demo",
)
#let parallel-edge-style(edge) = (
    offset: -0.5,
    length: 1.4,
    ratio: 0.5,
    resolve-length: "min",
    stroke: (paint: rgb("#2f6f4e"), thickness: 1.1pt, cap: "round"),
  )

#let focused-base-style(edge) = (
  stroke: (paint: gray, thickness: 0.7pt, cap: "round"),
  pattern: "coil",
  pattern-amplitude: 0.14,
  pattern-wavelength: 0.45,
  pattern-coil-longitudinal-scale: 1.5,
)
#let focused-source-style(edge) = (focused-base-style(edge), parallel-edge-style(edge))
#let focused-sink-style(edge) = (focused-base-style(edge), parallel-edge-style(edge) + (
  mark: (end: ">"),
))
#draw(layout(p, g-center: 0.004, label-steps: 0,), unit: 1.25, source-style: focused-
source-style, sink-style: focused-sink-style)
```

```
#let g = graph.build({
  graph.node(<a>, pos: graph.pos(x: 0, y: 0, mode: "pin"))
```

```
  graph.node(<c>, pos: graph.pos(x: 3, y: 0, mode: "pin"))
  graph.node(<d>, pos: graph.pos(x: 3, y: -1, mode: "pin"))
  graph.edge(graph.source(<a>), <ac>, graph.sink(<c>), orientation: "default")
  graph.edge(graph.source(<a>), <ad>, graph.sink(<d>), orientation: "reversed")
},
  name: "oriented marks",
)
#let arrow = (
  end: (symbol: ">", fill: black, anchor: "center", shorten-to: auto),
  scale: 0.75,
)
#let oriented-arrow = (
  stroke: black + 0.7pt,
  mark: arrow,
  mark-position: "center-if-dangling",
  mark-orientation: "edge",
)
#draw(layout(g), unit: 1.4, source-style: oriented-arrow, sink-style: oriented-arrow)
```

A style layer may set `crossing-under` to a visible edge's integer id or Typst label name and `crossing-gap` (default `0.55`) to the total arc length hidden around each proper centerline intersection. References are resolved independently of edge order. The current layer is split with Kurvst, so patterned phases continue across the hidden spans. A mark on the same layer is emitted once on its uncut carrier, so splitting the painted path neither removes nor duplicates it. Dangling layers and paired layers with one continuous source/sink paint style are supported. On a paired layer both half styles must specify the same target and gap. The cut layer cannot participate in a subgraph underlay. References to self, unknown, or invisible edges are errors. A valid pair with no proper interior intersection is left unchanged.

17.2.3.1 Parameters

```
draw(
  graph: dictionary,
  scope: dictionary,
  unit: auto int float length ratio,
  title: none auto content string,
  subgraph: none dictionary array,
  debug: bool int,
  show-half-edge-ids: bool,
  node-radius: auto int float array,
  node-min-radius: int float,
  node-label-padding: int float,
  node-fill: any,
  node-stroke: any,
  node-outset: auto int float,
  node-label-style: dictionary,
  node-style: dictionary function none,
  node-label: auto content string function none,
  draw-node: auto function,
  draw-after: none array function,
  edge-stroke: any,
  edge-style: dictionary array function none,
  edge-offset: int float,
  edge-length: none int float,
  edge-ratio: none int float,
  edge-resolve-length: string function,
  edge-accuracy: float,
  edge-optimize: bool,
  edge-split-gap: int float,
  edge-dangling-tangent: auto string,
  source-style: dictionary array function none,
  sink-style: dictionary array function none,
  edge-label: content string function none,
  edge-label-style: dictionary function none,
  edge-omega: float,
  edge-trim-accuracy: float,
  padding: none int float array dictionary,
  debug-edge-radius: int float,
  debug-edge-fill: any,
  debug-edge-stroke: any,
  debug-edge-label-fill: any,
  subgraph-edge-style: dictionary,
  subgraph-edge-underlay: bool
) -> content
```

17.2.3.1.1 graph `dictionary`

Graph object with positions from `layout` or explicit graph API `pos` fields.

17.2.3.1.2 scope `dictionary`

Additional values merged into node and edge callback dictionaries.

Default: `(:)`

17.2.3.1.3 unit `auto` or `int` or `float` or `length` or `ratio`

Coordinate length for one graph-layout unit. `auto` uses any unit stored by `graph.style`, falling back to `1em`. Numbers are interpreted as em.

Default: `auto`

17.2.3.1.4 title `none` or `auto` or `content` or `string`

Optional title displayed above the diagram. Use `auto` for the graph name.

Default: `none`

17.2.3.1.5 subgraph `none` or `dictionary` or `array`

Optional subgraph or array of subgraphs whose half-edges are shaded. Array entries may be subgraph objects or records like `(subgraph: sg, edge-style: (stroke: red + 2pt))`. Each selection must have topology compatible with the graph.

Default: `none`

17.2.3.1.6 debug `bool` or `int`

Debug level. `1` enables CeTZ canvas debug; `2` also marks edge positions.

Default: `false`

17.2.3.1.7 show-half-edge-ids `bool`

Show `h_i` beside every half-edge without an explicit endpoint label.

Default: `false`

17.2.3.1.8 node-radius `auto` or `int` or `float` or `array`

Default CeTZ node radius. Use `auto` to fit the node label.

Default: `auto`

17.2.3.1.9 node-min-radius `int` or `float`

Minimum radius used when `node-radius` is `auto`.

Default: `0.16`

17.2.3.1.10 node-label-padding `int` or `float`

Extra canvas-unit padding around labels when `node-radius` is `auto`.

Default: `0.08`

17.2.3.1.11 node-fill `any`

Default CeTZ node fill.

Default: `white`

17.2.3.1.12 node-stroke `any`

Default CeTZ node stroke.

Default: `black`

17.2.3.1.13 node-outset `auto` or `int` or `float`

Edge clearance from node centers. `auto` uses each node circle radius. Increase this when labels extend beyond the circle.

Default: `auto`

17.2.3.1.14 node-label-style `dictionary`

Default node-label style forwarded to `cetz.draw.content`.

Default: `(:)`

17.2.3.1.15 node-style `dictionary` or `function` or `none`

Node style dictionary or callback. A callback receives node data.

Default: `(:)`

17.2.3.1.16 node-label `auto` or `content` or `string` or `function` or `none`

Node label content or callback. `auto` uses the node name.

Default: `auto`

17.2.3.1.17 draw-node `auto` or `function`

CeTZ-compatible node drawing callback. `auto` draws the default circular node and label. A callback receives `(node, box)` and should return CeTZ draw elements; `box` contains `name`, `center`, `width`, `height`, `unit`, `label`, `label-style`, `style`, `radius`, and `node`. Generated cut boundary nodes skip this callback and all node painting; they retain a named position and zero-size anchor box for incident edges.

Default: `auto`

17.2.3.1.18 draw-after `none` or `array` or `function`

Additional CeTZ elements drawn after the edges, labels, and nodes in the same canvas. A callback receives `(graph, bounds)` and returns CeTZ elements. `bounds` contains numeric `left`, `right`, `top`, `bottom`, `width`, and `height` in graph units, enclosing the rendered graph before overlays and canvas padding. It includes shapes, curves, and label boxes; CeTZ does not add stroke thickness to path bounds. `top` is the greatest y coordinate, and all fields are zero for an empty drawing. Use `graph.nodes` / `graph.edges` to inspect the positioned graph. These elements share `unit` and contribute to canvas bounds, but do not participate in graph layout.

Default: `none`

17.2.3.1.19 edge-stroke `any`

Default CeTZ edge stroke.

Default: `0.1em`

17.2.3.1.20 edge-style `dictionary` or `array` or `function` or `none`

Default logical-edge style dictionary, layers, or callback. An edge's `style:` value patches this default; `auto` delegates and `none` hides the edge without changing its topology. Stored `graph.style` edge and endpoint defaults are composed first, followed by this style, record edge style, draw endpoint style, and endpoint-local style. Shared logical-edge callbacks run once before the two halves are composed.

Default: `(:)`

17.2.3.1.21 edge-offset `int` or `float`

Default normal offset for edge paths. Applied to the base edge geometry before patterns; node outsets then trim the shifted path.

Default: `0`

17.2.3.1.22 edge-length `none` or `int` or `float`

Maximum visible arc length for centered parallel edge paths. `none` keeps the full shifted path.

Default: `none`

17.2.3.1.23 edge-ratio `none` or `int` or `float`

Maximum visible fraction of the full offset path length for centered edge paths. Combined with `edge-length` according to `edge-resolve-length`.

Default: `none`

17.2.3.1.24 edge-resolve-length `string` or `function`

Resolve `edge-length` and `edge-ratio`. Accepted string values are `"min"`/`"shorter"`, `"max"`/`"longer"`, `"length"`/`"fixed"`, `"ratio"`/`"relative"`, or `"none"`/`"full"`. A function receives `(offset-path-length, length, ratio)`.

Default: `"min"`

17.2.3.1.25 edge-accuracy `float`

Arc-length accuracy for fitted parallel edge paths.

Default: `0.001`

17.2.3.1.26 edge-optimize `bool`

Let Kurbo optimize the fitted parallel path.

Default: `true`

17.2.3.1.27 edge-split-gap `int` or `float`

Total arc-length gap centered on the source/sink split at the edge layout point. A per-layer `split-gap` overrides this value.

Default: `0`

17.2.3.1.28 edge-dangling-tangent `auto` or `string`

Tangent constraint at dangling edge positions. `"horizontal"` and `"vertical"` preserve `bend` while making the corresponding endpoint tangent exact. A per-layer `dangling-tangent` overrides this value.

Default: `auto`

17.2.3.1.29 source-style `dictionary` or `array` or `function` or `none`

Source half-edge style dictionary, array of layer dictionaries, or callback. A `"center"` or numeric mark on a common source/sink paint layer is placed once on the complete derived layer, so it follows `shift`. `mark-position` also accepts a numeric path-length ratio from `0` to `1`; `mark-shift` moves a centered or numeric mark by signed arc length, with positive values moving toward the derived path's end. This also works on patterned dangling and asymmetric paired layers without resetting pattern phase. `mark-position: "center-if-dangling"` centers dangling marks while retaining the paired-edge split point selected by `mark-orientation`. `mark-orientation: "edge"` makes a mark follow `edge.orientation` instead of raw path direction; reversed edges move the mark to the sink half and flip it, while undirected edges suppress it. Heads span a chord between two points on the full carrier: triangle and straight heads use their geometric tip and back; other marks use their declared tip/base anchors. Centered heads straddle the requested arc position, so their chord center can lie off the curve. Near endpoints the interval moves inward; the head fits the chord longitudinally while keeping its width. Interior marks overlay the unshortened curve; end shafts meet the appropriate head contact. `source-anchor` may be a CeTZ anchor name such as `"north"` or `"south"` to route this endpoint from a measured node-box anchor. By default, anchored paired edges use two smooth cubic halves through the edge layout point while preserving the endpoint anchor tangents. Set `route: "hobby-through"` to instead build a single Hobby spline through the source anchor, source guide, edge point, sink guide, and sink anchor, then split source/sink styling at the edge point. `route: "direct"` keeps the same anchored cubic routing but suppresses the default edge-position Hobby route. `route-points: "through"` also threads any layout-provided half-edge route points through that same Hobby path. `route: "straight-through"` draws the two straight force springs from source to edge position and from edge position to sink. `pattern-natural-endpoints` defaults to `false`. For built-in coil strings (`"coil"`, `"helix"`, `"spring"`) on complete continuous paths with both endpoints anchored, `true` constructs a fitted `kurvst.coil` dictionary in Typst and applies it once via ordinary `kurvst.pattern`: full-amplitude inward endpoint phases and half-integer coil periods, with exact endpoints and no taper, straight stubs, or connectors. This overrides `pattern-phase`, bypasses `pattern-fit` integer fitting, and ignores `pattern-endpoint-slope` (`endpoint-ramp: false`). Fitted dictionaries also work directly as `pattern` without this flag; see the Kurvst manual's Path Patterns section for the fitting formula and one-pass application settings. Other patterns ignore this flag. Without it, anchored coils use the 75%-wavelength endpoint taper (capped at half the path length), with a squared longitudinal envelope. Without automatic fitting, `pattern-fit: true` adjusts the wavelength to the nearest whole number of periods on a complete, unbroken edge. `pattern-phase` is in radians; `calc.pi / 2` gives coils matching, gently tapered endpoint phases without straight end sections. `pattern-endpoint-slope` sets the taper's initial slope, from 0 to 3. Zero (default) keeps tangential ends; positive values allow angled ends without detaching them. The angle also depends on phase, amplitude, and wavelength. When `source-style` and `sink-style` share a pattern and differ only in stroke (for example colour), the whole edge is patterned once, fitted like a single edge, and cut at the edge layout point, so each half keeps its own stroke while the coil runs continuously through the split. Other split-style halves and partially anchored paths, such as crossing-gap fragments, keep the requested wavelength, taper only anchored endpoints, and preserve phase continuity across hidden spans. Neither automatic coil construction nor integer fitting applies. A path with gaps is patterned per continuous run, carrying the phase across each gap. A finite layer can set `shift` to move along the logical edge, with positive values moving toward its end. `label` attaches content near the layer midpoint; `label-shift` (default `0`) moves its reference point by signed arc length on that derived path, clamped to the path's endpoints. Positive values move toward its end without moving or trimming the layer. `label-side` is `auto`, `"left"`, `"right"`, or a signed number; `auto` follows the side selected by ordinary edge-label layout. If a label-side offset has no layout direction yet, both offset and label follow the sign of `offset` (positive means left). With `label-style.anchor` omitted or set to `auto` (also `"auto"`), the centered label's actual CeTZ box clears the local tangent line by `label-gap`, accounting for text bounds, wrapping, padding and rotation. This is not minimum distance to the finite curved shaft. An explicit anchor, including `"center"`, instead sits at the shifted reference point plus `label-gap` along the chosen normal, without box correction. Negative gaps are clamped to zero. Other `label-style` fields are forwarded to `cetz.draw.content`. An attached label replaces the ordinary painted edge label, which may still provide its pre-layout measurement and side. Attached labels do not add separate pre-layout collision constraints. `label: auto` reuses the ordinary edge label and its content style. `label-only: true` makes the layer an invisible label carrier: strokes, fills, patterns, marks, crossings and subgraph edge styling cannot paint it. Its geometry still controls label placement. If no visible endpoint layer carries a label, the ordinary edge label is drawn.

17.2.3.1.30 sink-style `dictionary` or `array` or `function` or `none`

Sink half-edge style dictionary, array of layer dictionaries, or callback. A callback receives edge data. `mark-position: "center-if-dangling"` has the same dangling-edge behavior as for `source-style`, and `mark-orientation: "edge"` participates in the same orientation-aware mark placement. `sink-anchor` may be a CeTZ anchor name such as `"north"` or `"south"` to route this endpoint into a measured node-box anchor. `route: "direct"` has the same meaning as in `source-style`.

Default: `(:)`

17.2.3.1.31 edge-label `content` or `string` or `function` or `none`

Edge label content or callback. A callback receives edge data.

Default: `none`

17.2.3.1.32 edge-label-style `dictionary` or `function` or `none`

Edge-label style forwarded to `cetz.draw.content`; use `anchor` to choose which point of the label is placed at the layout label position. A callback receives edge data.

Default: `(:)`

17.2.3.1.33 edge-omega `float`

Hobby curl used at the endpoints of paired edge curves.

Default: `1.0`

17.2.3.1.34 edge-trim-accuracy `float`

Optional style key for anchored source/sink routes. Set `anchor-control-distance` in `source-style` or `sink-style` to override that endpoint's automatic guide distance in graph units. Per-edge endpoint styles control each end independently; `auto` is resolved independently. Set it in `edge-style` to supply a shared distance for both ends. -> auto | int | float Arc-length accuracy for trimming edge curves at node outsets.

Default: `0.001`

17.2.3.1.35 padding `none` or `int` or `float` or `array` or `dictionary`

CeTZ canvas padding.

Default: `0.4`

17.2.3.1.36 debug-edge-radius `int` or `float`

Radius for edge-position markers shown at `debug >= 2`.

Default: `0.08`

17.2.3.1.37 debug-edge-fill `any`

Fill for edge-position markers shown at `debug >= 2`.

Default: `rgb("#ff9f1c")`

17.2.3.1.38 debug-edge-stroke `any`

Stroke for edge-position markers shown at `debug >= 2`.

Default: `rgb("#d72638") + 0.35pt`

17.2.3.1.39 debug-edge-label-fill `any`

Label fill for edge-position markers shown at `debug >= 2`.

Default: `rgb("#7a1020")`

17.2.3.1.40 subgraph-edge-style `dictionary`

Default CeTZ style used to shade half-edges included in `subgraph`. Individual subgraph records may override this with `edge-style`.

Default: `(stroke: rgb("#ffd166") + 4.5pt)`

17.2.3.1.41 subgraph-edge-underlay `bool`

Draw subgraph shading below the normal half-edge style.

Default: `true`

18 Domain styles and templates

Linnest provides generic graph styling and drawing callbacks. Application domains compose those primitives in ordinary Typst code or provide a custom V1 template; there is no built-in physics module.

GammaLoop owns its particle-line, momentum, amplitude, and cross-section template. Those policies are available when rendering through GammaLoop's template bundle, not by importing them from Linnest. See GammaLoop's physics drawing template.

18.1 Concepts

18.1.1 Domain Styles And Templates

Linnest's drawing layer is domain-neutral. A caller can map arbitrary node, edge, and half-edge data to reusable `draw` callbacks without changing topology or adding a domain-specific core option. The following map uses an application-defined `kind` field; the names and styles have no meaning to Linnest itself:

```
#import "../src/lib.typ": draw, graph, layout
#import graph: build, edge, node, sink, source
```

```
#let kinds = (
  dependency: (stroke: rgb("#315f9f") + 0.8pt),
  event: (
    stroke: rgb("#a24b36") + 0.8pt,
    mark: (end: ">"),
    mark-position: "center",
    mark-orientation: "edge",
  ),
)
#let edge-style(edge) = kinds.at(edge.fields.at("kind", default: "dependency"))
#let g = build({
  node(<queued>, label: [queued])
  node(<running>, label: [running])
  node(<done>, label: [done])
  edge(source(<queued>), <starts>, sink(<running>), kind: "event", label: [starts])
  edge(source(<running>), <needs>, sink(<done>), kind: "dependency", label: [needs])
})
#let g = graph.style(g, edge-label: edge => edge.label)
#draw(layout(g, layout-algo: "stable-layered"), edge-style: edge-style)
```

The Feynman example style is configured before layout. `graph-style` is a function returning options for `graph.style`; `draw-style` remains a dictionary of drawing callbacks and defaults:

```
#import "../examples/map-style.typ" as feynman
#let styled = graph.style(g, ..feynman.graph-style(
  unit: 1.35, line-width: 0.7pt, node-radius: 0.2,
  momentum-line-width: 0.4pt,
))
#draw(layout(styled), ..feynman.draw-style)
```

Call `feynman.graph-style()` for the original appearance; replace the former dictionary spread `..feynman.graph-style` with `..feynman.graph-style()`. Its named options are:

- `unit: 1.35`: graph coordinate unit; a number multiplies the current `1em`, or supply an absolute length such as `10pt`.
- `line-width: 0.5pt`: ordinary particle stroke thickness.
- `node-radius: 0.18`: visible node radius in graph units.
- `node-line-width: auto`: node outline, defaulting to `line-width`.
- `massive-line-width: auto`: scalar/ghost stroke, defaulting to `2 * line-width`.
- `fermion-arrow-line-width: auto`: fermion arrowhead outline, defaulting to `0.4 * line-width`.
- `momentum-line-width: auto`: momentum shaft and arrowhead thickness, both defaulting to `0.8 * line-width`.

Width overrides are Typst lengths, independent of the graph unit and node radius. With all defaults, the node and ordinary edge strokes are `0.5pt`, scalar/ghost strokes `1pt`, fermion arrowheads `0.2pt`, and momentum strokes `0.4pt`. The constructor stores resolved `node-style`, `fermion`, `particles`, `momentum-stroke`, and `momentum-mark` presets in `scope.feynman`. `graph.style` uses this scope for measurement, and `draw` inherits it after layout; no repeated draw-time configuration is needed. Separate styles do not share configuration. Direct calls to the exported `feynman.node-style` and `feynman.edge-style` without scope still use the original defaults.

Particle aliases remain `a`/`photon`, `g`/`gluon`, and `scalar`/`ghG`; other particle names use the fermion preset. Hidden nodes always have radius zero and no fill or stroke. Per-edge momentum fields, including sparse `mom(...)` patches described below, retain priority over inherited edge data and keep their arrow/label placement independent of these width options.

Momentum visibility belongs to edge data: set `show-momentum: false` on an `edge(...)` or in a `graph.map` edge patch to omit its momentum arrow and label while retaining the propagator and fermion arrow. It defaults to `true`. Set `default-edge-data: (show-momentum: false)` on `graph.build` to hide all momenta, then

opt individual edges back in with `show-momentum: true`. Cut fragments inherit the flag like other edge data. Use `..feynman.draw-style` unchanged; `edge-style` no longer takes a `show-momentum` argument. Only drawing layers are suppressed: physical momentum data is retained, and hidden layout labels still participate, so the switch does not change solved graph geometry.

An edge can sparsely patch the `edge-style` passed to `draw` with `style: (...)`, compute the patch with a callback, use `auto` to delegate, or use `none` to hide its paint while retaining topology. Local style callbacks receive the complete post-layout drawing record. Custom templates can package the same policy behind their mandatory `render(config)` export; the selected application owns the meaning and schema of any extra options.

Set `edge-offset` on `draw`, or `offset` in a `source-style`/`sink-style` dictionary, to draw a fitted parallel path. Style callbacks may also return an array of dictionaries; the layers are drawn in order on the same graph edge, so parallel strokes do not require duplicate graph edges.

```
#let base-style(edge) = (
  stroke: (paint: gray, thickness: 0.7pt, cap: "round"),
)
#let offset-style(edge) = (
  offset: 0.18,
  length: 1.4,
  ratio: 0.5,
  resolve-length: "min",
  shift: 0.2,
  stroke: (paint: rgb("#2f6f4e"), thickness: 1pt, cap: "round"),
  label: [active],
  label-side: "left",
  label-gap: 0.12,
)
#let source-style(edge) = (base-style(edge), offset-style(edge))
#let sink-style(edge) = (base-style(edge), offset-style(edge) + (mark: (end: ">")))
```

The parallel path is applied to the base edge geometry before patterns and other decorations; node outsets then trim the shifted path, so shifted paths still start and end outside fitted node circles. Add `edge-length` or `length` to center-trim the shifted path to a fixed arc length, and add `edge-ratio` or `ratio` to cap it by a fraction of the full offset path length. `edge-resolve-length`
`resolve-length` decides how to combine both limits `"min"`/`"shorter"` (default), `"max"`/`"longer"`,

`"length"`/`"fixed"`, `"ratio"`/`"relative"`, `"none"`/`"full"`, or a function receiving `(offset-path-length, length, ratio)`. Lengths and shortening distances are measured after offsetting, since a curved parallel path need not have the same length as its centerline. For a finite layer, `shift` is an arc-length displacement along the complete offset path: positive values move toward the path end and values that would cross an endpoint are clamped. A layer can attach `label` content to its own path. `label-shift` (default `0`) moves the label's reference point by signed arc length from that derived path's midpoint, clamped to its endpoints: `clamp(path-length / 2 + label-shift, 0, path-length)`. Positive values move toward the path end, without moving or trimming the layer itself. `label-side` chooses `"left"` or `"right"` relative to the local path direction at the shifted point; `auto` follows the side selected by ordinary edge-label layout. With `label-style.anchor` omitted or set to `auto` (also `"auto"`), the label is centered and moved until its entire CeTZ box clears the local tangent line by `label-gap`. This measures the box, including text bounds, wrapping, padding and rotation, not the nearest point of the finite curved shaft. An explicit anchor, including `"center"`, instead sits at the shifted reference point plus `label-gap` along the chosen normal, without any box-clearance correction. Negative gaps are clamped to zero in both modes. Other `label-style` fields are forwarded to CeTZ content drawing. Automatic clearance is a local tangent-line heuristic, not collision avoidance; explicit anchors intentionally allow the box to cross that line. In `examples/map-style.typ`, `momentum-label-anchor` selects this behavior: omit it or use `auto` for box clearance, or choose e.g. `"east"` for direct anchor placement. `momentum-label-gap` supplies the gap in graph units. The momentum label uses a separate full, unpainted offset-path layer, so `momentum-label-shift` is independent of `momentum-arrow-length`, even when arrow and label shifts are equal. It defaults to the requested `momentum-arrow-shift`, not the arrow's clamped

center. Near endpoints, the label point can therefore travel farther than the finite arrow's center; changing arrow length never moves the label.

Import `momentum as mom` from `examples/map-style.typ` for compact, sparse options. `mom(side: "right", length: .7, shift: -.4, label: (gap: .2, shift: -.75, anchor: "south-west"))` expands to the existing `momentum-arrow-*` and `momentum-label-*` fields. Arrow options are `side`, `offset`, `length`, and `shift`; label options are `gap`, `shift`, and `anchor`. Only supplied options are emitted, so `mom(label: (gap: .2))` preserves an inherited arrow offset or label shift. Combine the result with other edge patches using dictionary addition, or spread it into `edge(..)`.

The attached label replaces the ordinary painted edge label, while that ordinary label may still supply the pre-layout size used by label layout and side selection; attached labels do not add a second collision constraint. When compatible source and sink layers request a centered mark, Linnest emits that mark once on the complete derived layer. The mark therefore follows `shift` instead of being centered separately on either half-edge. Set `offset-side: "label"` on an offset layer to choose the sign of `offset` so the layer is drawn on the same side of the curve as the edge label.

Pattern layers are generated over each continuous path, so internal curve segments do not restart the phase or shorten the endpoint taper.

`pattern-natural-endpoints` defaults to `false`. For built-in coil strings (`"coil"`, `"helix"`, or `"spring"`) on complete continuous paths with both endpoints anchored, `true` constructs a fitted `kurvst.coil` dictionary in Typst from the path length and layer's pattern settings, then applies it once through ordinary `kurvst.pattern`. Its half-integer coil periods retain full amplitude, inward endpoint phases, and exact endpoints, without taper, stubs, or connectors. This overrides `pattern-phase`, bypasses `pattern-fit` integer fitting, and ignores `pattern-endpoint-slope` (`endpoint-ramp: false`).

Other patterns ignore the flag. A fitted dictionary can be passed directly as `pattern` without this flag, with matching `pattern-*` settings; see the Kurvst manual's Path Patterns section for fitting and one-pass application. On curved carriers, fitted coils use local tangent/normal offsets, not evaluation at a corrected arc distance.

The gluons in `examples/xbox-common.typ` enable this option on top of the `examples/map-style.typ` preset. Without it, anchored coils use a 75%-wavelength endpoint taper (capped at half the path length), with a squared longitudinal envelope and no straight end sections. Without automatic fitting, `pattern-fit: true` adjusts `pattern-wavelength` to the nearest whole number of periods on a complete edge; `pattern-phase` is in radians, with `calc.pi / 2` giving coils matching endpoint phases. Set `pattern-endpoint-slope: 1` on a layer to allow an angled endpoint instead of a tangential one; the default is `0` and the valid range is `0` to `3`. This controls the taper envelope, not an angle, so the final direction also depends on phase, amplitude, and wavelength. Endpoints remain attached.

When `source-style` and `sink-style` share a pattern and differ only in stroke, for example to colour the two halves of a gluon differently, the whole edge is patterned once, fitted like a single edge, and cut at the edge layout point. Each half keeps its own stroke while the coil runs continuously through the split. Other split-style halves and partially anchored paths, such as crossing-gap fragments, keep the requested wavelength, taper only anchored endpoints, and preserve phase continuity across hidden spans. Neither automatic coil construction nor integer fitting applies. A path with gaps is patterned per continuous run, carrying the phase across each gap.

Paired edges are Kurvst paths split at their edge layout point. Set `edge-split-gap` on `draw` to open a centered arc-length gap there, or set `split-gap` on an individual source/sink style layer to override the global value for only that layer. Wave and coil phases continue across the hidden span. `edge-halves` and `to-cetz-edge-halves` accept the same `split-gap` control and report the effective gap if either half is too short. This cuts at `edge.pos`; it does not detect arbitrary crossings.

For a known centerline crossing, put `crossing-under: 5` (using the target's integer edge id) or `crossing-under: <bridge>` (using its Typst edge name), and optionally `crossing-gap` (default `0.55`) on the style layer that should be interrupted. Targets are resolved independently of edge order. Kurvst locates proper intersections and trims the current path by arc length; wave and coil phases continue across every hidden span. This works for dangling layers and for paired layers whose source and sink have one continuous style. Both paired half styles must name the same target and gap. A cut layer's mark is placed once on the original uncut carrier, while only its painted

path is split; this keeps the mark present without duplicating it on every fragment. A cut layer cannot participate in a subgraph underlay. Self, unknown, and invisible references are reported as errors. A valid edge pair with no proper interior intersection, including one that only shares an endpoint, is left unchanged.

```
#let crossed-style(edge) = if edge.eid == 7 {
  (stroke: black, pattern: "coil", crossing-under: 5, crossing-gap: 0.55)
} else {
  (stroke: black)
}
```

For dangling edges, `edge-dangling-tangent: "horizontal"` or `"vertical"` constrains the tangent at the free edge position while retaining the edge's `bend`; the per-layer override is `dangling-tangent`. `auto` keeps the ordinary bent route, and `anchor-control-distance` controls the constrained handle length. `bend` affects dangling edges only. A paired edge instead follows its source node, `edge.pos`, and sink node, with `edge-omega` controlling its Hobby curve.

For anchored paired edges, `anchor-control-distance` sets the control-handle distance in graph units, not the endpoint position. Each edge can specify `source-style: (anchor-control-distance: 4)` and `sink-style: (anchor-control-distance: 1.5)` to tune the two ends independently. Set `source-anchor` and `sink-anchor` in its `edge-style` to choose the tangent directions. A shared `edge-style: (anchor-control-distance: 2)` supplies both ends unless an endpoint style overrides it; `auto` is computed independently for each end. An override no longer supplies the opposite endpoint's distance.

```
#draw(
  g,
  edge-split-gap: 0.28,
  edge-dangling-tangent: "horizontal",
  source-style: edge => (stroke: black, split-gap: 0.4),
  sink-style: edge => (stroke: black, split-gap: 0.4),
)
```

Data defaults are also the global styling hook for all sources, sinks, nodes, and edges. More specific data can be added with captured named arguments on node, edge, source, and sink items, or by running `graph.map`/`graph.eval-fields` after construction. This keeps evaluated Typst values in the native data channel instead of adding renderer-specific eval fields to the Rust topology spec:

```
#let g = build({
  node(<a>)
  node(<c>)
  edge(
    source(<a>),
    <a-c>,
    sink(<c>),
    label: [a-c],
    kind: "highlight",
  )
},
  name: "demo",
  default-source-data: (style: (stroke: red + 0.5pt)),
  default-sink-data: (style: (stroke: blue + 0.5pt)),
)
#let endpoint-style(edge, side) = {
  let endpoint = edge.at(side + "-half-edge", default: none)
  if endpoint == none { return (:) }
  endpoint.data.at("style", default: (:))
}
#draw(
  layout(g),
```

```
  source-style: edge => endpoint-style(edge, "source"),
  sink-style: edge => endpoint-style(edge, "sink"),
)
```

`graph.build` also accepts comma-separated items:

```
#let g = build(
  node(<a>),
  node(<b>),
  edge(
    source(<a>, compass: "e"),
    <ab>,
    sink(<b>, compass: "w"),
    label: [ab],
    statements: (color: "0055ff", label: "ab"),
  ),
  name: "demo",
  statements: (full_num: "x + y"),
  default-node-statements: (shape: "circle"),
  default-edge-statements: (
    color: "000000",
    display-label: "$#label$",
  ),
)
```

19 subgraph module

Use `subgraph` to construct and inspect opaque zero-copy selections of half-edges. Pass those selections to graph queries, drawing, and layout functions that accept a `subgraph` argument.

19.1 Concepts

19.1.1 Subgraphs

A subgraph is a dictionary containing half-edge membership, a topology signature, subgraph-wide `data`, and per-half-edge `hedge-data`. Annotations remain Typst values: content and callbacks are not serialized through Wasm. Independent selections can annotate the same hedge without changing its graph data or each other.

- `subgraph.select(g, nodes: (), edges: (), source: (), sink: (), hedges: ())` selects the union of the supplied references. Node and edge references are Typst labels or numeric IDs, supplied in arrays. `nodes` selects incident hedges, `edges` selects both available halves, and `source` / `sink` selects the named edges' structural halves, independently of drawing orientation. `hedges` selects exact numeric hedge IDs. Unknown references and missing requested halves are errors; an isolated node contributes no hedges.
- `subgraph.label(g, label)` constructs a subgraph from a base62 label.
- `subgraph.bits(g, bits)` constructs a subgraph from a boolean hedge array.
- `subgraph.compass(g, compass)` selects half edges with a DOT compass point.
- `subgraph.hedges(sg)`, `subgraph.contains(sg, hedge)`, and `subgraph.contains-edge(sg, edge)` inspect membership. The last checks whether either half of an edge record is selected.
- `subgraph.complement(g, sg)` selects all other hedges. It preserves subgraph-wide data, but the newly selected hedges have no annotations.

`subgraph.with-data(g, sg, data: value, hedge: updater)` returns an annotated selection. `data: auto` preserves subgraph-wide data and `hedge: none` preserves hedge annotations. A non-function `hedge` value replaces every selected hedge's annotation. A callback receives its original endpoint record plus `edge`, `edge-name`, and `flow`; `data` holds the existing selection annotation and `graph-data` holds the graph's half-edge data. Its return value

replaces the annotation, even when it is `none`. To store a callback as an annotation, return it from the updater rather than passing it as the updater itself. `subgraph.hedge-data(sg, h)` reads one selected hedge's annotation.

Graph-aware consumers check a signature of IDs, names, and source/sink incidence, not graph identity or placement. Selections survive layout and drawing/data changes, but incompatible topology or names are rejected. Re-select on a cut view rather than reusing its master's selections.

Breaking change: raw subgraph bytes are no longer accepted by public APIs. Use the constructors above, or the wrapped results of `graph.cycles` and `graph.forests`, and pass the whole object, not its internal `bytes` field. `subgraph.to-label(sg)` exports membership only: rebuilding with `subgraph.label(g, label)` binds it to that graph but does not restore annotations or the old topology signature. Keep the object to preserve Typst sidecar data; a base62 label is not a complete serialization of an annotated selection.

Passing `subgraph: sg` to `draw` highlights half-edges, while `layout` restricts placement or optimization. Neither opens edges; use `graph.cut` for that.

19.1.2 Directed Cut Views

`graph.cut(g, left: left, right: right, boundary: patch)` opens **one weighted directed cut** and leaves `g` unchanged. The two selections are its drawing sides, not separate cuts or a partition of the vertices. They must be disjoint and contain exactly opposite halves of every selected paired edge in `g`. Already dangling edges cannot be selected for opening.

Think of the drawing boundaries as L and R, with a positive seam passage from R to L. Selecting an edge's source on R and sink on L follows that positive direction; swapping them reverses it. This is independent of superficial drawing orientation, including reversed fermion arrows. Side names do not place endpoints automatically: `boundary` supplies the view's geometry.

An annotation `(winding: n)` on either selected half requests a positive integer number of same-direction passages; it defaults to one. If both halves specify winding, they must agree. Each selected edge becomes a source stub, a sink stub, and `n - 1` middle segments. Thus winding two produces three segments without separate named seams or a caller-supplied ordering of cut objects. Winding is not a net count of cancelling back-and-forth crossings.

This small example uses the manual's existing imports. It stores content on the original edge and endpoints, annotates the cut sides, and opens one edge with winding two. The boundary callback places both ordinary free endpoints and the generated middle endpoints using the same record interface:

```
#let master = graph.build({
  node(<a>, pos: graph.pos(x: 0, y: 0))
  node(<b>, pos: graph.pos(x: 0, y: -2))
  edge(
    <link>,
    source(<a>, caption: [output]),
    sink(<b>, caption: [input]),
    label: [connection],
  )
})
#let left = subgraph.with-data(
  master, subgraph.select(master, sink: (<link>,)),
  data: [left boundary],
  hedge: h => (winding: 2, caption: h.graph-data.caption),
)
#let right = subgraph.with-data(
  master, subgraph.select(master, source: (<link>,)),
  data: [right boundary],
  hedge: h => (caption: h.graph-data.caption),
)
#let view = graph.cut(master, left: left, right: right, boundary: record => {
  let b = record.boundary
  (pos: graph.pos(
```

```
    x: if b.side == "left" { -3 } else { 3 },
    y: -2 * b.crossing,
  ))
})
#let view = layout(view, layout-algo: "tree")
#assert(graph.edges(master).len() == 1)
#assert(graph.edges(view).len() == 3)
#assert(graph.edge-data(view, <link.1>).label == [connection])
#assert(graph.boundaries(view).len() == 4)
#draw(view, edge-label: edge => edge.label)
```

Fragments are named `<link.0>` through `<link.n>` in underlying source-to-sink order, irrespective of L/R or drawing arrows. Unnamed edges use the native prefix `__linnest_cut_edge_ID`; generated name collisions are errors. Prefer explicit edge names when patching a view. Node, edge, and hedge `origin` refer to the immediate input graph, including on repeated cuts. Edge origins contain the input `edge` ID and `name`, plus `segment` and `winding`; uncut edges have `segment: none` and `winding: 0`. Node and hedge origins are input IDs; newly generated nodes have `origin: none`.

Physical graph, node, edge, and half-edge Typst data are remapped, including content and callbacks. Middle source hedges inherit the original sink hedge's data, and middle sink hedges inherit the original source's data, following the boundary side they continue. The cut resets fragment geometry for re-layout, rather than copying an old midpoint pin to new endpoints. Apply view-specific bends, placement, labels, and `crossing-under` targets with `graph.map`; a reference to a split edge needs the appropriate fragment name.

The `boundary` callback runs only on newly created endpoints, receiving a node or dangling-edge record and returning ordinary `graph.map`-style placement/data patches. Its `record.boundary` has output `edge` and optional `node` IDs, the original side's `hedge`, `side` (`"left"` or `"right"`), zero-based `crossing` along underlying flow, `data` from that hedge's annotation, `cut-data` from that side's subgraph-wide data, and the fragment's `origin` at creation. For example, the code above preserves each endpoint caption in `boundary.data.caption`.

On subsequent cuts, `boundary.origin` and `boundary.hedge` still refer to the input to the cut that created the endpoint; `side`, `crossing`, `data`, and `cut-data` are retained. Only `boundary.edge` and `boundary.node` remap to current anchors. Inherited boundaries retain their placements and annotations without invoking the callback again.

`graph.boundaries(view)` returns these descriptors plus current `pos`, not coordinates cached at cut time. A dangling endpoint uses its edge position and has `node: none`; a middle endpoint uses its auxiliary node position. Query after layout, or inside `draw-after`, to group solved endpoints into background boxes with existing CeTZ primitives. Auxiliary boundary nodes are automatically zero-sized and unpainted, including their labels and custom node-drawing callbacks; they remain addressable anchors for incident edges.

19.2 subgraph

- label()
- bits()
- compass()
- to-label()
- hedges()
- contains()
- select()
- with-data()
- hedge-data()
- complement()
- contains-edge()
- node-depths()

19.2.1 label

Construct a subgraph object from a base62 label.

```
#let g = graph.build({
  graph.node(<a>)
  graph.node(<b>)
  graph.edge(graph.source(<a>, compass: "e"), graph.sink(<b>))
})
#let east = subgraph.compass(g, "e")
#let same = subgraph.label(g, subgraph.to-label(east))
#subgraph.hedges(same).len()
```

19.2.1.1 Parameters

```
label(
  graph: dictionary,
  label: string
) -> dictionary
```

19.2.1.1.1 graph `dictionary`

Graph object whose half-edge set the label refers to.

19.2.1.1.2 label `string`

Base62 subgraph label returned by `to-label` or produced by Linnest.

19.2.2 bits

Construct a subgraph object from a boolean hedge array.

```
#let g = graph.build({
  graph.node(<a>)
  graph.node(<b>)
  graph.edge(graph.source(<a>), graph.sink(<b>))
})
#let first = subgraph.bits(g, (true, false))
#subgraph.contains(first, 0)
```

19.2.2.1 Parameters

```
bits(
  graph: dictionary,
  bits: array
) -> dictionary
```

19.2.2.1.1 graph `dictionary`

Graph object whose half-edge order defines the bit array.

19.2.2.1.2 bits `array`

Boolean array selecting half edges by graph half-edge index.

19.2.3 compass

Construct a subgraph object from a DOT compass point such as `"n"` or `"s"`.

```
#let g = graph.build({
  graph.node(<a>)
  graph.node(<b>)
  graph.edge(graph.source(<a>, compass: "e"), graph.sink(<b>))
})
#subgraph.hedges(subgraph.compass(g, "e")).len()
```

19.2.3.1 Parameters

```
compass(
  graph: dictionary,
  compass: string
) -> dictionary
```

19.2.3.1.1 graph `dictionary`

Graph object to filter by half-edge compass statement.

19.2.3.1.2 compass `string`

DOT compass point such as `"n"`, `"s"`, `"e"`, or `"w"`.

19.2.4 to-label

Convert a subgraph object to its base62 label.

The label encodes only membership, not topology or annotations.

```
#let g = graph.build({
  graph.node(<a>)
  graph.node(<b>)
  graph.edge(graph.source(<a>), graph.sink(<b>))
})
#subgraph.to-label(subgraph.bits(g, (true, false)))
```

19.2.4.1 Parameters

```
to-label(subgraph: dictionary) -> string
```

19.2.4.1.1 subgraph `dictionary`

Subgraph object returned by this module or by `graph.cycles`/`graph.forests`.

19.2.5 hedges

Return the hedge indices included in a subgraph object.

```
#let g = graph.build({
  graph.node(<a>)
  graph.node(<b>)
  graph.edge(graph.source(<a>), graph.sink(<b>))
})
#subgraph.hedges(subgraph.bits(g, (true, false)))
```

19.2.5.1 Parameters

`hedges(subgraph: dictionary) -> array`

19.2.5.1.1 subgraph `dictionary`

Subgraph object to inspect.

19.2.6 contains

Test whether a subgraph object includes a hedge.

```
#let g = graph.build({
  graph.node(<a>)
  graph.node(<b>)
  graph.edge(graph.source(<a>), graph.sink(<b>))
})
#subgraph.contains(subgraph.bits(g, (true, false)), 0)
```

19.2.6.1 Parameters

```
contains(
  subgraph: dictionary,
  hedge: int
) -> bool
```

19.2.6.1.1 subgraph `dictionary`

Subgraph object to inspect.

19.2.6.1.2 hedge `int`

Half-edge index to test.

19.2.7 select

Select the union of node incidences, whole edges, edge halves, and exact hedges.

Node and edge references are Typst label names or numeric IDs. `source` and `sink` refer to the structural halves, independently of drawing orientation. Missing references or absent requested halves are errors. An isolated node contributes no hedges: subgraphs represent half-edge sets, not node sets.

```
#let g = graph.build({
  graph.node(<a>)
  graph.edge(<D1>, graph.source(<a>))
  graph.edge(<D6>, graph.sink(<a>))
})
#let left = subgraph.select(g, source: (<D1>,), sink: (<D6>,))
#subgraph.hedges(left)
```

19.2.7.1 Parameters

```
select(
  g: dictionary,
  nodes: array,
  edges: array,
  source: array,
  sink: array,
  hedges: array
) -> dictionary
```

19.2.7.1.1 g `dictionary`

Graph whose records define names and IDs.

19.2.7.1.2 nodes `array`

Select all half edges incident to these nodes.

Default: `()`

19.2.7.1.3 edges `array`

Select both available halves of these edges.

Default: `()`

19.2.7.1.4 source `array`

Select only the source half of these edges.

Default: `()`

19.2.7.1.5 sink `array`

Select only the sink half of these edges.

Default: `()`

19.2.7.1.6 hedges `array`

Select these exact numeric half-edge IDs.

Default: `()`

19.2.8 with-data

Attach arbitrary Typst data without serializing it through Wasm.

`data: auto` preserves subgraph-wide data; `hedge: none` preserves annotations. A hedge callback receives the original endpoint record plus numeric `edge`, label-valued `edge-name` (or `none`), and `flow` (`"source"` or `"sink"`). Its `data` is the existing subgraph annotation; `graph-data` holds the original half-edge data. The return value replaces the annotation, including `none`. Other non-function hedge values replace every selected hedge's annotation. Only selected hedges retain annotations; callback values are never evaluated unless supplied as the `hedge` updater itself.

19.2.8.1 Parameters

```
with-data(
  g: dictionary,
  s: dictionary,
  data: any,
  hedge: any
) -> dictionary
```

19.2.8.1.1 g `dictionary`

Compatible graph supplying endpoint records and graph data.

19.2.8.1.2 s `dictionary`

Subgraph to annotate.

19.2.8.1.3 data `any`

Replacement subgraph-wide data, or `auto` to preserve it.

Default: `auto`

19.2.8.1.4 hedge `any`

Half-edge updater, constant annotation, or `none` to preserve annotations.

Default: `none`

19.2.9 hedge-data

Return a selected hedge's annotation, or `none` when it has no annotation. Unselected and out-of-range half-edge IDs are errors.

19.2.9.1 Parameters

```
hedge-data(
  s: dictionary,
  h: int
) -> any
```

19.2.9.1.1 s `dictionary`

Subgraph whose annotation is read.

19.2.9.1.2 h `int`

Selected numeric half-edge ID.

19.2.10 complement

Return the complement of a subgraph's selected half edges.

Subgraph-wide data is preserved; newly selected hedges have no annotations.

19.2.10.1 Parameters

```
complement(
  graph: dictionary,
  selected: dictionary
) -> dictionary
```

19.2.10.1.1 graph `dictionary`

Graph object whose half-edge order defines the result.

19.2.10.1.2 selected `dictionary`

Subgraph object to invert.

19.2.11 contains-edge

Test whether either half-edge of an edge record is selected.

19.2.11.1 Parameters

```
contains-edge(
  subgraph: dictionary,
  edge: dictionary
) -> bool
```

19.2.11.1.1 subgraph `dictionary`

Subgraph object to inspect.

19.2.11.1.2 edge `dictionary`

Edge record from `graph.edges(...)` or draw callback data.

19.2.12 node-depths

Compute breadth-first node depths inside a selected edge set.

19.2.12.1 Parameters

```
node-depths(
  graph: dictionary,
  selected: dictionary,
  root: int
) -> dictionary
```

19.2.12.1.1 graph `dictionary`

Graph object to inspect.

19.2.12.1.2 selected `dictionary`

Subgraph whose selected half edges define the traversal edges.

19.2.12.1.3 root `int`

Root node index.

Default: `0`

20 Version history

> **History coverage**
> Linnet and Clinnet have canonical histories. The independently versioned `linnet-py` distribution does not yet have a separate changelog in this repository.

20.1 Available histories

- Linnet Rust library versions
- Clinnet command-line renderer versions
- `linnet-py`: no standalone history yet

The changelog records user-visible graph, parser, drawing, and algorithm changes. In particular, users moving across releases should check index ordering, DOT serialization, subgraph behavior, and feature-gated integration changes rather than assuming a serialized graph or incidental ordering is stable.

20.2 Choosing the matching documentation

The `latest` channel may describe unreleased behavior. Documentation under `snapshots/<tag>` is fixed to a tagged repository revision. Choose a snapshot whose component information lists your Rust crate version, and check the `linnet-py` version separately when using the Python bindings.

20.3 Upgrade checklist

Review changes to index ordering, DOT serialization, subgraph behavior, and enabled features. Re-validate serialized graphs and any code that depends on traversal or iteration order.

20.4 Reproducibility record

Record the repository revision, `linnet` and `linnet-py` versions, enabled features, and the input graph format. These distinguish a library change from a parser, serializer, or binding change.

21 Linnet versions

This page renders the canonical Linnet changelog source as part of the Linnet website, navigation, and search.

All notable changes to this project will be documented in this file.

The format is based on Keep a Changelog, and this project adheres to Semantic Versioning.

22.1 Unreleased

22.2 0.17.0 - 2026-01-28

22.2.1 Other

- force-directed layout
- make the powerset iterator properly size the subset
- upgrade to all spanning *forests* generation
- Refactor PowersetIterator to support generic ID type
- double ended iterator for subsets
- remove unused crates

22.3 0.16.1 - 2026-01-13

22.3.1 Other

- Quote string values in graph and parser outputs
- revert edge, node and hedge index order to be direct (not based on sort)

22.4 0.16.0 - 2026-01-08

22.4.1 Other

- Add methods for identifying bridges and non-bridges in subgraphs
- Make `trace_unfold` implementable outside of linnet

22.5 0.15.1 - 2026-01-05

22.5.1 Fixed

- fixed ordering problem in merge function and make ordering based on sorting not on direct map

22.5.2 Other

- update quote stripping
- Add agent context files and update project tooling
- Add graph algorithms module with topological sort and transitive ops

22.6 0.15.0 - 2025-12-08

22.6.1 Other

- Refactor Figment handling and improve data merging logic
- *(clinnet)* release v0.1.8

22.7 0.14.1 - 2025-12-03

22.7.1 Other

- Add parallel processing and directional constraints
- update test
- only swap one edge when connecting identities

22.8 0.14.0 - 2025-11-28

22.8.1 Other

- change output name and make it short, and add directional groups

22.9 0.13.3 - 2025-11-24

22.9.1 Other

- update symbolica

22.10 0.13.2 - 2025-11-12

22.10.1 Other

- update to symbolica 0.20
- better templates
- clinnet for nested dots to pdf
- Add Figment integration for configuration handling

22.11 0.13.1 - 2025-11-05

22.11.1 Other

- remove prints

22.12 0.12.0 - 2025-08-18

22.12.1 Other

- all tests pass
- clippy fixes
- write global data outside of `graph[]` scope
- Add ID=ID statements to global data when parsing
- newline after nodes
- dot serialize with name
- write graph name
- correctly parse orientations
- make `iter_edges` iterate with `edge_id` order not involution order

22.13 0.11.0 - 2025-07-29

22.13.1 Other

- serialize global data

22.14 0.10.0 - 2025-07-29

22.14.1 Other

- Add check graph function and extracting by nodes
- Add `is_empty` to Swap trait and update implementations

22.15 0.9.1 - 2025-07-28

22.15.1 Other

- impl asref for newtypes

22.16 0.9.0 - 2025-07-27

22.16.1 Other

- Allow specifying a subgraph in the DOT format.
- Rewrite DOT output to include node, hedge and edge indices and fix bidirectional serialization
- DotGraph has concrete data. Parser now supports global edge and vertex data. Can set edge, vertex and hedge index
- add multi-graph parsing and global graph data

22.17 0.8.0 - 2025-07-14

22.17.1 Other

- clippy fix
- Rename HedgeVec to EdgeVec and introduce HedgeVec
- Add H parameter to HedgeGraph in `iter_edges` and orientation
- add H to SignedCycle
- rename `as_ref` to `to_ref`
- Add `hedge_data` to `hedge_graph`
- add perm by key
- update symbolica
- Fix tree traversal and modify ForestNodeStore API to use Option

22.18 0.7.1 - 2025-05-24

22.18.1 Other

- Fix crown computation by inverting subgraph inclusion check and add inclusion for HedgePair

22.19 0.7.0 - 2025-05-23

22.19.1 Other

- unify iterator signature and remove redundant and misleading functions
- Add zenodo doi

22.20 0.6.3 - 2025-05-23

22.20.1 Other

- improve README

22.21 0.6.2 - 2025-05-22

22.21.1 Other

- remove useless check and set
- Correctly forget identification history and adds subgraph contraction

22.22 0.6.1 - 2025-05-22

22.22.1 Other

- Merge pull request #11 from `alphal00p/jules_wip_3253120278989361716`

22.23 0.6.0 - 2025-05-22

22.23.1 Fixed

- fix all tests and update symbolica
- fixes `forget_identification_history`, If you delete a subgraph you need

22.23.2 Other

- Add crown functions and implement spanning tree enumeration algo based on graph contraction
- PartialEq and Eq for hedgevec
- `from_raw` for hedgevec

22.24 0.5.2 - 2025-05-09

22.24.1 Fixed

- fix involution extraction

22.24.2 Other

- Add Permutation iterators, new graph extract test

22.25 0.5.1 - 2025-05-05

22.25.1 Other

- Clean up debug prints and add cycle test

22.26 0.5.0 - 2025-05-05

22.26.1 Fixed

- fixes all around

22.26.2 Other

- boundary->crown
- hairs->boundary

22.27 0.4.0 - 2025-04-30

22.27.1 Added

- feature flag bincode and serde

22.28 0.3.0 - 2025-04-30

22.28.1 Fixed

- fixed extraction for childvec forest, all tests pass and fix warnings

22.28.2 Other

- adds extract to SmartHedgeVec and involution
- maps now take a FnMut
- derive more (Decode and Encode)

22.29 0.2.3 - 2025-04-08

22.29.1 Other

- unpin serde

22.30 0.2.2 - 2025-04-01

22.30.1 Fixed

- fix test snapshots

22.30.2 Other

- raw image

22.31 0.2.1 - 2025-04-01

22.31.1 Other

- Update README.md, logo is a github asset
- put crates badge

22.32 0.2.0 - 2025-04-01

22.32.1 Fixed

- fix cetz orientation

22.32.2 Other

- Create release-plz.yml

23 Clinnet versions

Clinnet is versioned independently from the Linnet library. Its canonical version history remains the Clinnet changelog source. This rendered copy makes that history part of the manual, navigation, search, and product PDF.

All notable changes to this project will be documented in this file.

The format is based on Keep a Changelog, and this project adheres to Semantic Versioning.

24.1 Unreleased

24.2 0.1.8 - 2025-12-04

24.2.1 Other

- redo cli to have subcommands

24.3 0.1.7 - 2025-12-04

24.3.1 Other

- properly forward inputs for a single figure

24.4 0.1.6 - 2025-12-03

24.4.1 Other

- Add parallel processing and directional constraints
- add inputs

24.5 0.1.5 - 2025-11-28

24.5.1 Other

- change output name and make it short, and add directional groups

24.6 0.1.4 - 2025-11-26

24.6.1 Other

- Make figure and grid templates optional with defaults

24.7 0.1.3 - 2025-11-26

24.7.1 Other

- don't overwrite existing templates
- strip quotes strings for typst

24.8 0.1.2 - 2025-11-24

24.8.1 Other

- update symbolica

24.9 0.1.1 - 2025-11-12

24.9.1 Other

- remove unnecessary outputs
- better templates

Python API reference

The supported Python packages available in this version are listed below. The HTML manual provides a separate page for every public class, function, and constant; this printable edition keeps a compact inventory. Rust APIs use canonical Rustdoc in the HTML manual.

`linnet-py`

`Exports registered by linnet-py.`

`AUTO`

`Exported constant of type `Auto`.`

`Exported constant of type `Auto`.`

`AUTO: Auto`

Requires features: `extension-module, abi3`

Exhaustive reference · Source

`Angle`

`A Typst angle in degrees or radians.`

`A Typst angle in degrees or radians.`

`class Angle:`

Requires features: `extension-module, abi3`

`__new__`

Kind: `AssociatedFunction`

`def __new__(cls, value: float, unit: Literal['deg', 'rad']) -> Angle:`

`value`

Kind: `Parameter`

`float`

`unit`

Kind: `Parameter`

`Literal['deg', 'rad']`

`degrees`

Kind: `AssociatedFunction`

`def degrees(value: float) -> Angle:`

`value`

Kind: `Parameter`

`float`

`radians`

Kind: `AssociatedFunction`

`def radians(value: float) -> Angle:`

`value`

Kind: `Parameter`

float

__repr__
Kind: Method

def __repr__(self) -> str:

Exhaustive reference · Source

Auto
Type of the `AUTO` sentinel, which requests automatic selection.

Type of the `AUTO` sentinel, which requests automatic selection.

class Auto:

Requires features: extension-module, abi3

__repr__
Kind: Method

def __repr__(self) -> str:

__copy__
Kind: Method

def __copy__(self) -> Auto:

__deepcopy__
Kind: Method

def __deepcopy__(self, _memo: Any) -> Auto:

_memo
Kind: Parameter

Any

Exhaustive reference · Source

Color
A safe Typst color value.

A safe Typst color value.

class Color:

Requires features: extension-module, abi3

__new__
Kind: AssociatedFunction

def __new__(cls, value: str) -> Color:

value
Kind: Parameter

str

rgb
Kind: AssociatedFunction

def rgb(red: int, green: int, blue: int, alpha: Optional[int] = None) -> Color:

red
Kind: `Parameter`

`int`

green
Kind: `Parameter`

`int`

blue
Kind: `Parameter`

`int`

alpha
Kind: `Parameter`

`Optional[int]`

Default: `None`

__repr__
Kind: `Method`

```
def __repr__(self) -> str:
```

Exhaustive reference · Source

CutPartition
`One source-side, oriented-boundary, target-side cut partition.`

`One source-side, oriented-boundary, target-side cut partition.`

```
class CutPartition:
```

Requires features: `extension-module, abi3`

source_side
Kind: `Getter`

```
def source_side(self) -> Subgraph:
```

boundary
Kind: `Getter`

```
def boundary(self) -> OrientedCut:
```

target_side
Kind: `Getter`

```
def target_side(self) -> Subgraph:
```

edges
Kind: `Getter`

```
def edges(self) -> list[Edge]:
```

__repr__
Kind: `Method`

```
def __repr__(self) -> str:
```

Exhaustive reference · Source

`Cycle`

`A cycle in a particular graph revision.`

`A cycle in a particular graph revision.`

```
class Cycle:
```

Requires features: `extension-module, abi3`

`filter`

Kind: `Getter`

```
def filter(self) -> Subgraph:
```

`loop_count`

Kind: `Getter`

```
def loop_count(self) -> Optional[int]:
```

`__repr__`

Kind: `Method`

```
def __repr__(self) -> str:
```

Exhaustive reference · Source

`Dash`

`A named or explicit Typst dash pattern.`

`A named or explicit Typst dash pattern.`

```
class Dash:
```

Requires features: `extension-module, abi3`

`__new__`

Kind: `AssociatedFunction`

```
def __new__(cls, pattern: DashPattern) -> Dash:
```

`pattern`

Kind: `Parameter`

`DashPattern`

`pattern`

Kind: `AssociatedFunction`

```
def pattern(values: _LengthArray, *, phase: _LengthValue | None = None) -> Dash:
```

`values`

Kind: `Parameter`

`_LengthArray`

`phase`

Kind: `Parameter`

`_LengthValue | None`

Default: `None`

`__repr__`

Kind: `Method`

```
def __repr__(self) -> str:
```

Exhaustive reference · Source

`DebugLevel`

```
Finite debug detail accepted by Linnest's drawing layer.
```


```
Finite debug detail accepted by Linnest's drawing layer.
```


```
class DebugLevel(enum.Enum):
```

Requires features: `extension-module, abi3`

`Off`

Kind: `Variant`

`Canvas`

Kind: `Variant`

`EdgePositions`

Kind: `Variant`

Exhaustive reference · Source

`DirectionBasis`

```
Selects which notion of edge direction a directed algorithm follows.
```


```
Selects which notion of edge direction a directed algorithm follows.
```


```
class DirectionBasis(enum.Enum):
```

Requires features: `extension-module, abi3`

`Underlying`

Kind: `Variant`

```
Follow the source/sink roles stored by the half-edge involution.
```

`Superficial`

Kind: `Variant`

```
Apply each edge's superficial orientation and ignore undirected edges.
```

Exhaustive reference · Source

`DotCodec`

```
An explicit mapping between graph values and DOT-representable records.
```


```
An explicit mapping between graph values and DOT-representable records.
```


```
class DotCodec:
```

Requires features: `extension-module, abi3`

`__new__`

Kind: `AssociatedFunction`

```
def __new__(cls, *, encode_node: Callable[[NodeValue], DotVertexData], decode_node:
Callable[[DotVertexData], NodeValue], encode_edge: Callable[[EdgeValue],
DotEdgeData], decode_edge: Callable[[DotEdgeData], EdgeValue], encode_half_edge:
```

```
Callable[[HalfEdgeValue], DotHalfEdgeData], decode_half_edge:
Callable[[DotHalfEdgeData], HalfEdgeValue]) -> DotCodec:
```

encode_node

Kind: Parameter

```
Callable[[NodeValue], DotVertexData]
```

decode_node

Kind: Parameter

```
Callable[[DotVertexData], NodeValue]
```

encode_edge

Kind: Parameter

```
Callable[[EdgeValue], DotEdgeData]
```

decode_edge

Kind: Parameter

```
Callable[[DotEdgeData], EdgeValue]
```

encode_half_edge

Kind: Parameter

```
Callable[[HalfEdgeValue], DotHalfEdgeData]
```

decode_half_edge

Kind: Parameter

```
Callable[[DotHalfEdgeData], HalfEdgeValue]
```

linnest

Kind: AssociatedFunction

```
def linnest(cls) -> DotCodec:
```

topology

Kind: AssociatedFunction

```
def topology(cls) -> DotCodec:
```

Exhaustive reference · Source

DotEdgeData

```
The DOT-representable record exchanged for one edge by a `DotCodec`.
```


```
The DOT-representable record exchanged for one edge by a `DotCodec`.
```


```
class DotEdgeData:
```

Requires features: `extension-module, abi3`

edge_id

Kind: Getter

```
def edge_id(self) -> Optional[int]:
```

payload

Kind: Getter

```
def payload(self) -> Optional[list[int]]:
```

statements

Kind: Getter

```
def statements(self) -> dict[str, str]:
```

local_statements

Kind: Getter

```
def local_statements(self) -> dict[str, str]:
```

__new__

Kind: AssociatedFunction

```
def __new__(cls, *, edge_id: Optional[int] = None, payload: Optional[Sequence[int]] =
None, statements: Optional[Mapping[str, str]] = None, local_statements:
Optional[Mapping[str, str]] = None) -> DotEdgeData:
```

edge_id

Kind: Parameter

```
Optional[int]
```

Default: None

payload

Kind: Parameter

```
Optional[Sequence[int]]
```

Default: None

statements

Kind: Parameter

```
Optional[Mapping[str, str]]
```

Default: None

local_statements

Kind: Parameter

```
Optional[Mapping[str, str]]
```

Default: None

Exhaustive reference · Source

DotHalfEdgeData

```
The DOT-representable record exchanged for one half-edge by a `DotCodec`.
```


```
The DOT-representable record exchanged for one half-edge by a `DotCodec`.
```


```
class DotHalfEdgeData:
```

Requires features: `extension-module, abi3`

statement

Kind: Getter

```
def statement(self) -> Optional[str]:
```

index

Kind: Getter

```
def index(self) -> Optional[int]:
```

payload

Kind: Getter

```
def payload(self) -> Optional[list[int]]:
```

port_label

Kind: Getter

```
def port_label(self) -> Optional[str]:
```

compass

Kind: Getter

```
def compass(self) -> Optional[str]:
```

__new__

Kind: AssociatedFunction

```
def __new__(cls, *, statement: Optional[str] = None, index: Optional[int] = None,
payload: Optional[Sequence[int]] = None, port_label: Optional[str] = None, compass:
Optional[str] = None) -> DotHalfEdgeData:
```

statement

Kind: Parameter

```
Optional[str]
```

Default: None

index

Kind: Parameter

```
Optional[int]
```

Default: None

payload

Kind: Parameter

```
Optional[Sequence[int]]
```

Default: None

port_label

Kind: Parameter

```
Optional[str]
```

Default: None

compass

Kind: Parameter

```
Optional[str]
```

Default: None

Exhaustive reference · Source

DotVertexData

The DOT-representable record exchanged for one node by a `DotCodec`.

The DOT-representable record exchanged for one node by a `DotCodec`.

```
class DotVertexData:
```

Requires features: `extension-module, abi3`

name

Kind: `Getter`

```
def name(self) -> Optional[str]:
```

index

Kind: `Getter`

```
def index(self) -> Optional[int]:
```

payload

Kind: `Getter`

```
def payload(self) -> Optional[list[int]]:
```

statements

Kind: `Getter`

```
def statements(self) -> dict[str, str]:
```

__new__

Kind: `AssociatedFunction`

```
def __new__(cls, *, name: Optional[str] = None, index: Optional[int] = None, payload:
Optional[Sequence[int]] = None, statements: Optional[Mapping[str, str]] = None) ->
DotVertexData:
```

name

Kind: `Parameter`

`Optional[str]`

Default: `None`

index

Kind: `Parameter`

`Optional[int]`

Default: `None`

payload

Kind: `Parameter`

`Optional[Sequence[int]]`

Default: `None`

statements

Kind: `Parameter`

`Optional[Mapping[str, str]]`

Default: `None`

Exhaustive reference · Source

`DrawOptions`

`Full typed option surface for `linnest.draw`.`

`Full typed option surface for `linnest.draw`.`

`class DrawOptions:`

Requires features: `extension-module, abi3`

`__repr__`

Kind: `Method`

```
def __repr__(self) -> str:
```

`__new__`

Kind: `AssociatedFunction`

```
def __new__(cls, *, scope: _Dictionary = ..., unit: _AutoLengthValue = ..., title:
_AutoOptionalStaticContent = ..., subgraph: _DrawSubgraphs = ..., debug: _DebugValue
= ..., show_half_edge_ids: _Boolean = ..., node_radius: _AutoRadius = ...,
node_min_radius: _Number = ..., node_label_padding: _Number = ..., node_fill: _Paint
= ..., node_stroke: _StrokeValue = ..., node_outset: _AutoNumber = ...,
node_label_style: _Style = ..., node_style: _OptionalStyle = ..., node_label:
_AutoOptionalContent = ..., draw_node: _AutoFunction = ..., edge_stroke: _StrokeValue
= ..., edge_offset: _Number = ..., edge_length: _OptionalNumber = ..., edge_ratio:
_OptionalNumber = ..., edge_resolve_length: _EdgeLengthResolver = ..., edge_accuracy:
_Number = ..., edge_optimize: _Boolean = ..., edge_split_gap: _Number = ...,
edge_dangling_tangent: _DanglingTangentValue = ..., source_style:
_OptionalStyleLayers = ..., sink_style: _OptionalStyleLayers = ..., edge_label:
_OptionalContent = ..., edge_label_style: _OptionalStyle = ..., edge_omega: _Number
= ..., edge_trim_accuracy: _Number = ..., padding: _OptionalPadding = ...,
debug_edge_radius: _Number = ..., debug_edge_fill: _Paint = ..., debug_edge_stroke:
_StrokeValue = ..., debug_edge_label_fill: _Paint = ..., subgraph_edge_style: _Style
= ..., subgraph_edge_underlay: _Boolean = ...) -> DrawOptions:
```

`scope`

Kind: `Parameter`

`_Dictionary`

Default: `...`

`unit`

Kind: `Parameter`

`_AutoLengthValue`

Default: `...`

`title`

Kind: `Parameter`

`_AutoOptionalStaticContent`

Default: `...`

subgraph
Kind: Parameter

_DrawSubgraphs

Default: ...

debug
Kind: Parameter

_DebugValue

Default: ...

show_half_edge_ids
Kind: Parameter

_Boolean

Default: ...

node_radius
Kind: Parameter

_AutoRadius

Default: ...

node_min_radius
Kind: Parameter

_Number

Default: ...

node_label_padding
Kind: Parameter

_Number

Default: ...

node_fill
Kind: Parameter

_Paint

Default: ...

node_stroke
Kind: Parameter

_StrokeValue

Default: ...

node_outset
Kind: Parameter

_AutoNumber

Default: ...

node_label_style
Kind: Parameter

_Style

Default: ...

node_style
Kind: Parameter

_OptionalStyle

Default: ...

node_label
Kind: Parameter

_AutoOptionalContent

Default: ...

draw_node
Kind: Parameter

_AutoFunction

Default: ...

edge_stroke
Kind: Parameter

_StrokeValue

Default: ...

edge_offset
Kind: Parameter

_Number

Default: ...

edge_length
Kind: Parameter

_OptionalNumber

Default: ...

edge_ratio
Kind: Parameter

_OptionalNumber

Default: ...

edge_resolve_length
Kind: Parameter

_EdgeLengthResolver

Default: ...

edge_accuracy
Kind: Parameter

_Number

Default: ...

edge_optimize
Kind: Parameter

_Boolean

Default: ...

edge_split_gap
Kind: Parameter

_Number

Default: ...

edge_dangling_tangent
Kind: Parameter

_DanglingTangentValue

Default: ...

source_style
Kind: Parameter

_OptionalStyleLayers

Default: ...

sink_style
Kind: Parameter

_OptionalStyleLayers

Default: ...

edge_label
Kind: Parameter

_OptionalContent

Default: ...

edge_label_style
Kind: Parameter

_OptionalStyle

Default: ...

edge_omega
Kind: Parameter

_Number

Default: ...

edge_trim_accuracy
Kind: Parameter

_Number

Default: ...

padding
Kind: Parameter

_OptionalPadding

Default: ...

debug_edge_radius
Kind: Parameter

_Number

Default: ...

debug_edge_fill
Kind: Parameter

_Paint

Default: ...

debug_edge_stroke
Kind: Parameter

_StrokeValue

Default: ...

debug_edge_label_fill
Kind: Parameter

_Paint

Default: ...

subgraph_edge_style
Kind: Parameter

_Style

Default: ...

subgraph_edge_underlay
Kind: Parameter

_Boolean

Default: ...

Exhaustive reference · Source

DrawingSelectors
Per-render Python callbacks returning typed drawing patches.

Per-render Python callbacks returning typed drawing patches.

class DrawingSelectors:

Requires features: extension-module, abi3

__repr__
Kind: Method

def __repr__(self) -> str:

__new__
Kind: AssociatedFunction

def __new__(cls, *, node: _NodeDrawingSelector = ..., edge: _EdgeDrawingSelector = ..., source: _HalfEdgeDrawingSelector = ..., sink: _HalfEdgeDrawingSelector = ...) -> DrawingSelectors:

node
Kind: Parameter

_NodeDrawingSelector

Default: ...

edge
Kind: Parameter

_EdgeDrawingSelector

Default: ...

source
Kind: Parameter

_HalfEdgeDrawingSelector

Default: ...

sink
Kind: Parameter

_HalfEdgeDrawingSelector

Default: ...

Exhaustive reference · Source

EdgeSpec
A reusable declarative edge description accepted by `build()` and `Graph.add_edge()`.

A reusable declarative edge description accepted by `build()` and `Graph.add_edge()`.

class EdgeSpec:

Requires features: extension-module, abi3

name
Kind: Getter

def name(self) -> Optional[str]:

data
Kind: Getter

```
def data(self) -> Any:
```

data
Kind: Setter

```
def data(self, value: Any) -> None:
```

value
Kind: Parameter

```
Any
```

drawing
Kind: Getter

```
def drawing(self) -> EdgeDrawing:
```

orientation
Kind: Getter

```
def orientation(self) -> Orientation:
```

Exhaustive reference · Source

Flow
```
The directed role of a half-edge within its edge.
```

```
The directed role of a half-edge within its edge.
```

```
class Flow(enum.Enum):
```

Requires features: `extension-module, abi3`

Source
Kind: Variant

Sink
Kind: Variant

Exhaustive reference · Source

Fraction
```
A Typst fractional track size such as `1fr`.
```

```
A Typst fractional track size such as `1fr`.
```

```
class Fraction:
```

Requires features: `extension-module, abi3`

value
Kind: Getter

```
def value(self) -> float:
```

__new__
Kind: AssociatedFunction

```
def __new__(cls, value: float) -> Fraction:
```

value
Kind: Parameter

`float`

`__repr__`
Kind: `Method`

```
def __repr__(self) -> str:
```

Exhaustive reference · Source

`GlobalData`
`DOT graph metadata kept separate from arbitrary Python element data.`

`DOT graph metadata kept separate from arbitrary Python element data.`

```
class GlobalData:
```

Requires features: `extension-module, abi3`

`name`
Kind: `Getter`

```
def name(self) -> str:
```

`name`
Kind: `Setter`

```
def name(self, value: str) -> None:
```

`value`
Kind: `Parameter`

```
str
```

`payload`
Kind: `Getter`

```
def payload(self) -> Optional[list[int]]:
```

`payload`
Kind: `Setter`

```
def payload(self, value: Optional[list[int]]) -> None:
```

`value`
Kind: `Parameter`

```
Optional[list[int]]
```

`statements`
Kind: `Getter`

```
def statements(self) -> dict[str, str]:
```

`statements`
Kind: `Setter`

```
def statements(self, value: dict[str, str]) -> None:
```

`value`
Kind: `Parameter`

```
dict[str, str]
```

edge_statements

Kind: Getter

```
def edge_statements(self) -> dict[str, str]:
```

edge_statements

Kind: Setter

```
def edge_statements(self, value: dict[str, str]) -> None:
```

value

Kind: Parameter

```
dict[str, str]
```

node_statements

Kind: Getter

```
def node_statements(self) -> dict[str, str]:
```

node_statements

Kind: Setter

```
def node_statements(self, value: dict[str, str]) -> None:
```

value

Kind: Parameter

```
dict[str, str]
```

__new__

Kind: AssociatedFunction

```
def __new__(cls, name: str = '', *, payload: Optional[Sequence[int]] = None,
statements: Optional[Mapping[str, str]] = None, edge_statements:
Optional[Mapping[str, str]] = None, node_statements: Optional[Mapping[str, str]] =
None) -> GlobalData:
```

name

Kind: Parameter

```
str
```

Default: `''`

payload

Kind: Parameter

```
Optional[Sequence[int]]
```

Default: `None`

statements

Kind: Parameter

```
Optional[Mapping[str, str]]
```

Default: `None`

edge_statements

Kind: Parameter

Optional[Mapping[str, str]]

Default: None

node_statements
Kind: Parameter

Optional[Mapping[str, str]]

Default: None

__repr__
Kind: Method

def __repr__(self) -> str:

Exhaustive reference · Source

Graph
An owned topology with arbitrary element data and typed rendering configuration.

An owned topology with arbitrary element data and typed rendering configuration.

class Graph:

Requires features: extension-module, abi3

name
Kind: Getter

def name(self) -> Optional[str]:

global_data
Kind: Getter

def global_data(self) -> GlobalData:

global_data
Kind: Setter

def global_data(self, value: GlobalData) -> None:

value
Kind: Parameter

GlobalData

render_config
Kind: Getter

def render_config(self) -> RenderConfig:

render_config
Kind: Setter

def render_config(self, value: RenderConfig) -> None:

value
Kind: Parameter

RenderConfig

n_nodes

Kind: Getter

```
def n_nodes(self) -> int:
```

n_edges

Kind: Getter

```
def n_edges(self) -> int:
```

n_half_edges

Kind: Getter

```
def n_half_edges(self) -> int:
```

node_store

Kind: Getter

```
def node_store(self) -> NodeStore:
```

Return the node-storage strategy used by this graph.

__new__

Kind: AssociatedFunction

```
def __new__(cls, *, name: Optional[str] = None, global_data: Optional[GlobalData] =
None, codec: Optional[DotCodec] = None, render_config: RenderConfig | None = None,
node_store: NodeStore = NodeStore.Vec) -> Graph:
```

name

Kind: Parameter

```
Optional[str]
```

Default: None

global_data

Kind: Parameter

```
Optional[GlobalData]
```

Default: None

codec

Kind: Parameter

```
Optional[DotCodec]
```

Default: None

render_config

Kind: Parameter

```
RenderConfig | None
```

Default: None

node_store

Kind: Parameter

```
NodeStore
```

Default: `NodeStore.Vec`

`__len__`
Kind: `Method`

```
def __len__(self) -> int:
```

`to_node_store`
Kind: `Method`

```
def to_node_store(self, node_store: NodeStore) -> Graph:
```

Copy this graph into another node-storage strategy.

`node_store`
Kind: `Parameter`

```
NodeStore
```

`node`
Kind: `Method`

```
def node(self, key: int | str) -> Node:
```

`key`
Kind: `Parameter`

```
int | str
```

`edge`
Kind: `Method`

```
def edge(self, key: int | str) -> Edge:
```

`key`
Kind: `Parameter`

```
int | str
```

`half_edge`
Kind: `Method`

```
def half_edge(self, index: int) -> HalfEdge:
```

`index`
Kind: `Parameter`

```
int
```

`nodes`
Kind: `Method`

```
def nodes(self) -> list[Node]:
```

`edges`
Kind: `Method`

```
def edges(self) -> list[Edge]:
```

`half_edges`
Kind: `Method`

```
def half_edges(self) -> list[HalfEdge]:
```

reorder_nodes

Kind: Method

```
def reorder_nodes(self, order: Sequence[int]) -> None:
```

order

Kind: Parameter

```
Sequence[int]
```

reorder_edges

Kind: Method

```
def reorder_edges(self, order: Sequence[int]) -> None:
```

order

Kind: Parameter

```
Sequence[int]
```

reverse_edge

Kind: Method

```
def reverse_edge(self, key: int | str) -> None:
```

key

Kind: Parameter

```
int | str
```

map

Kind: Method

```
def map(self, *, node: Callable[[Node], Any] | None = None, edge: Callable[[Edge],
Any] | None = None, source: Callable[[HalfEdge], Any] | None = None, sink:
Callable[[HalfEdge], Any] | None = None) -> Graph:
```

node

Kind: Parameter

```
Callable[[Node], Any] | None
```

Default: None

edge

Kind: Parameter

```
Callable[[Edge], Any] | None
```

Default: None

source

Kind: Parameter

```
Callable[[HalfEdge], Any] | None
```

Default: None

sink

Kind: Parameter

```
Callable[[HalfEdge], Any] | None
```

Default: None

from_dot

Kind: AssociatedFunction

```
def from_dot(cls, source: str, codec: DotCodec, *, node_store: NodeStore =
NodeStore.Vec) -> Graph:
```

source

Kind: Parameter

```
str
```

codec

Kind: Parameter

```
DotCodec
```

node_store

Kind: Parameter

```
NodeStore
```

Default: NodeStore.Vec

from_dot_set

Kind: AssociatedFunction

```
def from_dot_set(cls, source: str, codec: DotCodec, *, node_store: NodeStore =
NodeStore.Vec) -> list[Graph]:
```

source

Kind: Parameter

```
str
```

codec

Kind: Parameter

```
DotCodec
```

node_store

Kind: Parameter

```
NodeStore
```

Default: NodeStore.Vec

from_dot_file

Kind: AssociatedFunction

```
def from_dot_file(cls, path: str | os.PathLike[str], codec: DotCodec, *, node_store:
NodeStore = NodeStore.Vec) -> Graph:
```

path

Kind: Parameter

```
str | os.PathLike[str]
```

codec

Kind: Parameter

```
DotCodec
```

node_store

Kind: Parameter

```
NodeStore
```

Default: `NodeStore.Vec`

to_dot

Kind: Method

```
def to_dot(self, codec: Optional[DotCodec] = None) -> str:
```

codec

Kind: Parameter

```
Optional[DotCodec]
```

Default: `None`

prepare_render

Kind: Method

```
def prepare_render(self, *, config: RenderConfig | None = None) -> PreparedRender:
```

Stage one render so its exact Typst source and compiled output stay correlated.

config

Kind: Parameter

```
RenderConfig | None
```

Default: `None`

render

Kind: Method

```
def render(self, output: str | os.PathLike[str], *, config: RenderConfig | None =
None) -> pathlib.Path:
```

output

Kind: Parameter

```
str | os.PathLike[str]
```

config

Kind: Parameter

```
RenderConfig | None
```

Default: `None`

to_svg

Kind: Method

```
def to_svg(self, *, config: RenderConfig | None = None) -> str:
```

config

Kind: Parameter

```
RenderConfig | None
```

Default: None

_repr_svg_

Kind: Method

```
def _repr_svg_(self) -> str:
```

full_subgraph

Kind: Method

```
def full_subgraph(self) -> Subgraph:
```

empty_subgraph

Kind: Method

```
def empty_subgraph(self) -> Subgraph:
```

subgraph

Kind: Method

```
def subgraph(self, *, nodes: Sequence[int | str] = ..., edges: Sequence[int | str]
= ..., half_edges: Sequence[int] = ...) -> Subgraph:
```

nodes

Kind: Parameter

```
Sequence[int | str]
```

Default: ...

edges

Kind: Parameter

```
Sequence[int | str]
```

Default: ...

half_edges

Kind: Parameter

```
Sequence[int]
```

Default: ...

filter

Kind: Method

```
def filter(self, *, node: Callable[[Node], bool] | None = None, edge:
Callable[[Edge], bool] | None = None, half_edge: Callable[[HalfEdge], bool] | None =
None) -> Subgraph:
```

node

Kind: Parameter

```
Callable[[Node], bool] | None
```

Default: `None`

edge
Kind: `Parameter`

`Callable[[Edge], bool] | None`

Default: `None`

half_edge
Kind: `Parameter`

`Callable[[HalfEdge], bool] | None`

Default: `None`

nodes_of
Kind: `Method`

```
def nodes_of(self, subgraph: Subgraph) -> list[Node]:
```

subgraph
Kind: `Parameter`

`Subgraph`

edges_of
Kind: `Method`

```
def edges_of(self, subgraph: Subgraph) -> list[Edge]:
```

subgraph
Kind: `Parameter`

`Subgraph`

half_edges_of
Kind: `Method`

```
def half_edges_of(self, subgraph: Subgraph) -> list[HalfEdge]:
```

subgraph
Kind: `Parameter`

`Subgraph`

internal_boundary
Kind: `Method`

```
def internal_boundary(self, subgraph: Subgraph) -> Subgraph:
```

Return selected half-edges that lie on the selection's internal boundary.

subgraph
Kind: `Parameter`

`Subgraph`

boundary
Kind: `Method`

```
def boundary(self, subgraph: Subgraph) -> Subgraph:
```

Return all boundary half-edges incident to nodes touched by the selection.

subgraph

Kind: Parameter

Subgraph

external_half_edges

Kind: Method

```
def external_half_edges(self) -> Subgraph:
```

Return dangling/external half-edges as a composable structural selection.

connected_components

Kind: Method

```
def connected_components(self, subgraph: Optional[Subgraph] = None) ->
list[Subgraph]:
```

subgraph

Kind: Parameter

Optional[Subgraph]

Default: None

count_connected_components

Kind: Method

```
def count_connected_components(self, subgraph: Optional[Subgraph] = None) -> int:
```

subgraph

Kind: Parameter

Optional[Subgraph]

Default: None

is_connected

Kind: Method

```
def is_connected(self, subgraph: Optional[Subgraph] = None) -> bool:
```

subgraph

Kind: Parameter

Optional[Subgraph]

Default: None

cyclomatic_number

Kind: Method

```
def cyclomatic_number(self, subgraph: Optional[Subgraph] = None) -> int:
```

subgraph

Kind: Parameter

Optional[Subgraph]

Default: `None`

`is_reachable`

Kind: `Method`

```
def is_reachable(self, source: Node | int | str, target: Node | int | str, *,
subgraph: Optional[Subgraph] = None, direction: DirectionBasis =
DirectionBasis.Underlying) -> bool:
```

Test directed reachability within an optional structural selection.

Both endpoints must belong to the selected subgraph; otherwise this raises
`ValueError`.

`source`

Kind: `Parameter`

```
Node | int | str
```

`target`

Kind: `Parameter`

```
Node | int | str
```

`subgraph`

Kind: `Parameter`

```
Optional[Subgraph]
```

Default: `None`

`direction`

Kind: `Parameter`

```
DirectionBasis
```

Default: `DirectionBasis.Underlying`

`topological_order`

Kind: `Method`

```
def topological_order(self, *, subgraph: Optional[Subgraph] = None, direction:
DirectionBasis = DirectionBasis.Underlying) -> list[Node]:
```

Return a deterministic topological order for the selected directed graph.

Explicitly selected zero-crown nodes appear first in ascending graph order.

`subgraph`

Kind: `Parameter`

```
Optional[Subgraph]
```

Default: `None`

`direction`

Kind: `Parameter`

```
DirectionBasis
```

Default: `DirectionBasis.Underlying`

transitive_reduction

Kind: Method

```
def transitive_reduction(self, *, direction: DirectionBasis =
DirectionBasis.Underlying) -> Graph:
```

Return a new graph with directionally redundant DAG edges removed.

direction

Kind: Parameter

```
DirectionBasis
```

Default: `DirectionBasis.Underlying`

depth_first_traverse

Kind: Method

```
def depth_first_traverse(self, root: int | str, *, subgraph: Optional[Subgraph] =
None, include: Optional[int] = None) -> TraversalTree:
```

root

Kind: Parameter

```
int | str
```

subgraph

Kind: Parameter

```
Optional[Subgraph]
```

Default: `None`

include

Kind: Parameter

```
Optional[int]
```

Default: `None`

breadth_first_traverse

Kind: Method

```
def breadth_first_traverse(self, root: int | str, *, subgraph: Optional[Subgraph] =
None, include: Optional[int] = None) -> TraversalTree:
```

root

Kind: Parameter

```
int | str
```

subgraph

Kind: Parameter

```
Optional[Subgraph]
```

Default: `None`

include

Kind: Parameter

```
Optional[int]
```

Default: None

bridges

Kind: Method

```
def bridges(self, subgraph: Optional[Subgraph] = None) -> Subgraph:
```

subgraph

Kind: Parameter

```
Optional[Subgraph]
```

Default: None

cycle_basis

Kind: Method

```
def cycle_basis(self, subgraph: Optional[Subgraph] = None) -> tuple[list[Cycle],
Subgraph]:
```

subgraph

Kind: Parameter

```
Optional[Subgraph]
```

Default: None

all_spanning_forests

Kind: Method

```
def all_spanning_forests(self, subgraph: Optional[Subgraph] = None) ->
list[Subgraph]:
```

subgraph

Kind: Parameter

```
Optional[Subgraph]
```

Default: None

all_bonds

Kind: Method

```
def all_bonds(self, *, subgraph: Optional[Subgraph] = None, min_size: Optional[int] =
None, max_size: Optional[int] = None) -> list[Subgraph]:
```

```
Enumerate inclusion-minimal cutsets with an inclusive boundary-size range.

Bounds default to `[1, unbounded]`; `min_size=0` is rejected because a bond is non-
empty.
```

subgraph

Kind: Parameter

```
Optional[Subgraph]
```

Default: None

min_size

Kind: Parameter

Optional[int]

Default: None

max_size
Kind: Parameter

Optional[int]

Default: None

find_bond
Kind: Method

def find_bond(self, *, subgraph: Optional[Subgraph] = None, min_size: Optional[int] = None, max_size: Optional[int] = None) -> Optional[Subgraph]:

Find one native inclusion-minimal cutset within an inclusive size range.

subgraph
Kind: Parameter

Optional[Subgraph]

Default: None

min_size
Kind: Parameter

Optional[int]

Default: None

max_size
Kind: Parameter

Optional[int]

Default: None

all_cycles
Kind: Method

def all_cycles(self, *, max_results: int, subgraph: Optional[Subgraph] = None) -> list[Cycle]:

Enumerate circuits only when the complete cycle-space candidate count fits the bound.

`max_results` bounds all non-empty combinations of a cycle basis, not only
the combinations that ultimately form circuits.

max_results
Kind: Parameter

int

subgraph
Kind: Parameter

Optional[Subgraph]

Default: None

tadpoles

Kind: Method

```
def tadpoles(self, externals: Sequence[Node | int | str]) -> list[Subgraph]:
```

Enumerate tadpole components after identifying a non-empty terminal node set.

externals

Kind: Parameter

```
Sequence[Node | int | str]
```

all_cuts

Kind: Method

```
def all_cuts(self, source: Sequence[int | str], target: Sequence[int | str]) ->
list[CutPartition]:
```

Enumerate all native separating partitions between two disjoint, non-empty node groups.

source

Kind: Parameter

```
Sequence[int | str]
```

target

Kind: Parameter

```
Sequence[int | str]
```

add_node

Kind: Method

```
def add_node(self, spec: NodeSpec) -> Node:
```

Append one declarative node and return its fresh live view.

spec

Kind: Parameter

```
NodeSpec
```

add_edge

Kind: Method

```
def add_edge(self, spec: EdgeSpec) -> Edge:
```

Append one declarative internal or dangling edge and return its fresh live view.

spec

Kind: Parameter

```
EdgeSpec
```

split_edge

Kind: Method

```
def split_edge(self, at: _HalfEdgeTarget, replacement: EdgeValue, *, name:
Optional[str] = None, orientation: Orientation = Orientation.Default) -> tuple[Edge,
Edge]:
```

Split a paired edge at one half-edge, preserving the opposite edge value.

at

Kind: Parameter

`_HalfEdgeTarget`

replacement

Kind: Parameter

`EdgeValue`

name

Kind: Parameter

`Optional[str]`

Default: `None`

orientation

Kind: Parameter

`Orientation`

Default: `Orientation.Default`

connect

Kind: Method

```
def connect(self, source: _HalfEdgeTarget, sink: _HalfEdgeTarget, replacement:
EdgeValue, *, name: Optional[str] = None, orientation: Orientation =
Orientation.Default) -> Edge:
```

Connect two dangling half-edges; the first becomes the source endpoint.

source

Kind: Parameter

`_HalfEdgeTarget`

sink

Kind: Parameter

`_HalfEdgeTarget`

replacement

Kind: Parameter

`EdgeValue`

name

Kind: Parameter

`Optional[str]`

Default: `None`

orientation

Kind: Parameter

`Orientation`

Default: `Orientation.Default`

concretize

Kind: Method

```
def concretize(self, subgraph: Subgraph) -> Graph:
```

Copy a structural subgraph into an independent graph.

subgraph

Kind: Parameter

Subgraph

extract

Kind: Method

```
def extract(self, subgraph: Subgraph) -> Graph:
```

Remove a structural subgraph and return it as an independent graph.

subgraph

Kind: Parameter

Subgraph

delete

Kind: Method

```
def delete(self, subgraph: Subgraph) -> None:
```

Delete a structural subgraph.

subgraph

Kind: Parameter

Subgraph

contract

Kind: Method

```
def contract(self, subgraph: Subgraph, replacement: NodeValue, *, name: Optional[str]
= None) -> None:
```

Contract a non-empty subgraph into one replacement node.

subgraph

Kind: Parameter

Subgraph

replacement

Kind: Parameter

NodeValue

name

Kind: Parameter

Optional[str]

Default: None

append

Kind: Method

```
def append(self, other: Graph) -> Graph:
```

Append another graph without matching dangling half-edges.

other

Kind: Parameter

Graph

append_mut

Kind: Method

```
def append_mut(self, other: Graph) -> None:
```

Append another graph in place without matching dangling half-edges.

other

Kind: Parameter

Graph

join

Kind: Method

```
def join(self, other: Graph, *, matching: Callable[[HalfEdge, HalfEdge], bool],
merge: Callable[[HalfEdge, HalfEdge], tuple[Flow, Orientation, str | None,
EdgeValue]]) -> Graph:
```

Join dangling half-edges selected by Python callbacks.

other

Kind: Parameter

Graph

matching

Kind: Parameter

Callable[[HalfEdge, HalfEdge], bool]

merge

Kind: Parameter

Callable[[HalfEdge, HalfEdge], tuple[Flow, Orientation, str | None, EdgeValue]]

join_mut

Kind: Method

```
def join_mut(self, other: Graph, *, matching: Callable[[HalfEdge, HalfEdge], bool],
merge: Callable[[HalfEdge, HalfEdge], tuple[Flow, Orientation, str | None,
EdgeValue]]) -> None:
```

Join another graph in place while leaving the other graph unchanged.

other

Kind: Parameter

Graph

matching

Kind: Parameter

Callable[[HalfEdge, HalfEdge], bool]

merge

Kind: Parameter

Callable[[HalfEdge, HalfEdge], tuple[Flow, Orientation, str | None, EdgeValue]]

__repr__

Kind: Method

def __repr__(self) -> str:

Exhaustive reference · Source

GraphStyleOptions

Options applied by `linnest.graph.style` before layout measurement.

Options applied by `linnest.graph.style` before layout measurement.

class GraphStyleOptions:

Requires features: extension-module, abi3

__repr__

Kind: Method

def __repr__(self) -> str:

__new__

Kind: AssociatedFunction

def __new__(cls, *, scope: _Dictionary = ..., unit: _LengthValue = ..., node_label: _AutoOptionalContent = ..., node_label_style: _Style = ..., node_style: _OptionalStyle = ..., edge_label: _OptionalContent = ..., edge_label_style: _Style = ...) -> GraphStyleOptions:

scope

Kind: Parameter

_Dictionary

Default: ...

unit

Kind: Parameter

_LengthValue

Default: ...

node_label

Kind: Parameter

_AutoOptionalContent

Default: ...

node_label_style

Kind: Parameter

`_Style`

Default: `...`

`node_style`
Kind: `Parameter`

`_OptionalStyle`

Default: `...`

`edge_label`
Kind: `Parameter`

`_OptionalContent`

Default: `...`

`edge_label_style`
Kind: `Parameter`

`_Style`

Default: `...`

Exhaustive reference · Source

`HalfEdgeSpec`
`A declarative edge endpoint produced by `source()` or `sink()`.`

`A declarative edge endpoint produced by `source()` or `sink()`.`

`class HalfEdgeSpec:`

Requires features: `extension-module, abi3`

`flow`
Kind: `Getter`

`def flow(self) -> Flow:`

`data`
Kind: `Getter`

`def data(self) -> Any:`

`data`
Kind: `Setter`

`def data(self, value: Any) -> None:`

`value`
Kind: `Parameter`

`Any`

`drawing`
Kind: `Getter`

`def drawing(self) -> HalfEdgeDrawing:`

Exhaustive reference · Source

`INHERIT`

Exported constant of type `Inherit`.

Exported constant of type `Inherit`.

```
INHERIT: Inherit
```

Requires features: `extension-module, abi3`

Exhaustive reference · Source

`Inherit`

Type of the `INHERIT` sentinel, which preserves a lower-precedence setting.

Type of the `INHERIT` sentinel, which preserves a lower-precedence setting.

```
class Inherit:
```

Requires features: `extension-module, abi3`

`__repr__`

Kind: `Method`

```
def __repr__(self) -> str:
```

`__copy__`

Kind: `Method`

```
def __copy__(self) -> Inherit:
```

`__deepcopy__`

Kind: `Method`

```
def __deepcopy__(self, _memo: Any) -> Inherit:
```

`_memo`

Kind: `Parameter`

```
Any
```

Exhaustive reference · Source

`Insets`

Typed CeTZ/Typst inset values.

Typed CeTZ/Typst inset values.

```
class Insets:
```

Requires features: `extension-module, abi3`

`__repr__`

Kind: `Method`

```
def __repr__(self) -> str:
```

`__new__`

Kind: `AssociatedFunction`

```
def __new__(cls, *, all: _LengthValue = ..., x: _LengthValue = ..., y: _LengthValue
= ..., left: _LengthValue = ..., right: _LengthValue = ..., top: _LengthValue = ...,
bottom: _LengthValue = ...) -> Insets:
```

all
Kind: Parameter

_LengthValue

Default: ...

x
Kind: Parameter

_LengthValue

Default: ...

y
Kind: Parameter

_LengthValue

Default: ...

left
Kind: Parameter

_LengthValue

Default: ...

right
Kind: Parameter

_LengthValue

Default: ...

top
Kind: Parameter

_LengthValue

Default: ...

bottom
Kind: Parameter

_LengthValue

Default: ...

Exhaustive reference · Source

LayoutFrame
A position snapshot from one continuous force-layout run.

A position snapshot from one continuous force-layout run.

class LayoutFrame:

Requires features: extension-module, abi3

nodes
Kind: Getter

```
def nodes(self) -> list[tuple[float, float]]:
```

edges

Kind: Getter

```
def edges(self) -> list[tuple[float, float]]:
```

iteration

Kind: Getter

```
def iteration(self) -> int:
```

done

Kind: Getter

```
def done(self) -> bool:
```

max_movement

Kind: Getter

```
def max_movement(self) -> float:
```

Exhaustive reference · Source

LayoutOptions

One or more ordered Linnest layout passes.

One or more ordered Linnest layout passes.

```
class LayoutOptions:
```

Requires features: `extension-module, abi3`

pass_count

Kind: Getter

```
def pass_count(self) -> int:
```

sequence

Kind: AssociatedFunction

```
def sequence(passes: Sequence[LayoutOptions]) -> LayoutOptions:
```

passes

Kind: Parameter

```
Sequence[LayoutOptions]
```

__repr__

Kind: Method

```
def __repr__(self) -> str:
```

__new__

Kind: AssociatedFunction

```
def __new__(cls, *, subgraph: _OptionalHedgeSelection = ..., viewport_width: _Number
= ..., viewport_height: _Number = ..., tree_dx: _Number = ..., tree_dy: _Number
= ..., steps: _Integer = ..., seed: _Integer = ..., step: _Number = ..., step_shrink:
_Number = ..., cool: _Number = ..., accept_floor: _Number = ..., early_tolerance:
_Number = ..., temperature: _Number = ..., delta: _Number = ..., beta: _Number = ...,
```

```
spring_strength: _Number = ..., centering_strength: _Number = ..., epochs: _Integer
= ..., crossing_penalty: _Number = ..., dangling_repulsion: _Number = ...,
dangling_centroid_repulsion: _Number = ..., edge_edge_repulsion: _Number = ...,
directional_force: _Number = ..., label_length_scale: _Number = ..., label_spring:
_Number = ..., label_charge: _Number = ..., label_steps: _Integer = ...,
label_layout: _LabelLayoutValue = ..., label_step: _Number = ...,
label_early_tolerance: _Number = ..., label_max_delta_scale: _Number = ...,
edge_vertex_repulsion: _Number = ..., epsilon: _Number = ..., incremental_energy:
_Boolean = ..., algorithm: _LayoutAlgorithmValue = ..., nodes: _LayoutNodesValue
= ..., direction: _LayoutDirectionValue = ..., rank_align: _RankAlignmentValue = ...,
roots: _NodeIndices = ..., rank_same: _NodeGroups = ..., route_edge_weight: _Number
= ..., route_exit_weight: _Number = ..., route_label_width_scale: _Number = ...,
route_label_width_cap: _Number = ..., z_spring: _Number = ..., z_spring_growth:
_Number = ..., length_scale: _Number = ...) -> LayoutOptions:
```

subgraph

Kind: Parameter

_OptionalHedgeSelection

Default: ...

viewport_width

Kind: Parameter

_Number

Default: ...

viewport_height

Kind: Parameter

_Number

Default: ...

tree_dx

Kind: Parameter

_Number

Default: ...

tree_dy

Kind: Parameter

_Number

Default: ...

steps

Kind: Parameter

_Integer

Default: ...

seed

Kind: Parameter

_Integer

Default: ...

step
Kind: Parameter

_Number

Default: ...

step_shrink
Kind: Parameter

_Number

Default: ...

cool
Kind: Parameter

_Number

Default: ...

accept_floor
Kind: Parameter

_Number

Default: ...

early_tolerance
Kind: Parameter

_Number

Default: ...

temperature
Kind: Parameter

_Number

Default: ...

delta
Kind: Parameter

_Number

Default: ...

beta
Kind: Parameter

_Number

Default: ...

spring_strength
Kind: Parameter

_Number

Default: ...

centering_strength
Kind: Parameter

_Number

Default: ...

epochs
Kind: Parameter

_Integer

Default: ...

crossing_penalty
Kind: Parameter

_Number

Default: ...

dangling_repulsion
Kind: Parameter

_Number

Default: ...

dangling_centroid_repulsion
Kind: Parameter

_Number

Default: ...

edge_edge_repulsion
Kind: Parameter

_Number

Default: ...

directional_force
Kind: Parameter

_Number

Default: ...

label_length_scale
Kind: Parameter

_Number

Default: ...

label_spring
Kind: Parameter

_Number

Default: ...

label_charge
Kind: Parameter

_Number

Default: ...

label_steps
Kind: Parameter

_Integer

Default: ...

label_layout
Kind: Parameter

_LabelLayoutValue

Default: ...

label_step
Kind: Parameter

_Number

Default: ...

label_early_tolerance
Kind: Parameter

_Number

Default: ...

label_max_delta_scale
Kind: Parameter

_Number

Default: ...

edge_vertex_repulsion
Kind: Parameter

_Number

Default: ...

epsilon
Kind: Parameter

_Number

Default: ...

incremental_energy
Kind: Parameter

_Boolean

Default: ...

algorithm
Kind: Parameter

_LayoutAlgorithmValue

Default: ...

nodes
Kind: Parameter

_LayoutNodesValue

Default: ...

direction
Kind: Parameter

_LayoutDirectionValue

Default: ...

rank_align
Kind: Parameter

_RankAlignmentValue

Default: ...

roots
Kind: Parameter

_NodeIndices

Default: ...

rank_same
Kind: Parameter

_NodeGroups

Default: ...

route_edge_weight
Kind: Parameter

_Number

Default: ...

route_exit_weight
Kind: Parameter

_Number

Default: ...

route_label_width_scale
Kind: Parameter

_Number

Default: ...

route_label_width_cap

Kind: Parameter

_Number

Default: ...

z_spring

Kind: Parameter

_Number

Default: ...

z_spring_growth

Kind: Parameter

_Number

Default: ...

length_scale

Kind: Parameter

_Number

Default: ...

then

Kind: Method

```
def then(self, *, subgraph: _OptionalHedgeSelection = ..., viewport_width: _Number
= ..., viewport_height: _Number = ..., tree_dx: _Number = ..., tree_dy: _Number
= ..., steps: _Integer = ..., seed: _Integer = ..., step: _Number = ..., step_shrink:
_Number = ..., cool: _Number = ..., accept_floor: _Number = ..., early_tolerance:
_Number = ..., temperature: _Number = ..., delta: _Number = ..., beta: _Number = ...,
spring_strength: _Number = ..., centering_strength: _Number = ..., epochs: _Integer
= ..., crossing_penalty: _Number = ..., dangling_repulsion: _Number = ...,
dangling_centroid_repulsion: _Number = ..., edge_edge_repulsion: _Number = ...,
directional_force: _Number = ..., label_length_scale: _Number = ..., label_spring:
_Number = ..., label_charge: _Number = ..., label_steps: _Integer = ...,
label_layout: _LabelLayoutValue = ..., label_step: _Number = ...,
label_early_tolerance: _Number = ..., label_max_delta_scale: _Number = ...,
edge_vertex_repulsion: _Number = ..., epsilon: _Number = ..., incremental_energy:
_Boolean = ..., algorithm: _LayoutAlgorithmValue = ..., nodes: _LayoutNodesValue
= ..., direction: _LayoutDirectionValue = ..., rank_align: _RankAlignmentValue = ...,
roots: _NodeIndices = ..., rank_same: _NodeGroups = ..., route_edge_weight: _Number
= ..., route_exit_weight: _Number = ..., route_label_width_scale: _Number = ...,
route_label_width_cap: _Number = ..., z_spring: _Number = ..., z_spring_growth:
_Number = ..., length_scale: _Number = ...) -> LayoutOptions:
```

subgraph

Kind: Parameter

_OptionalHedgeSelection

Default: ...

viewport_width
Kind: Parameter

_Number

Default: ...

viewport_height
Kind: Parameter

_Number

Default: ...

tree_dx
Kind: Parameter

_Number

Default: ...

tree_dy
Kind: Parameter

_Number

Default: ...

steps
Kind: Parameter

_Integer

Default: ...

seed
Kind: Parameter

_Integer

Default: ...

step
Kind: Parameter

_Number

Default: ...

step_shrink
Kind: Parameter

_Number

Default: ...

cool
Kind: Parameter

_Number

Default: ...

accept_floor
Kind: Parameter

_Number

Default: ...

early_tolerance
Kind: Parameter

_Number

Default: ...

temperature
Kind: Parameter

_Number

Default: ...

delta
Kind: Parameter

_Number

Default: ...

beta
Kind: Parameter

_Number

Default: ...

spring_strength
Kind: Parameter

_Number

Default: ...

centering_strength
Kind: Parameter

_Number

Default: ...

epochs
Kind: Parameter

_Integer

Default: ...

crossing_penalty
Kind: Parameter

_Number

Default: ...

dangling_repulsion
Kind: Parameter

_Number

Default: ...

dangling_centroid_repulsion
Kind: Parameter

_Number

Default: ...

edge_edge_repulsion
Kind: Parameter

_Number

Default: ...

directional_force
Kind: Parameter

_Number

Default: ...

label_length_scale
Kind: Parameter

_Number

Default: ...

label_spring
Kind: Parameter

_Number

Default: ...

label_charge
Kind: Parameter

_Number

Default: ...

label_steps
Kind: Parameter

_Integer

Default: ...

label_layout
Kind: Parameter

_LabelLayoutValue

Default: ...

label_step
Kind: Parameter

_Number

Default: ...

label_early_tolerance
Kind: Parameter

_Number

Default: ...

label_max_delta_scale
Kind: Parameter

_Number

Default: ...

edge_vertex_repulsion
Kind: Parameter

_Number

Default: ...

epsilon
Kind: Parameter

_Number

Default: ...

incremental_energy
Kind: Parameter

_Boolean

Default: ...

algorithm
Kind: Parameter

_LayoutAlgorithmValue

Default: ...

nodes
Kind: Parameter

_LayoutNodesValue

Default: ...

direction
Kind: Parameter

_LayoutDirectionValue

Default: ...

rank_align
Kind: Parameter

_RankAlignmentValue

Default: ...

roots
Kind: Parameter

_NodeIndices

Default: ...

rank_same
Kind: Parameter

_NodeGroups

Default: ...

route_edge_weight
Kind: Parameter

_Number

Default: ...

route_exit_weight
Kind: Parameter

_Number

Default: ...

route_label_width_scale
Kind: Parameter

_Number

Default: ...

route_label_width_cap
Kind: Parameter

_Number

Default: ...

z_spring
Kind: Parameter

_Number

Default: ...

z_spring_growth
Kind: Parameter

_Number

Default: ...

length_scale

Kind: Parameter

_Number

Default: ...

Exhaustive reference · Source

LayoutStream

A synchronous iterator that advances the native force solver in batches.

A synchronous iterator that advances the native force solver in batches.

Topology is fixed for the lifetime of the stream. Frames contain only
coordinates and progress, so notebook viewers can retain their SVG elements.

class LayoutStream:

Requires features: extension-module, abi3

node_names

Kind: Getter

def node_names(self) -> list[str]:

Node names in the same stable index order as every frame's coordinates.

endpoints

Kind: Getter

def endpoints(self) -> list[tuple[Optional[int], Optional[int]]]:

Source and sink node indices; a missing endpoint denotes a dangling edge.

from_dot

Kind: AssociatedFunction

def from_dot(dot: str, *, every: int = 4, steps: int = 200, epochs: int = 8, seed: int = 1, step: float = 0.02, cool: float = 0.85, spring_strength: float = 1.0, repulsion: float = 1.5, length_scale: float = 1.0, depth_scale: float = 1.0, flattening_end: float = 0.5, delta: float = 0.1, early_tolerance: float = 1e-06) -> LayoutStream:

Start a force-layout preview from a single DOT graph.

This preview uses native graph geometry, without Typst label measurement
or final edge-label relaxation. A new stream restarts from the same seed.

dot

Kind: Parameter

str

every

Kind: Parameter

int

Default: 4

steps

Kind: Parameter

int

Default: 200

epochs
Kind: Parameter

int

Default: 8

seed
Kind: Parameter

int

Default: 1

step
Kind: Parameter

float

Default: 0.02

cool
Kind: Parameter

float

Default: 0.85

spring_strength
Kind: Parameter

float

Default: 1.0

repulsion
Kind: Parameter

float

Default: 1.5

length_scale
Kind: Parameter

float

Default: 1.0

depth_scale
Kind: Parameter

float

Default: 1.0

flattening_end
Kind: Parameter

`float`

Default: `0.5`

`delta`
Kind: `Parameter`

`float`

Default: `0.1`

`early_tolerance`
Kind: `Parameter`

`float`

Default: `1e-06`

`__iter__`
Kind: `Method`

```
def __iter__(self) -> LayoutStream:
```

`__next__`
Kind: `Method`

```
def __next__(self) -> LayoutFrame:
```

Exhaustive reference · Source

`Length`
`A Typst length such as `2pt` or `1.2em`.`

`A Typst length such as `2pt` or `1.2em`.`

```
class Length:
```

Requires features: `extension-module, abi3`

`value`
Kind: `Getter`

```
def value(self) -> float:
```

`unit`
Kind: `Getter`

```
def unit(self) -> str:
```

`__new__`
Kind: `AssociatedFunction`

```
def __new__(cls, value: float, unit: Literal['pt', 'mm', 'cm', 'in', 'em']) ->
Length:
```

`value`
Kind: `Parameter`

`float`

`unit`
Kind: `Parameter`

```
Literal['pt', 'mm', 'cm', 'in', 'em']
```

pt

Kind: `AssociatedFunction`

```
def pt(value: float) -> Length:
```

value

Kind: `Parameter`

```
float
```

mm

Kind: `AssociatedFunction`

```
def mm(value: float) -> Length:
```

value

Kind: `Parameter`

```
float
```

cm

Kind: `AssociatedFunction`

```
def cm(value: float) -> Length:
```

value

Kind: `Parameter`

```
float
```

inches

Kind: `AssociatedFunction`

```
def inches(value: float) -> Length:
```

value

Kind: `Parameter`

```
float
```

em

Kind: `AssociatedFunction`

```
def em(value: float) -> Length:
```

value

Kind: `Parameter`

```
float
```

__repr__

Kind: `Method`

```
def __repr__(self) -> str:
```

Exhaustive reference · Source

Mark

Typed CeTZ mark configuration.

Typed CeTZ mark configuration.

```
class Mark:
```

Requires features: `extension-module, abi3`

barbed

Kind: `AssociatedFunction`

```
def barbed() -> Mark:
```

straight

Kind: `AssociatedFunction`

```
def straight() -> Mark:
```

__repr__

Kind: `Method`

```
def __repr__(self) -> str:
```

__new__

Kind: `AssociatedFunction`

```
def __new__(cls, *, start: _NativeValue = ..., end: _NativeValue = ..., fill:
_OptionalPaint = ..., stroke: _OptionalStrokeValue = ..., scale: _Number = ...,
anchor: _MarkAnchor = ..., shorten_to: _MarkShorten = ...) -> Mark:
```

start

Kind: `Parameter`

```
_NativeValue
```

Default: `...`

end

Kind: `Parameter`

```
_NativeValue
```

Default: `...`

fill

Kind: `Parameter`

```
_OptionalPaint
```

Default: `...`

stroke

Kind: `Parameter`

```
_OptionalStrokeValue
```

Default: `...`

scale

Kind: `Parameter`

`_Number`

Default: `...`

`anchor`
Kind: `Parameter`

`_MarkAnchor`

Default: `...`

`shorten_to`
Kind: `Parameter`

`_MarkShorten`

Default: `...`

Exhaustive reference · Source

`MathSymbol`
`Safe mathematical identifier content, optionally with scripts.`

`Safe mathematical identifier content, optionally with scripts.`

`class MathSymbol:`

Requires features: `extension-module, abi3`

`__new__`
Kind: `AssociatedFunction`

```
def __new__(cls, symbol: str, *, subscript: str | int | None = None, superscript: str | int | None = None) -> MathSymbol:
```

`symbol`
Kind: `Parameter`

`str`

`subscript`
Kind: `Parameter`

`str | int | None`

Default: `None`

`superscript`
Kind: `Parameter`

`str | int | None`

Default: `None`

`__repr__`
Kind: `Method`

`def __repr__(self) -> str:`

Exhaustive reference · Source

`NodeSpec`
`A reusable declarative node description accepted by `build()` and `Graph.add_node()`.`

A reusable declarative node description accepted by `build()` and `Graph.add_node()`.

class NodeSpec:

Requires features: extension-module, abi3

name
Kind: Getter

def name(self) -> Optional[str]:

data
Kind: Getter

def data(self) -> Any:

data
Kind: Setter

def data(self, value: Any) -> None:

value
Kind: Parameter

Any

drawing
Kind: Getter

def drawing(self) -> NodeDrawing:

__repr__
Kind: Method

def __repr__(self) -> str:

Exhaustive reference · Source

NodeStore
Storage strategy used for graph nodes.

Storage strategy used for graph nodes.

class NodeStore(enum.Enum):

Requires features: extension-module, abi3

Vec
Kind: Variant

Dense node records with one incidence bitset per node.

Forest
Kind: Variant

Identification-preserving forest node records.

Exhaustive reference · Source

Orientation
How an edge's declared endpoints determine its logical direction.

How an edge's declared endpoints determine its logical direction.

class Orientation(enum.Enum):

Requires features: extension-module, abi3

Default
Kind: Variant

Reversed
Kind: Variant

Undirected
Kind: Variant

Exhaustive reference · Source

OrientedCut
An oriented cut in a particular graph revision.

An oriented cut in a particular graph revision.

class OrientedCut:

Requires features: extension-module, abi3

left
Kind: Getter

def left(self) -> Subgraph:

right
Kind: Getter

def right(self) -> Subgraph:

edges
Kind: Getter

def edges(self) -> list[Edge]:

Return the cut edges in stable graph order.

side
Kind: Method

def side(self, left: bool) -> Subgraph:

left
Kind: Parameter

bool

orientation
Kind: Method

def orientation(self, edge: Edge | int | str) -> Orientation:

Return one edge's orientation relative to this cut.

edge
Kind: Parameter

Edge | int | str

winding_number

Kind: Method

```
def winding_number(self, cycle: Cycle) -> int:
```

Return the signed intersection count with a cycle from the same graph revision.

cycle

Kind: Parameter

Cycle

__repr__

Kind: Method

```
def __repr__(self) -> str:
```

Exhaustive reference · Source

PreparedRender

One Typst render whose virtual project and generated entrypoint share a lifetime.

One Typst render whose virtual project and generated entrypoint share a lifetime.

```
class PreparedRender:
```

Requires features: extension-module, abi3

typst_source

Kind: Getter

```
def typst_source(self) -> str:
```

Return the exact generated Typst entrypoint for this preparation.

render

Kind: Method

```
def render(self, output: str | os.PathLike[str]) -> pathlib.Path:
```

Compile this preparation to a PDF, SVG, or PNG selected by the suffix.

output

Kind: Parameter

```
str | os.PathLike[str]
```

to_svg

Kind: Method

```
def to_svg(self) -> str:
```

Compile this preparation and return its one-page SVG document.

Exhaustive reference · Source

Anchor

A finite CeTZ box or content anchor.

A finite CeTZ box or content anchor.

```
class Anchor(enum.Enum):
```

Requires features: extension-module, abi3

Center
Kind: Variant

North
Kind: Variant

NorthEast
Kind: Variant

East
Kind: Variant

SouthEast
Kind: Variant

South
Kind: Variant

SouthWest
Kind: Variant

West
Kind: Variant

NorthWest
Kind: Variant

Exhaustive reference · Source

Compass
`A finite DOT compass point.`

```
A finite DOT compass point.
```

```
class Compass(enum.Enum):
```

Requires features: `extension-module, abi3`

N
Kind: Variant

NE
Kind: Variant

E
Kind: Variant

SE
Kind: Variant

S
Kind: Variant

SW
Kind: Variant

`W`
Kind: `Variant`

`NW`
Kind: `Variant`

`Center`
Kind: `Variant`

`Any`
Kind: `Variant`

Exhaustive reference · Source

`DanglingTangent`
`Tangent constraint at the free endpoint of a dangling edge.`

`Tangent constraint at the free endpoint of a dangling edge.`

`class DanglingTangent(enum.Enum):`

Requires features: `extension-module, abi3`

`Horizontal`
Kind: `Variant`

`Vertical`
Kind: `Variant`

Exhaustive reference · Source

`DashPattern`
`Built-in Typst stroke dash pattern.`

`Built-in Typst stroke dash pattern.`

`class DashPattern(enum.Enum):`

Requires features: `extension-module, abi3`

`Solid`
Kind: `Variant`

`Dotted`
Kind: `Variant`

`DenselyDotted`
Kind: `Variant`

`LooselyDotted`
Kind: `Variant`

`Dashed`
Kind: `Variant`

`DenselyDashed`
Kind: `Variant`

LooselyDashed
Kind: Variant

DashDotted
Kind: Variant

DenselyDashDotted
Kind: Variant

LooselyDashDotted
Kind: Variant

Exhaustive reference · Source

Edge
A live edge view whose data and drawing metadata update its graph.

```
A live edge view whose data and drawing metadata update its graph.
class Edge:
```

Requires features: `extension-module, abi3`

index
Kind: Getter

```
def index(self) -> int:
```

name
Kind: Getter

```
def name(self) -> Optional[str]:
```

data
Kind: Getter

```
def data(self) -> Any:
```

data
Kind: Setter

```
def data(self, value: Any) -> None:
```

value
Kind: Parameter

```
Any
```

drawing
Kind: Getter

```
def drawing(self) -> EdgeDrawing:
```

orientation
Kind: Getter

```
def orientation(self) -> Orientation:
```

source
Kind: Getter

```
def source(self) -> Optional[HalfEdge]:
```

sink
Kind: `Getter`

```
def sink(self) -> Optional[HalfEdge]:
```

__repr__
Kind: `Method`

```
def __repr__(self) -> str:
```

Exhaustive reference · Source

EdgeDrawing
`Mutable typed drawing metadata for an edge.`

`Mutable typed drawing metadata for an edge.`

```
class EdgeDrawing:
```

Requires features: `extension-module, abi3`

label
Kind: `Getter`

```
def label(self) -> _OptionalStaticContent:
```

label
Kind: `Setter`

```
def label(self, value: _OptionalStaticContent) -> None:
```

value
Kind: `Parameter`

```
_OptionalStaticContent
```

placement
Kind: `Getter`

```
def placement(self) -> _PlacementValue:
```

placement
Kind: `Setter`

```
def placement(self, value: _PlacementValue) -> None:
```

value
Kind: `Parameter`

```
_PlacementValue
```

label_position
Kind: `Getter`

```
def label_position(self) -> _DrawingPoint:
```

label_position
Kind: `Setter`

```
def label_position(self, value: _DrawingPoint) -> None:
```

value
Kind: Parameter

_DrawingPoint

label_offset
Kind: Getter

def label_offset(self) -> _OptionalNumber:

label_offset
Kind: Setter

def label_offset(self, value: _OptionalNumber) -> None:

value
Kind: Parameter

_OptionalNumber

label_angle
Kind: Getter

def label_angle(self) -> _DrawingAngle:

label_angle
Kind: Setter

def label_angle(self, value: _DrawingAngle) -> None:

value
Kind: Parameter

_DrawingAngle

bend
Kind: Getter

def bend(self) -> _DrawingAngle:

bend
Kind: Setter

def bend(self, value: _DrawingAngle) -> None:

value
Kind: Parameter

_DrawingAngle

routing
Kind: Getter

def routing(self) -> _RoutingValue:

routing
Kind: Setter

def routing(self, value: _RoutingValue) -> None:

value
Kind: Parameter

_RoutingValue

minimum_length
Kind: Getter

def minimum_length(self) -> _OptionalInteger:

minimum_length
Kind: Setter

def minimum_length(self, value: _OptionalInteger) -> None:

value
Kind: Parameter

_OptionalInteger

same_rank
Kind: Getter

def same_rank(self) -> _OptionalBoolean:

same_rank
Kind: Setter

def same_rank(self, value: _OptionalBoolean) -> None:

value
Kind: Parameter

_OptionalBoolean

style
Kind: Getter

def style(self) -> _OptionalStyleLayers:

style
Kind: Setter

def style(self, value: _OptionalStyleLayers) -> None:

value
Kind: Parameter

_OptionalStyleLayers

label_style
Kind: Getter

def label_style(self) -> _OptionalStyle:

label_style
Kind: Setter

def label_style(self, value: _OptionalStyle) -> None:

value

Kind: Parameter

`_OptionalStyle`

decoration

Kind: Getter

```
def decoration(self) -> _DrawingDecoration:
```

decoration

Kind: Setter

```
def decoration(self, value: _DrawingDecoration) -> None:
```

value

Kind: Parameter

`_DrawingDecoration`

extensions

Kind: Getter

```
def extensions(self) -> dict[str, _NativeValue]:
```

extensions

Kind: Setter

```
def extensions(self, value: dict[str, _NativeValue]) -> None:
```

value

Kind: Parameter

`dict[str, _NativeValue]`

__repr__

Kind: Method

```
def __repr__(self) -> str:
```

__new__

Kind: AssociatedFunction

```
def __new__(cls, *, label: _OptionalStaticContent = ..., placement: _PlacementValue
= ..., label_position: _DrawingPoint = ..., label_offset: _OptionalNumber = ...,
label_angle: _DrawingAngle = ..., bend: _DrawingAngle = ..., routing: _RoutingValue
= ..., minimum_length: _OptionalInteger = ..., same_rank: _OptionalBoolean = ...,
style: _OptionalStyleLayers = ..., label_style: _OptionalStyle = ..., decoration:
_DrawingDecoration = ..., extensions: _NativeDict = ...) -> EdgeDrawing:
```

label

Kind: Parameter

`_OptionalStaticContent`

Default: `...`

placement

Kind: Parameter

`_PlacementValue`

Default: `...`

`label_position`
Kind: `Parameter`

`_DrawingPoint`

Default: `...`

`label_offset`
Kind: `Parameter`

`_OptionalNumber`

Default: `...`

`label_angle`
Kind: `Parameter`

`_DrawingAngle`

Default: `...`

`bend`
Kind: `Parameter`

`_DrawingAngle`

Default: `...`

`routing`
Kind: `Parameter`

`_RoutingValue`

Default: `...`

`minimum_length`
Kind: `Parameter`

`_OptionalInteger`

Default: `...`

`same_rank`
Kind: `Parameter`

`_OptionalBoolean`

Default: `...`

`style`
Kind: `Parameter`

`_OptionalStyleLayers`

Default: `...`

`label_style`
Kind: `Parameter`

`_OptionalStyle`

Default: `...`

`decoration`
Kind: `Parameter`

`_DrawingDecoration`

Default: `...`

`extensions`
Kind: `Parameter`

`_NativeDict`

Default: `...`

Exhaustive reference · Source

`EdgeLengthResolution`
`How fixed and relative edge-length limits are combined.`

`How fixed and relative edge-length limits are combined.`

`class EdgeLengthResolution(enum.Enum):`

Requires features: `extension-module, abi3`

`Min`
Kind: `Variant`

`Shorter`
Kind: `Variant`

`Max`
Kind: `Variant`

`Longer`
Kind: `Variant`

`Length`
Kind: `Variant`

`Fixed`
Kind: `Variant`

`Ratio`
Kind: `Variant`

`Relative`
Kind: `Variant`

`Disabled`
Kind: `Variant`

`Full`
Kind: `Variant`

Exhaustive reference · Source

EdgeValue

Detached edge data and drawing metadata passed across a `DotCodec` boundary.

Detached edge data and drawing metadata passed across a `DotCodec` boundary.

```
class EdgeValue:
```

Requires features: `extension-module, abi3`

data

Kind: `Getter`

```
def data(self) -> Any:
```

data

Kind: `Setter`

```
def data(self, value: Any) -> None:
```

value

Kind: `Parameter`

```
Any
```

drawing

Kind: `Getter`

```
def drawing(self) -> EdgeDrawing:
```

__new__

Kind: `AssociatedFunction`

```
def __new__(cls, *, data: Any = None, drawing: Optional[EdgeDrawing] = None) ->
EdgeValue:
```

data

Kind: `Parameter`

```
Any
```

Default: `None`

drawing

Kind: `Parameter`

```
Optional[EdgeDrawing]
```

Default: `None`

Exhaustive reference · Source

HalfEdge

A live half-edge view whose data and drawing metadata update its graph.

A live half-edge view whose data and drawing metadata update its graph.

```
class HalfEdge:
```

Requires features: `extension-module, abi3`

index

Kind: `Getter`

```
def index(self) -> int:
```

flow

Kind: Getter

```
def flow(self) -> Flow:
```

data

Kind: Getter

```
def data(self) -> Any:
```

data

Kind: Setter

```
def data(self, value: Any) -> None:
```

value

Kind: Parameter

```
Any
```

drawing

Kind: Getter

```
def drawing(self) -> HalfEdgeDrawing:
```

node

Kind: Getter

```
def node(self) -> Node:
```

edge

Kind: Getter

```
def edge(self) -> Edge:
```

pair

Kind: Getter

```
def pair(self) -> Optional[HalfEdge]:
```

__repr__

Kind: Method

```
def __repr__(self) -> str:
```

Exhaustive reference · Source

HalfEdgeDrawing

Mutable typed drawing metadata for one endpoint of an edge.

Mutable typed drawing metadata for one endpoint of an edge.

```
class HalfEdgeDrawing:
```

Requires features: `extension-module, abi3`

label

Kind: Getter

```
def label(self) -> _OptionalStaticContent:
```

label
Kind: Setter

```
def label(self, value: _OptionalStaticContent) -> None:
```

value
Kind: Parameter

```
_OptionalStaticContent
```

statement
Kind: Getter

```
def statement(self) -> _DrawingString:
```

statement
Kind: Setter

```
def statement(self, value: _DrawingString) -> None:
```

value
Kind: Parameter

```
_DrawingString
```

port_label
Kind: Getter

```
def port_label(self) -> _DrawingString:
```

port_label
Kind: Setter

```
def port_label(self, value: _DrawingString) -> None:
```

value
Kind: Parameter

```
_DrawingString
```

compass
Kind: Getter

```
def compass(self) -> _CompassValue:
```

compass
Kind: Setter

```
def compass(self, value: _CompassValue) -> None:
```

value
Kind: Parameter

```
_CompassValue
```

anchor
Kind: Getter

```
def anchor(self) -> _AnchorValue:
```

anchor
Kind: Setter

```
def anchor(self, value: _AnchorValue) -> None:
```

value
Kind: Parameter

```
_AnchorValue
```

routing
Kind: Getter

```
def routing(self) -> _RoutingValue:
```

routing
Kind: Setter

```
def routing(self, value: _RoutingValue) -> None:
```

value
Kind: Parameter

```
_RoutingValue
```

style
Kind: Getter

```
def style(self) -> _OptionalStyleLayers:
```

style
Kind: Setter

```
def style(self, value: _OptionalStyleLayers) -> None:
```

value
Kind: Parameter

```
_OptionalStyleLayers
```

extensions
Kind: Getter

```
def extensions(self) -> dict[str, _NativeValue]:
```

extensions
Kind: Setter

```
def extensions(self, value: dict[str, _NativeValue]) -> None:
```

value
Kind: Parameter

```
dict[str, _NativeValue]
```

__repr__
Kind: Method

```
def __repr__(self) -> str:
```

__new__

Kind: AssociatedFunction

```
def __new__(cls, *, label: _OptionalStaticContent = ..., statement: _DrawingString
= ..., port_label: _DrawingString = ..., compass: _CompassValue = ..., anchor:
_AnchorValue = ..., routing: _RoutingValue = ..., style: _OptionalStyleLayers = ...,
extensions: _NativeDict = ...) -> HalfEdgeDrawing:
```

label

Kind: Parameter

_OptionalStaticContent

Default: ...

statement

Kind: Parameter

_DrawingString

Default: ...

port_label

Kind: Parameter

_DrawingString

Default: ...

compass

Kind: Parameter

_CompassValue

Default: ...

anchor

Kind: Parameter

_AnchorValue

Default: ...

routing

Kind: Parameter

_RoutingValue

Default: ...

style

Kind: Parameter

_OptionalStyleLayers

Default: ...

extensions

Kind: Parameter

_NativeDict

Default: `...`

Exhaustive reference · Source

`HalfEdgeValue`

`Detached half-edge data and drawing metadata passed across a `DotCodec` boundary.`

`Detached half-edge data and drawing metadata passed across a `DotCodec` boundary.`

`class HalfEdgeValue:`

Requires features: `extension-module, abi3`

`data`

Kind: `Getter`

`def data(self) -> Any:`

`data`

Kind: `Setter`

`def data(self, value: Any) -> None:`

`value`

Kind: `Parameter`

`Any`

`drawing`

Kind: `Getter`

`def drawing(self) -> HalfEdgeDrawing:`

`__new__`

Kind: `AssociatedFunction`

`def __new__(cls, *, data: Any = None, drawing: Optional[HalfEdgeDrawing] = None) -> HalfEdgeValue:`

`data`

Kind: `Parameter`

`Any`

Default: `None`

`drawing`

Kind: `Parameter`

`Optional[HalfEdgeDrawing]`

Default: `None`

Exhaustive reference · Source

`LabelLayout`

`Strategy used to position edge labels.`

`Strategy used to position edge labels.`

`class LabelLayout(enum.Enum):`

Requires features: `extension-module, abi3`

`Normal`
Kind: `Variant`

`DanglingTangent`
Kind: `Variant`

`FixedLength`
Kind: `Variant`

Exhaustive reference · Source

`LayoutAlgorithm`
`Algorithm used by a Linnest layout pass.`

`Algorithm used by a Linnest layout pass.`

`class LayoutAlgorithm(enum.Enum):`

Requires features: `extension-module, abi3`

`Force`
Kind: `Variant`

`Anneal`
Kind: `Variant`

`Tree`
Kind: `Variant`

`Dot`
Kind: `Variant`

`StableLayered`
Kind: `Variant`

Exhaustive reference · Source

`LayoutDirection`
`Rank direction used by tree and layered layouts.`

`Rank direction used by tree and layered layouts.`

`class LayoutDirection(enum.Enum):`

Requires features: `extension-module, abi3`

`Down`
Kind: `Variant`

`Right`
Kind: `Variant`

Exhaustive reference · Source

`LayoutNodes`
`Whether a pass lays out or preserves its selected nodes.`

`Whether a pass lays out or preserves its selected nodes.`

`class LayoutNodes(enum.Enum):`

Requires features: `extension-module, abi3`

`Layout`
Kind: `Variant`

`Fixed`
Kind: `Variant`

Exhaustive reference · Source

`MarkDirection`
`Direction of a mark along an edge path.`

`Direction of a mark along an edge path.`

`class MarkDirection(enum.Enum):`

Requires features: `extension-module, abi3`

`Forward`
Kind: `Variant`

`Backward`
Kind: `Variant`

Exhaustive reference · Source

`MarkOrientation`
`Frame used to orient an edge mark.`

`Frame used to orient an edge mark.`

`class MarkOrientation(enum.Enum):`

Requires features: `extension-module, abi3`

`Path`
Kind: `Variant`

`Edge`
Kind: `Variant`

Exhaustive reference · Source

`MarkPosition`
`Location of a mark on a rendered edge path.`

`Location of a mark on a rendered edge path.`

`class MarkPosition(enum.Enum):`

Requires features: `extension-module, abi3`

`End`
Kind: `Variant`

`Center`
Kind: `Variant`

`CenterIfDangling`
Kind: `Variant`

Exhaustive reference · Source

`MarkSymbol`

`Built-in arrowhead symbol used by CeTZ edge marks.`

`Built-in arrowhead symbol used by CeTZ edge marks.`

```
class MarkSymbol(enum.Enum):
```

Requires features: `extension-module, abi3`

`Barbed`

Kind: `Variant`

`Straight`

Kind: `Variant`

Exhaustive reference · Source

`Node`

`A live node view whose data and drawing metadata update its graph.`

`A live node view whose data and drawing metadata update its graph.`

```
class Node:
```

Requires features: `extension-module, abi3`

`index`

Kind: `Getter`

```
def index(self) -> int:
```

`name`

Kind: `Getter`

```
def name(self) -> Optional[str]:
```

`data`

Kind: `Getter`

```
def data(self) -> Any:
```

`data`

Kind: `Setter`

```
def data(self, value: Any) -> None:
```

`value`

Kind: `Parameter`

`Any`

`drawing`

Kind: `Getter`

```
def drawing(self) -> NodeDrawing:
```

`incidence`

Kind: `Getter`

```
def incidence(self) -> list[HalfEdge]:
```

__repr__
Kind: `Method`

```
def __repr__(self) -> str:
```

Exhaustive reference · Source

NodeDrawing
`Mutable typed drawing metadata for a node.`

`Mutable typed drawing metadata for a node.`

```
class NodeDrawing:
```

Requires features: `extension-module, abi3`

label
Kind: `Getter`

```
def label(self) -> _OptionalStaticContent:
```

label
Kind: `Setter`

```
def label(self, value: _OptionalStaticContent) -> None:
```

value
Kind: `Parameter`

```
_OptionalStaticContent
```

placement
Kind: `Getter`

```
def placement(self) -> _PlacementValue:
```

placement
Kind: `Setter`

```
def placement(self, value: _PlacementValue) -> None:
```

value
Kind: `Parameter`

```
_PlacementValue
```

shift
Kind: `Getter`

```
def shift(self) -> _DrawingPoint:
```

shift
Kind: `Setter`

```
def shift(self, value: _DrawingPoint) -> None:
```

value
Kind: `Parameter`

```
_DrawingPoint
```

rank
Kind: Getter

```
def rank(self) -> _OptionalInteger:
```

rank
Kind: Setter

```
def rank(self, value: _OptionalInteger) -> None:
```

value
Kind: Parameter

```
_OptionalInteger
```

minimum_size
Kind: Getter

```
def minimum_size(self) -> _OptionalNumber:
```

minimum_size
Kind: Setter

```
def minimum_size(self, value: _OptionalNumber) -> None:
```

value
Kind: Parameter

```
_OptionalNumber
```

maximum_size
Kind: Getter

```
def maximum_size(self) -> _OptionalNumber:
```

maximum_size
Kind: Setter

```
def maximum_size(self, value: _OptionalNumber) -> None:
```

value
Kind: Parameter

```
_OptionalNumber
```

style
Kind: Getter

```
def style(self) -> _OptionalStyle:
```

style
Kind: Setter

```
def style(self, value: _OptionalStyle) -> None:
```

value
Kind: Parameter

```
_OptionalStyle
```

label_style

Kind: Getter

```
def label_style(self) -> _OptionalStyle:
```

label_style

Kind: Setter

```
def label_style(self, value: _OptionalStyle) -> None:
```

value

Kind: Parameter

```
_OptionalStyle
```

extensions

Kind: Getter

```
def extensions(self) -> dict[str, _NativeValue]:
```

extensions

Kind: Setter

```
def extensions(self, value: dict[str, _NativeValue]) -> None:
```

value

Kind: Parameter

```
dict[str, _NativeValue]
```

__repr__

Kind: Method

```
def __repr__(self) -> str:
```

__new__

Kind: AssociatedFunction

```
def __new__(cls, *, label: _OptionalStaticContent = ..., placement: _PlacementValue
= ..., shift: _DrawingPoint = ..., rank: _OptionalInteger = ..., minimum_size:
_OptionalNumber = ..., maximum_size: _OptionalNumber = ..., style: _OptionalStyle
= ..., label_style: _OptionalStyle = ..., extensions: _NativeDict = ...) ->
NodeDrawing:
```

label

Kind: Parameter

```
_OptionalStaticContent
```

Default: `...`

placement

Kind: Parameter

```
_PlacementValue
```

Default: `...`

shift

Kind: Parameter

_DrawingPoint

Default: `...`

`rank`
Kind: Parameter

_OptionalInteger

Default: `...`

`minimum_size`
Kind: Parameter

_OptionalNumber

Default: `...`

`maximum_size`
Kind: Parameter

_OptionalNumber

Default: `...`

`style`
Kind: Parameter

_OptionalStyle

Default: `...`

`label_style`
Kind: Parameter

_OptionalStyle

Default: `...`

`extensions`
Kind: Parameter

_NativeDict

Default: `...`

Exhaustive reference · Source

`NodeValue`
Detached node data and drawing metadata passed across a `DotCodec` boundary.

Detached node data and drawing metadata passed across a `DotCodec` boundary.

class NodeValue:

Requires features: `extension-module, abi3`

`data`
Kind: Getter

def data(self) -> Any:

data

Kind: `Setter`

```
def data(self, value: Any) -> None:
```

value

Kind: `Parameter`

```
Any
```

drawing

Kind: `Getter`

```
def drawing(self) -> NodeDrawing:
```

__new__

Kind: `AssociatedFunction`

```
def __new__(cls, *, data: Any = None, drawing: Optional[NodeDrawing] = None) ->
NodeValue:
```

data

Kind: `Parameter`

```
Any
```

Default: `None`

drawing

Kind: `Parameter`

```
Optional[NodeDrawing]
```

Default: `None`

Exhaustive reference · Source

Pattern

```
Kurvst path decoration pattern.
```

```
Kurvst path decoration pattern.
```

```
class Pattern(enum.Enum):
```

Requires features: `extension-module, abi3`

Wave

Kind: `Variant`

Zigzag

Kind: `Variant`

Coil

Kind: `Variant`

Normal

Kind: `Variant`

Curve

Kind: `Variant`

Exhaustive reference · Source

`Placement`

`Coordinate-constraint mode for an explicit placement.`

`Coordinate-constraint mode for an explicit placement.`

`class Placement(enum.Enum):`

Requires features: `extension-module, abi3`

`Pin`

Kind: `Variant`

`Start`

Kind: `Variant`

Exhaustive reference · Source

`RankAlignment`

`Alignment within a layered-layout rank.`

`Alignment within a layered-layout rank.`

`class RankAlignment(enum.Enum):`

Requires features: `extension-module, abi3`

`Center`

Kind: `Variant`

`Start`

Kind: `Variant`

`Left`

Kind: `Variant`

`End`

Kind: `Variant`

`Right`

Kind: `Variant`

Exhaustive reference · Source

`RoutePoints`

`Whether layout-provided route points participate in edge routing.`

`Whether layout-provided route points participate in edge routing.`

`class RoutePoints(enum.Enum):`

Requires features: `extension-module, abi3`

`Ignore`

Kind: `Variant`

`Through`

Kind: `Variant`

Exhaustive reference · Source

Routing

Route construction used for an anchored edge.

Route construction used for an anchored edge.

class Routing(enum.Enum):

Requires features: extension-module, abi3

EdgePosition

Kind: Variant

Direct

Kind: Variant

HobbyThrough

Kind: Variant

StraightThrough

Kind: Variant

Exhaustive reference · Source

StrokeCap

Endpoint shape for a Typst stroke.

Endpoint shape for a Typst stroke.

class StrokeCap(enum.Enum):

Requires features: extension-module, abi3

Butt

Kind: Variant

Round

Kind: Variant

Square

Kind: Variant

Exhaustive reference · Source

StrokeJoin

Join shape for a Typst stroke.

Join shape for a Typst stroke.

class StrokeJoin(enum.Enum):

Requires features: extension-module, abi3

Miter

Kind: Variant

Round

Kind: Variant

Bevel

Kind: Variant

Exhaustive reference · Source

TextStyle

Finite Typst text slant.

Finite Typst text slant.

```
class TextStyle(enum.Enum):
```

Requires features: `extension-module, abi3`

Normal

Kind: Variant

Italic

Kind: Variant

Oblique

Kind: Variant

Exhaustive reference · Source

Ratio

A Typst ratio expressed in percent.

A Typst ratio expressed in percent.

```
class Ratio:
```

Requires features: `extension-module, abi3`

percent

Kind: Getter

```
def percent(self) -> float:
```

__new__

Kind: AssociatedFunction

```
def __new__(cls, percent: float) -> Ratio:
```

percent

Kind: Parameter

```
float
```

from_fraction

Kind: AssociatedFunction

```
def from_fraction(value: float) -> Ratio:
```

value

Kind: Parameter

```
float
```

__repr__

Kind: Method

```
def __repr__(self) -> str:
```

Exhaustive reference · Source

RelativeLength

A sum of a Typst ratio and length.

A sum of a Typst ratio and length.

```
class RelativeLength:
```

Requires features: `extension-module, abi3`

__new__

Kind: `AssociatedFunction`

```
def __new__(cls, ratio: Optional[Ratio] = None, length: Optional[Length] = None) ->
RelativeLength:
```

ratio

Kind: `Parameter`

```
Optional[Ratio]
```

Default: `None`

length

Kind: `Parameter`

```
Optional[Length]
```

Default: `None`

__repr__

Kind: `Method`

```
def __repr__(self) -> str:
```

Exhaustive reference · Source

RenderConfig

Complete typed rendering configuration.

Complete typed rendering configuration.

```
class RenderConfig:
```

Requires features: `extension-module, abi3`

template

Kind: `Getter`

```
def template(self) -> _TemplatePath:
```

template

Kind: `Setter`

```
def template(self, value: _TemplatePath) -> None:
```

value

Kind: `Parameter`

```
_TemplatePath
```

source_root

Kind: `Getter`

```
def source_root(self) -> _SourceRootPath:
```

source_root
Kind: Setter

```
def source_root(self, value: _SourceRootPath) -> None:
```

value
Kind: Parameter

```
_SourceRootPath
```

title
Kind: Getter

```
def title(self) -> _AutoOptionalStaticContent:
```

title
Kind: Setter

```
def title(self, value: _AutoOptionalStaticContent) -> None:
```

value
Kind: Parameter

```
_AutoOptionalStaticContent
```

style
Kind: Getter

```
def style(self) -> _RenderStyle:
```

style
Kind: Setter

```
def style(self, value: _RenderStyle) -> None:
```

value
Kind: Parameter

```
_RenderStyle
```

layouts
Kind: Getter

```
def layouts(self) -> _RenderLayouts:
```

layouts
Kind: Setter

```
def layouts(self, value: _RenderLayouts) -> None:
```

value
Kind: Parameter

```
_RenderLayouts
```

drawing
Kind: Getter

```
def drawing(self) -> _RenderDrawing:
```

drawing

Kind: Setter

```
def drawing(self, value: _RenderDrawing) -> None:
```

value

Kind: Parameter

```
_RenderDrawing
```

selectors

Kind: Getter

```
def selectors(self) -> _RenderSelectors:
```

selectors

Kind: Setter

```
def selectors(self, value: _RenderSelectors) -> None:
```

value

Kind: Parameter

```
_RenderSelectors
```

template_options

Kind: Getter

```
def template_options(self) -> _TemplateOptions:
```

template_options

Kind: Setter

```
def template_options(self, value: _TemplateOptions) -> None:
```

value

Kind: Parameter

```
_TemplateOptions
```

overlay

Kind: Method

```
def overlay(self, overlay: RenderConfig) -> RenderConfig:
```

overlay

Kind: Parameter

```
RenderConfig
```

__repr__

Kind: Method

```
def __repr__(self) -> str:
```

__new__

Kind: AssociatedFunction

```
def __new__(cls, *, template: _TemplatePath = ..., source_root: _SourceRootPath
= ..., title: _AutoOptionalStaticContent = ..., style: _RenderStyle = ..., layouts:
_RenderLayouts = ..., drawing: _RenderDrawing = ..., selectors: _RenderSelectors
= ..., template_options: _TemplateOptions = ...) -> RenderConfig:
```

`template`

Kind: `Parameter`

`_TemplatePath`

Default: `...`

`source_root`

Kind: `Parameter`

`_SourceRootPath`

Default: `...`

`title`

Kind: `Parameter`

`_AutoOptionalStaticContent`

Default: `...`

`style`

Kind: `Parameter`

`_RenderStyle`

Default: `...`

`layouts`

Kind: `Parameter`

`_RenderLayouts`

Default: `...`

`drawing`

Kind: `Parameter`

`_RenderDrawing`

Default: `...`

`selectors`

Kind: `Parameter`

`_RenderSelectors`

Default: `...`

`template_options`

Kind: `Parameter`

`_TemplateOptions`

Default: `...`

Exhaustive reference · Source

Stroke

A typed Typst stroke dictionary.

A typed Typst stroke dictionary.

```
class Stroke:
```

Requires features: `extension-module, abi3`

__repr__

Kind: Method

```
def __repr__(self) -> str:
```

__new__

Kind: AssociatedFunction

```
def __new__(cls, *, paint: _OptionalPaint = ..., thickness: _OptionalLengthValue
= ..., cap: _OptionalStrokeCap = ..., join: _OptionalStrokeJoin = ..., dash:
_OptionalDashValue = ..., miter_limit: _Number = ...) -> Stroke:
```

paint

Kind: Parameter

```
_OptionalPaint
```

Default: `...`

thickness

Kind: Parameter

```
_OptionalLengthValue
```

Default: `...`

cap

Kind: Parameter

```
_OptionalStrokeCap
```

Default: `...`

join

Kind: Parameter

```
_OptionalStrokeJoin
```

Default: `...`

dash

Kind: Parameter

```
_OptionalDashValue
```

Default: `...`

miter_limit

Kind: Parameter

```
_Number
```

Default: `...`

Exhaustive reference · Source

Subgraph

A graph-bound structural selection.

A graph-bound structural selection.

```
class Subgraph:
```

Requires features: `extension-module, abi3`

graph

Kind: Getter

```
def graph(self) -> Graph:
```

revision

Kind: Getter

```
def revision(self) -> int:
```

size

Kind: Getter

```
def size(self) -> int:
```

n_half_edges

Kind: Getter

```
def n_half_edges(self) -> int:
```

n_isolated_nodes

Kind: Getter

```
def n_isolated_nodes(self) -> int:
```

__len__

Kind: Method

```
def __len__(self) -> int:
```

__bool__

Kind: Method

```
def __bool__(self) -> bool:
```

includes

Kind: Method

```
def includes(self, half_edge: int) -> bool:
```

half_edge

Kind: Parameter

```
int
```

half_edge_indices

Kind: Method

```
def half_edge_indices(self) -> list[int]:
```

Return stable indices suitable for constructing another selection on this revision.

isolated_node_indices

Kind: Method

```
def isolated_node_indices(self) -> list[int]:
```

Return the explicitly selected zero-crown node indices.

includes_node

Kind: Method

```
def includes_node(self, node: int) -> bool:
```

node

Kind: Parameter

```
int
```

to_half_edges

Kind: Method

```
def to_half_edges(self) -> list[HalfEdge]:
```

is_subset

Kind: Method

```
def is_subset(self, other: Subgraph) -> bool:
```

other

Kind: Parameter

```
Subgraph
```

is_superset

Kind: Method

```
def is_superset(self, other: Subgraph) -> bool:
```

other

Kind: Parameter

```
Subgraph
```

is_disjoint

Kind: Method

```
def is_disjoint(self, other: Subgraph) -> bool:
```

other

Kind: Parameter

```
Subgraph
```

union

Kind: Method

```
def union(self, other: Subgraph) -> Subgraph:
```

other

Kind: Parameter

```
Subgraph
```

intersection
Kind: Method

```
def intersection(self, other: Subgraph) -> Subgraph:
```

other
Kind: Parameter

Subgraph

symmetric_difference
Kind: Method

```
def symmetric_difference(self, other: Subgraph) -> Subgraph:
```

other
Kind: Parameter

Subgraph

difference
Kind: Method

```
def difference(self, other: Subgraph) -> Subgraph:
```

other
Kind: Parameter

Subgraph

complement
Kind: Method

```
def complement(self) -> Subgraph:
```

__contains__
Kind: Method

```
def __contains__(self, half_edge: int) -> bool:
```

half_edge
Kind: Parameter

int

__or__
Kind: Method

```
def __or__(self, other: Subgraph) -> Subgraph:
```

other
Kind: Parameter

Subgraph

__and__
Kind: Method

```
def __and__(self, other: Subgraph) -> Subgraph:
```

other
Kind: Parameter

Subgraph

__xor__
Kind: Method

```
def __xor__(self, other: Subgraph) -> Subgraph:
```

other
Kind: Parameter

Subgraph

__sub__
Kind: Method

```
def __sub__(self, other: Subgraph) -> Subgraph:
```

other
Kind: Parameter

Subgraph

__invert__
Kind: Method

```
def __invert__(self) -> Subgraph:
```

__eq__
Kind: Method

```
def __eq__(self, other: Subgraph) -> bool:
```

other
Kind: Parameter

Subgraph

__le__
Kind: Method

```
def __le__(self, other: Subgraph) -> bool:
```

other
Kind: Parameter

Subgraph

__lt__
Kind: Method

```
def __lt__(self, other: Subgraph) -> bool:
```

other
Kind: Parameter

Subgraph

__ge__
Kind: Method

```
def __ge__(self, other: Subgraph) -> bool:
```

other
Kind: Parameter

Subgraph

__gt__
Kind: Method

```
def __gt__(self, other: Subgraph) -> bool:
```

other
Kind: Parameter

Subgraph

__repr__
Kind: Method

```
def __repr__(self) -> str:
```

Exhaustive reference · Source

TextLabel
Literal text content with optional typed text styling.

Literal text content with optional typed text styling.

```
class TextLabel:
```

Requires features: `extension-module, abi3`

text
Kind: Getter

```
def text(self) -> str:
```

__repr__
Kind: Method

```
def __repr__(self) -> str:
```

__new__
Kind: AssociatedFunction

```
def __new__(cls, text: str, *, fill: _Paint = ..., size: _LengthValue = ..., weight:
_NativeValue = ..., style: _TextStyleValue = ...) -> TextLabel:
```

text
Kind: Parameter

str

fill
Kind: Parameter

_Paint

Default: `...`

`size`
Kind: `Parameter`

`_LengthValue`

Default: `...`

`weight`
Kind: `Parameter`

`_NativeValue`

Default: `...`

`style`
Kind: `Parameter`

`_TextStyleValue`

Default: `...`

Exhaustive reference · Source

`TraversalTree`
`A graph-bound DFS or BFS traversal tree.`

`A graph-bound DFS or BFS traversal tree.`

`class TraversalTree:`

Requires features: `extension-module, abi3`

`subgraph`
Kind: `Getter`

`def subgraph(self) -> Subgraph:`

`nodes`
Kind: `Getter`

`def nodes(self) -> list[Node]:`

`half_edges`
Kind: `Getter`

`def half_edges(self) -> list[HalfEdge]:`

`covers`
Kind: `Method`

`def covers(self, subgraph: Subgraph) -> Subgraph:`

`subgraph`
Kind: `Parameter`

`Subgraph`

`parent`
Kind: `Method`

```
def parent(self, node: Node | int | str) -> Optional[Node]:
```

Return the immediate parent, or `None` for the traversal root.

node
Kind: Parameter

```
Node | int | str
```

children
Kind: Method

```
def children(self, node: Node | int | str) -> list[Node]:
```

Return the immediate children in traversal discovery order.

node
Kind: Parameter

```
Node | int | str
```

ancestors
Kind: Method

```
def ancestors(self, node: Node | int | str) -> list[Node]:
```

Return strict ancestors from the immediate parent through the root.

node
Kind: Parameter

```
Node | int | str
```

fundamental_cycle
Kind: Method

```
def fundamental_cycle(self, half_edge: _HalfEdgeTarget) -> Optional[Cycle]:
```

Return the fundamental cycle closed by an internal edge, or `None` for a tree edge.

half_edge
Kind: Parameter

```
_HalfEdgeTarget
```

Exhaustive reference · Source

TypstBind
A Typst function partially applied through its native `.with` method.

A Typst function partially applied through its native `.with` method.

```
class TypstBind:
```

Requires features: `extension-module, abi3`

call
Kind: Method

```
def call(self, *args: _NativeValue, **kwargs: _NativeValue) -> TypstCall:
```

args
Kind: Parameter

`_NativeValue`

`kwargs`
Kind: `Parameter`

`_NativeValue`

`__repr__`
Kind: `Method`

`def __repr__(self) -> str:`

Exhaustive reference · Source

`TypstCall`
`A call to an explicitly imported Typst function.`

`A call to an explicitly imported Typst function.`

`class TypstCall:`

Requires features: `extension-module, abi3`

`__repr__`
Kind: `Method`

`def __repr__(self) -> str:`

Exhaustive reference · Source

`TypstModule`
`A local or package Typst module whose exports can be referenced safely.`

`A local or package Typst module whose exports can be referenced safely.`

`class TypstModule:`

Requires features: `extension-module, abi3`

`file`
Kind: `AssociatedFunction`

`def file(path: str | os.PathLike[str]) -> TypstModule:`

`path`
Kind: `Parameter`

`str | os.PathLike[str]`

`package`
Kind: `AssociatedFunction`

`def package(specification: str) -> TypstModule:`

`specification`
Kind: `Parameter`

`str`

`value`
Kind: `Method`

`def value(self, export: str) -> _TypstValueRef:`

export

Kind: `Parameter`

`str`

content

Kind: `Method`

```
def content(self, export: str) -> _TypstContentRef:
```

export

Kind: `Parameter`

`str`

function

Kind: `Method`

```
def function(self, export: str) -> _TypstFunctionRef:
```

export

Kind: `Parameter`

`str`

__repr__

Kind: `Method`

```
def __repr__(self) -> str:
```

Exhaustive reference · Source

TypstRef

`A typed export from a Typst module.`

`A typed export from a Typst module.`

```
class TypstRef:
```

Requires features: `extension-module, abi3`

call

Kind: `Method`

```
def call(self, *args: _NativeValue, **kwargs: _NativeValue) -> TypstCall:
```

args

Kind: `Parameter`

`_NativeValue`

kwargs

Kind: `Parameter`

`_NativeValue`

bind

Kind: `Method`

```
def bind(self, *args: _NativeValue, **kwargs: _NativeValue) -> TypstBind:
```

`args`

Kind: `Parameter`

`_NativeValue`

`kwargs`

Kind: `Parameter`

`_NativeValue`

`__repr__`

Kind: `Method`

```
def __repr__(self) -> str:
```

Exhaustive reference · Source

`build`

`Build a graph from declarative node and edge specs.`

`Build a graph from declarative node and edge specs.`

```
def build(*items: _GraphItem, name: _OptionalString = None, global_data:
_OptionalGlobalData = None, codec: _OptionalDotCodec = None, render_config:
_OptionalRenderConfig = None, node_store: NodeStore = NodeStore.Vec) -> Graph:
```

Requires features: `extension-module, abi3`

Exhaustive reference · Source

`edge`

`Describe an edge from one or two endpoint specs.`

`Describe an edge from one or two endpoint specs.`

```
def edge(first: HalfEdgeSpec, name: _OptionalString = None, second:
_OptionalHalfEdgeSpec = None, *, data: Any = None, orientation: Orientation =
Orientation.Default, label: _OptionalStaticContent = ..., placement: _PlacementValue
= ..., label_position: _DrawingPoint = ..., label_offset: _OptionalNumber = ...,
label_angle: _DrawingAngle = ..., bend: _DrawingAngle = ..., routing: _RoutingValue
= ..., minimum_length: _OptionalInteger = ..., same_rank: _OptionalBoolean = ...,
style: _OptionalStyleLayers = ..., label_style: _OptionalStyle = ..., decoration:
_DrawingDecoration = ..., extensions: _NativeDict = ...) -> EdgeSpec:
```

Requires features: `extension-module, abi3`

Exhaustive reference · Source

`node`

`Describe a node while preserving its arbitrary Python data by identity.`

`Describe a node while preserving its arbitrary Python data by identity.`

```
def node(name: _OptionalString = None, *, data: Any = None, label:
_OptionalStaticContent = ..., placement: _PlacementValue = ..., shift: _DrawingPoint
= ..., rank: _OptionalInteger = ..., minimum_size: _OptionalNumber = ...,
maximum_size: _OptionalNumber = ..., style: _OptionalStyle = ..., label_style:
_OptionalStyle = ..., extensions: _NativeDict = ...) -> NodeSpec:
```

Requires features: `extension-module, abi3`

Exhaustive reference · Source

sink

Attach a sink endpoint.

Attach a sink endpoint. Build resolves specs, names, indices, and live-node keys; incremental insertion resolves current graph references.

```
def sink(node: _EndpointTarget, *, data: Any = None, label: _OptionalStaticContent
= ..., statement: _DrawingString = ..., port_label: _DrawingString = ..., compass:
_CompassValue = ..., anchor: _AnchorValue = ..., routing: _RoutingValue = ..., style:
_OptionalStyleLayers = ..., extensions: _NativeDict = ...) -> HalfEdgeSpec:
```

Requires features: `extension-module, abi3`

Exhaustive reference · Source

source

Attach a source endpoint.

Attach a source endpoint. Build resolves specs, names, indices, and live-node keys; incremental insertion resolves current graph references.

```
def source(node: _EndpointTarget, *, data: Any = None, label: _OptionalStaticContent
= ..., statement: _DrawingString = ..., port_label: _DrawingString = ..., compass:
_CompassValue = ..., anchor: _AnchorValue = ..., routing: _RoutingValue = ..., style:
_OptionalStyleLayers = ..., extensions: _NativeDict = ...) -> HalfEdgeSpec:
```

Requires features: `extension-module, abi3`

Exhaustive reference · Source

Version information

Save these identifiers with published results and include them in issue reports so that collaborators can reproduce the same software environment.

- **Documentation channel:** `latest`
- **Documentation route:** `/products/linnet/latest/`
- **Source revision:** `8931c5ab5b71a2d517b9069cb0e9ae68ebd1c13e`

Package versions and enabled features

- `gammaloop/gammalooprs` — `0.3.4` (`no optional features`)
- `gammaloop/gammaloop-api` — `0.1.0` (`features: cli`)
- `gammaloop/gammaloop` — `0.3.4` (`features: python, abi3, python-stubs`)
- `linnet/linnet` — `0.17.0` (`features: native, serde, bincode, drawing, symbolica`)
- `linnet/linnet-py` — `0.1.0` (`features: extension-module, abi3`)
- `spenso/spenso` — `0.6.0` (`features: native, symbolica`)
- `spenso/spenso-macros` — `0.3.2` (`no optional features`)
- `spenso/spenso-hep-lib` — `0.1.7` (`features: native`)
- `spenso/spynso3` — `0.1.2` (`features: python-stubs`)
- `idenso/idenso` — `0.3.0` (`features: native, reference-cases`)
- `idenso/idenso` — `0.3.0` (`features: python, python-stubs`)
- `vakint/vakint` — `0.1.2` (`no optional features`)
- `vakint/vakint` — `0.1.2` (`features: python, python-stubs`)